

1. Determine if 2 strings are anagrams of each other

[python code]

```
string = "1a23"
string1 = "".join(char for char in string if char.isalpha()).lower()
string = "45a6"
string2 = "".join(char for char in string if char.isalpha()).lower()
list1 = list(string1)
list2 = list(string2)
if (len(list1) == 0) or (len(list2) == 0) or (len(list1) != len(list2)):
    print("string1 and string 2 are Not anagrams of each other")
    quit()
list1.sort()
list2.sort()

while len(list1) > 0:
    x = list1.pop()
    y = list2.pop()
    if (x != y):
        print("string 1 and string 2 are Not anagrams of each other")
        quit()
print("string 1 and string 2 Are anagrams of each other")
```

[pseudocode]

```
string <- STRING(1a23)
string1 <- EMPTY STRING
FOR EACH char IN string DO
    IF char IS A LETTER THEN
        string1 <- string1 + LOWERCASE(char)
    ENDIF
ENDFOR

string <- STRING(45a6)
string2 <- EMPTY STRING
FOR EACH char IN string DO
    IF char IS A LETTER THEN
        string2 <- string2 + LOWERCASE(char)
    ENDIF
ENDFOR

list1 <- LIST OF CHARACTERS IN string1
list2 <- LIST OF CHARACTERS IN string2

IF (LENGTH(list1) = 0) OR (LENGTH(list2) = 0) OR (LENGTH(list1) != LENGTH(list2)) THEN
    OUTPUT STRING(string1 and string2 are Not anagrams of each other)
    STOP
ENDIF

list1 <- SORT(list1)
list2 <- SORT(list2)

WHILE LENGTH(list1) > 0 DO
    x <- VALUE(REMOVE LAST ITEM FROM list1)
```

```

y <- VALUE(REMOVE LAST ITEM FROM list2)

IF x != y THEN
    OUTPUT STRING(string1 and string2 are Not anagrams of each other)
    STOP
ENDIF
ENDWHILE

```

2. Convert a non negative integer into its binary representation

[python code]

```

num = int(23)
numCopy = num
byteString = []
bits = int(0)

while (2 ** bits) <= num:
    bits += 1

if bits < 7:
    bits = 7

while bits >= 0:
    value = 2 ** bits
    if value > num:
        byteString.append(0)
    else:
        byteString.append(1)
        num -= value
    bits -= 1

if byteString[0] == 0 and len(byteString) > 8:
    byteString.pop(0)

bitString = "".join(map(str, byteString))

print(numCopy, "= 0b", bitString)

```

[pseudocode]

```

num <- INTEGER(23)
numCopy <- num
byteString <- EMPTY LIST
bits <- INTEGER(0)

WHILE (2 EXP bits) <= num DO
    bits <- bits + 1
ENDWHILE

IF bits < 7 THEN
    bits <- 7

```

ENDIF

WHILE bits >= 0 DO
 value <- 2 EXP bits

 IF value > num THEN
 APPEND 0 TO byteString
 ELSE
 APPEND 1 TO byteString
 num <- num - value
 ENDIF

 bits <- bits - 1
ENDWHILE

IF (byteString[0] = 0) AND (LENGTH(byteString) > 8) THEN
 REMOVE FIRST ITEM FROM byteString
ENDIF

bitString <- JOIN(byteString, EMPTY STRING)

OUTPUT STRING({numCopy} = 0b {bitString})

3. Determine if a year satisfies the leap year rules

[python code]

```
leap = False
loop = True
while loop == True:
    year = input("Enter Year: ")
    try:
        year = int(year);
        loop = False
    except:
        print("That's Not a Valid Year, Please Try Again!")
```

```
if year % 4 == 0:
    leap = True
else:
    leap = False
```

```
if year % 100 == 0 and not year % 400 == 0:
    leap = False
elif year % 400 == 0:
    leap = True
```

```
if not leap:
    print(f"{year} is Not a Leap Year!")
else:
```

```
print(f"{year} Is a Leap Year!")
```

[pseudocode]

```
leap <- False
```

```
loop <- True
```

```
WHILE loop = True DO
```

```
    year <- INPUT(STRING(Enter Year: ))
```

```
    TRY
```

```
        year <- INTEGER(year)
```

```
        loop <- False
```

```
    CATCH
```

```
        OUTPUT STRING(That's Not a Valid Year, Please Try Again!)
```

```
    ENDTRY
```

```
ENDWHILE
```

```
IF year MOD 4 = 0 THEN
```

```
    leap <- True
```

```
ELSE
```

```
    leap <- False
```

```
ENDIF
```

```
IF (year MOD 100 = 0) AND (NOT (year MOD 400 = 0)) THEN
```

```
    leap <- False
```

```
ELSEIF year MOD 400 = 0 THEN
```

```
    leap <- True
```

```
ENDIF
```

```
IF NOT leap THEN
```

```
    OUTPUT STRING({year} is Not a Leap Year!)
```

```
ELSE
```

```
    OUTPUT STRING({year} Is a Leap Year!"
```

```
ENDIF
```

4. Find the intersection of 2 sorted arrays and return the common elements

[python code]

```
originalset1 = ['a','b','c','d','e',1]
```

```
originalset2 = ['b','c','g',1]
```

```
set1 = [str(x) for x in originalset1]
```

```
set2 = [str(x) for x in originalset2]
```

```
set1.sort()
```

```
set2.sort()
```

```
unionset = []
```

```
len1 = len(set1)
```

```
len2 = len(set2)
```

```
listPointer1 = ""
```

```

listPointer2 = ""
if len1 <= len2:
    listPointer1 = set1
    listPointer2 = set2
else:
    listPointer1 = set2
    listPointer2 = set1
while len(listPointer1) > 0:
    element = listPointer1.pop(0)
    if element in listPointer2:
        unionset.append(element)
print(f"The Common elements from both sets are:\n", unionset)

```

[pseudocode]

```

originalset1 <- LIST OF [STRING(a), STRING(b), STRING(c), STRING(d), STRING(e), INTEGER(1)]
originalset2 <- LIST OF [STRING(b), STRING(c), STRING(g), INTEGER(1)]

```

```

set1 <- EMPTY LIST
FOR EACH x IN originalset1 DO
    APPEND STRING(x) TO set1
ENDFOR

```

```

set2 <- EMPTY LIST
FOR EACH x IN originalset2 DO
    APPEND STRING(x) TO set2
ENDFOR

```

```

set1 <- SORT(set1)
set2 <- SORT(set2)

```

```

unionset <- EMPTY LIST

```

```

len1 <- LENGTH(set1)
len2 <- LENGTH(set2)

```

```

listPointer1 <- EMPTY STRING
listPointer2 <- EMPTY STRING

```

```

IF len1 <= len2 THEN
    listPointer1 <- set1
    listPointer2 <- set2
ELSE
    listPointer1 <- set2
    listPointer2 <- set1
ENDIF

```

```

WHILE LENGTH(listPointer1) > 0 DO
    element <- VALUE(REMOVE FIRST ITEM FROM listPointer1)

```

```

    IF element IN listPointer2 THEN
        APPEND element TO unionset
    ENDIF
ENDWHILE

```

```

OUTPUT STRING(The Common elements from both sets are:\n {unionset})

```

5. Remove punctuation from a string

[python code]

```

import string
string1 = "This /_@!^*%<>is &*;:'a sentence, that has punctuation!"
string2 = list(string1)

x = ["".join(char for char in s if char not in string.punctuation) for s in string2]

final = "".join(x)
print(final)

```

[pseudocode]

```

IMPORT string

string1 <- STRING(This /_@!^*%<>is &*;:'a sentence, that has punctuation!)
string2 <- LIST OF CHARACTERS IN string1

x <- EMPTY LIST

FOR EACH s IN string2 DO
    temp <- EMPTY STRING

    FOR EACH char IN s DO
        IF char NOT IN string.punctuation THEN
            temp <- temp + char
        ENDIF
    ENDFOR

    APPEND temp TO x
ENDFOR

final <- EMPTY STRING
final <- final + JOIN(x, EMPTY STRING)

OUTPUT STRING({final})

```

6. Find the length of a given string

[python code]

```
string1 = "This /_@!^*%<>is &*;:'a sentence, that has unwanted punctuation!"
string2 = list(string1)
flag = True
count = 0
while flag:
    try:
        element = string2.pop(0)
        count += 1
    except:
        flag = False
print(f"String Length = {count} Characters")
```

[pseudocode]

```
string1 <- STRING(This /_@!^*%<>is &*;:'a sentence, that has unwanted punctuation!)
string2 <- LIST OF CHARACTERS IN string1

flag <- BOOLEAN(True)
count <- 0

WHILE flag = BOOLEAN(True) DO
    TRY
        element <- VALUE(REMOVE FIRST ITEM FROM string2)
        count <- count + 1
    CATCH
        flag <- BOOLEAN(False)
    ENDTRY
ENDWHILE

OUTPUT STRING(String Length = {count} Characters)
```

7. Split a sentence into individual words and store in a list

[python code]

```
import string
string1 = "This is a sentence!"
string2 = list(string1)
x = ["".join(char for char in s if char not in string.punctuation) for s in string2]
word = ""
words = []
flag = True
delimiter = " "
while flag:
    try:
        element = x.pop(0)
        if element != delimiter:
```

```

        word += element
    else:
        words.append(word)
        word = ""
    except:
        words.append(word)
        flag = False

print(words)

```

[pseudocode]

```

IMPORT string

string1 <- STRING(This is a sentence!)
string2 <- LIST OF CHARACTERS IN string1

x <- EMPTY LIST

FOR EACH s IN string2 DO
    temp <- EMPTY STRING

    FOR EACH char IN s DO
        IF char NOT IN string.punctuation THEN
            temp <- temp + char
        ENDIF
    ENDFOR

    APPEND temp TO x
ENDFOR

word <- EMPTY STRING
words <- EMPTY LIST
flag <- BOOLEAN(True)
delimiter <- STRING( )

WHILE flag = BOOLEAN(True) DO
    TRY
        element <- VALUE(REMOVE FIRST ITEM FROM x)

        IF element != delimiter THEN
            word <- word + element
        ELSE
            APPEND word TO words
            word <- EMPTY STRING
        ENDIF

    CATCH
        APPEND word TO words

```



```
    flag <- BOOLEAN(False)
ENDTRY
ENDWHILE
```

```
OUTPUT STRING({words})
```

8. Sort a list of words into alphabetical order

[python code]

```
words = ["hello", "again", "similar", "simulate", "halo", "sibner"]
n = len(words) - 1
for i in range(n):
    for j in range(n - i):
        if words[j] > words[j + 1]:
            words[j], words[j + 1] = words[j + 1], words[j]
print(words)
```

[pseudocode]

```
words <- LIST OF [STRING(hello), STRING(again), STRING(similar), STRING(simulate), STRING(halo), STRING(sibner)]
```

```
n <- LENGTH OF words
```

```
FOR i <- IN RANGE 0 TO n DO
```

```
    FOR j <- IN RANGE 0 TO (n - i) DO
```

```
        IF words[j] > words[j + 1] THEN
```

```
            temp <- words[j]
```

```
            words[j] <- words[j + 1]
```

```
            words[j + 1] <- temp
```

```
        ENDIF
```

```
    ENDFOR
```

```
ENDFOR
```

```
OUTPUT STRING({words})
```

9. Given an unsorted list of integers and a target value \$t. Find the exact position of \$t in the list or report that it doesn't exist

[python code]

```
list1 = [5,8,1,9,3,2,4,7,6]
```

```
target = 9
```

```
index = 0
```

```
flag = ""
```

```
for i in range(len(list1)):
```

```
    if list1[i] == target:
```

```
        index = i
```

```
        flag = True
```

```

        break
    else:
        flag = False
if flag == True:
    print(f"index of target is {index}")
else:
    print(f"{target} Is Not in the list!")

```

[pseudocode]

```

list1 <- LIST OF [5, 8, 1, 9, 3, 2, 4, 7, 6]
target <- 9
index <- 0
flag <- EMPTY STRING

```

```

FOR i <- IN RANGE 0 TO LENGTH OF list1 DO

```

```

    IF list1[i] == target THEN
        index <- i
        flag <- True
        BREAK
    ELSE
        flag <- False
    ENDIF

```

```

ENDFOR

```

```

IF flag == True THEN
    OUTPUT STRING(index of target is {index})
ELSE
    OUTPUT STRING({target} Is Not in the list!)
ENDIF

```

10. Scan a given paragraph of text and count how many times each individual vowel appears

[python code]

```

vowel = ["a", "e", "i", "o", "u"]
text = "This is a sentence that contains vowels"
a, e, i, o, u = 0,0,0,0,0
textlist = list(map(lambda x: x.lower(), text))

```

```

for b in range(len(textlist)):
    for j in range(len(vowel)):
        if vowel[j] == textlist[b]:
            if j == 0:
                a += 1
                break
            elif j == 1:
                e +=1

```

```

    break
elif j == 2:
    i += 1
    break
elif j == 3:
    o += 1
    break
elif j == 4:
    u += 1
    break

```

```

print(f"The text contains the vowel a {a} times")
print(f"The text contains the vowel e {e} times")
print(f"The text contains the vowel i {i} times")
print(f"The text contains the vowel o {o} times")
print(f"The text contains the vowel u {u} times")

```

[pseudocode]

```
vowel <- LIST OF [STRING(a), STRING(e), STRING(i), STRING(o), STRING(u)]
```

```
text <- STRING(This is a sentence that contains vowels)
```

```

a <- 0
e <- 0
i <- 0
o <- 0
u <- 0

```

```
textlist <- LIST OF CHARACTERS IN LOWERCASE(text)
```

```

FOR b <- IN RANGE 0 TO LENGTH OF textlist DO
  FOR j <- IN RANGE 0 TO LENGTH OF vowel DO

```

```
  IF vowel[j] == textlist[b] THEN
```

```

    IF j == 0 THEN
      a <- a + 1
      BREAK

```

```

    ELSEIF j == 1 THEN
      e <- e + 1
      BREAK

```

```

    ELSEIF j == 2 THEN
      i <- i + 1
      BREAK

```

```

ELSEIF j == 3 THEN
    o <- o + 1
    BREAK

ELSEIF j == 4 THEN
    u <- u + 1
    BREAK

ENDIF

ENDIF

ENDFOR
ENDFOR

OUTPUT STRING(The text contains the vowel a {a} times)
OUTPUT STRING(The text contains the vowel e {e} times)
OUTPUT STRING(The text contains the vowel i {i} times)
OUTPUT STRING(The text contains the vowel o {o} times)
OUTPUT STRING(The text contains the vowel u {u} times)

```

11. Given a list of IP addresses that accessed a server today remove all duplicates and return only the unique addresses

[python code]

```

iplist = ["192.168.1.15", "10.0.0.5", "172.16.254.1", "192.168.1.15", "203.0.113.45", "8.8.8.8", "172.16.254.1",
"198.51.100.12", "10.0.0.5", "192.168.0.100", "172.16.1.10", "10.0.0.5", "203.0.113.45", "192.168.1.15", "8.8.4.4",
"172.16.254.1", "192.168.0.254", "10.0.0.12", "192.168.1.100", "8.8.8.8"]
uniqueiplist = []
flag = True
while flag:
    try:
        ip = iplist.pop(0)
        if ip not in uniqueiplist:
            uniqueiplist.append(ip)
    except:
        flag = False

print(f"Unique IP Addresses Are:")
for x in range(len(uniqueiplist)):
    print(uniqueiplist[x])

```

[pseudocode]

```

iplist <- LIST OF [
    STRING(192.168.1.15), STRING(10.0.0.5), STRING(172.16.254.1), STRING(192.168.1.15),
    STRING(203.0.113.45), STRING(8.8.8.8), STRING(172.16.254.1), STRING(198.51.100.12),
    STRING(10.0.0.5), STRING(192.168.0.100), STRING(172.16.1.10), STRING(10.0.0.5),
    STRING(203.0.113.45), STRING(192.168.1.15), STRING(8.8.4.4), STRING(172.16.254.1),

```

```
    STRING(192.168.0.254), STRING(10.0.0.12), STRING(192.168.1.100), STRING(8.8.8.8)
]
```

```
uniqueplist <- EMPTY LIST
flag <- True
```

```
WHILE flag == True DO
    TRY
        ip <- VALUE(REMOVE FIRST ITEM FROM iplist)

        IF ip NOT IN uniqueplist THEN
            APPEND ip TO uniqueplist
        ENDIF

    CATCH
        flag <- False
    ENDTRY
ENDWHILE
```

```
OUTPUT STRING(Unique IP Addresses Are:)
```

```
FOR x <- IN RANGE 0 TO LENGTH OF uniqueplist DO
    OUTPUT STRING({uniqueplist[x]})
ENDFOR
```

12. Given a square grid numbers $n \times n$, calculate the sum of the numbers that lie on the main diagonal top left to bottom right

[python code]

```
import random
rows = random.randint(3, 15)
print(f"{rows} x {rows} Matrix")
x, y = (rows, rows)
matrix = [[[random.randint(0, 99)) for i in range(y)] for j in range(x))]
for t in range(rows):
    print(matrix[t])
```

```
total = 0
counter = 0
string = ""
for a in range(rows):
    total += matrix[counter][counter]
    if string == "":
        string += f"{matrix[counter][counter]} "
    else:
        string += f"+ {matrix[counter][counter]} "
    counter += 1
print(string, f"= {total}")
```

[pseudocode]

rows <- RANDOM INTEGER BETWEEN 3 AND 15

OUTPUT STRING({rows} x {rows} Matrix)

x <- rows

y <- rows

matrix <- EMPTY LIST

FOR j <- IN RANGE 0 TO x DO

 row <- EMPTY LIST

 FOR i <- IN RANGE 0 TO y DO

 value <- RANDOM INTEGER BETWEEN 0 AND 99

 APPEND INTEGER(value) TO row

 ENDFOR

 APPEND row TO matrix

 ENDFOR

FOR t <- IN RANGE 0 TO rows DO

 OUTPUT STRING({matrix[t]})

ENDFOR

total <- 0

counter <- 0

string <- EMPTY STRING

FOR a <- IN RANGE 0 TO rows DO

 total <- total + matrix[counter][counter]

 IF string == EMPTY STRING THEN

 string <- STRING({matrix[counter][counter]})

 ELSE

 string <- STRING({string}+ {matrix[counter][counter]})

 ENDIF

 counter <- counter + 1

ENDFOR

OUTPUT STRING({string} = {total})

13. Reverse a string without using built in reverse functions

[python code]

```
string = "This is a sentence that needs reversing"
stringlist = list(string)
revlist = ""
print(string)
for x in range(len(stringlist)):
    i = stringlist.pop(-1)
    revlist += i
print(revlist)
```

[pseudocode]

```
string <- STRING(This is a sentence that needs reversing)
stringlist <- LIST OF CHARACTERS IN string
```

```
revlist <- EMPTY STRING
```

```
OUTPUT STRING({string})
```

```
FOR x <- IN RANGE 0 TO LENGTH OF stringlist DO
    i <- VALUE( REMOVE LAST ITEM FROM stringlist)
    revlist <- STRING({revlist}{i})
ENDFOR
```

```
OUTPUT STRING({revlist})
```

14. Find the sum of all even numbers in a list

[python code]

```
list = [6,7,9,4,66,8,34,5,2]
total = 0
for i in range(len(list)):
    if list[i] % 2 == 0:
        total += list[i]
    else:
        continue
print(f"The Sum of Even list elements = {total}")
```

[pseudocode]

```
list <- LIST OF [6, 7, 9, 4, 66, 8, 34, 5, 2]
```

```
total <- 0
```

```
FOR i <- IN RANGE 0 TO LENGTH OF list DO
```

```

IF list[j] MOD 2 == 0 THEN
    total <- total + list[i]
ELSE
    CONTINUE
ENDIF

ENDFOR

OUTPUT STRING(The Sum of Even list elements = {total})

```

15. Find the second largest in a list

[python code]

```

numlist = [6,7,9,4,66,8,34,5,2]
print(numlist)
n = len(numlist) - 1
for i in range(n):
    for j in range(n - i):
        if numlist[j] > numlist[j + 1]:
            numlist[j], numlist[j + 1] = numlist[j + 1], numlist[j]
print(f"The Second highest number in the list = {numlist[-2]}")

```

[pseudocode]

```

numlist <- LIST OF [6, 7, 9, 4, 66, 8, 34, 5, 2]

```

```

OUTPUT STRING({numlist})

```

```

n <- LENGTH OF numlist - 1

```

```

FOR i <- IN RANGE 0 TO n DO

```

```

    FOR j <- IN RANGE 0 TO (n - i) DO

```

```

        IF numlist[j] > numlist[j + 1] THEN

```

```

            temp <- numlist[j]

```

```

            numlist[j] <- numlist[j + 1]

```

```

            numlist[j + 1] <- temp

```

```

        ENDIF

```

```

    ENDFOR

```

```

ENDFOR

```

```

OUTPUT STRING(The Second highest number in the list = {numlist[-2]})

```

16. Find GCD of 2 numbers using Euclidean algorithm

[python code]

```

firstNum = 0

```



```
secondNum = 0
errorString = "That's Not a Valid Number, Please Try Again!"
loop = True
```

```
while loop == True:
    firstNum = input("Enter First Number: ")
    try:
        firstNum = int(firstNum)
        if firstNum < 0:
            firstNum = abs(firstNum)
        loop = False
    except:
        print(errorString)
```

```
loop = True
while loop == True:
    secondNum = input("Enter Second Number: ")
    try:
        secondNum = int(secondNum)
        if secondNum < 0:
            secondNum = abs(secondNum)
        loop = False
    except:
        print(errorString)
```

```
def gcd(first, second):
    remainder = first % second
    while remainder != 0:
        first = second
        second = remainder
        remainder = first % second
    return second
```

```
if firstNum > secondNum:
    a = firstNum
    b = secondNum
else:
    a = secondNum
    b = firstNum
```

```
print("GCD =", gcd(a, b))
```

[pseudocode]

```
firstNum <- 0
secondNum <- 0
errorString <- STRING(That's Not a Valid Number, Please Try Again!)
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO
```

```

firstNum <- INPUT(STRING(Enter First Number:))

TRY
    firstNum <- INTEGER(firstNum)

    IF firstNum < 0 THEN
        firstNum <- ABS(firstNum)
    ENDIF

    loop <- BOOLEAN(False)

CATCH
    OUTPUT STRING({errorString})
ENDTRY
ENDWHILE

loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
    secondNum <- INPUT(STRING(Enter Second Number:))

    TRY
        secondNum <- INTEGER(secondNum)

        IF secondNum < 0 THEN
            secondNum <- ABS(secondNum)
        ENDIF

        loop <- BOOLEAN(False)

    CATCH
        OUTPUT STRING({errorString})
    ENDTRY
ENDWHILE

FUNCTION gcd(first, second)
    remainder <- first MOD second

    WHILE remainder != 0 DO
        first <- second
        second <- remainder
        remainder <- first MOD second
    ENDWHILE

    RETURN second
ENDFUNCTION

```

```

IF firstNum > secondNum THEN
    a <- firstNum
    b <- secondNum
ELSE
    a <- secondNum
    b <- firstNum
ENDIF

OUTPUT STRING(GCD = {gcd(a, b)})

```

17. Compute the sum and average of averages list of numbers

[python code]

```

numlist = [7,9,3,5,2.6,8.3]
total = 0
counter = 0
while len(numlist) > 0:
    total += numlist.pop(0)
    counter += 1
average = total / counter
total = format(total, '.2f')
average = format(average, '.2f')

print(f"The Sum of Numbers in the list = ", total)
print(f"The Average = ", average)

```

[pseudocode]

```

numlist <- LIST OF [7, 9, 3, 5, 2.6, 8.3]
total <- 0
counter <- 0

WHILE LENGTH OF numlist > 0 DO
    total <- total + VALUE(REMOVE FIRST ITEM FROM numlist)
    counter <- counter + 1
ENDWHILE

average <- total / counter

total <- FORMAT(total, 2 DECIMAL PLACES)
average <- FORMAT(average, 2 DECIMAL PLACES)

```

18. Compute the frequency of each character in a string

[python code]

```

string1 = "This is a Sentence, that needs the chars counted!!"
stringlist = list(string1)

```

```

countlist = {}
while len(stringlist) > 0:
    char = stringlist.pop(0)
    if char not in countlist:
        countlist[char] = 1
    else:
        countlist[char] += 1

for key, val in countlist.items():
    print(f"Character {key} appears {val} times")

```

[pseudocode]

```

string1 <- STRING(This is a Sentence, that needs the chars counted!!)
stringlist <- LIST OF CHARACTERS IN string1
countlist <- EMPTY DICTIONARY

```

```

WHILE LENGTH OF stringlist > 0 DO
    char <- VALUE(REMOVE FIRST ITEM FROM stringlist)

    IF char NOT IN countlist THEN
        countlist[char] <- 1
    ELSE
        countlist[char] <- countlist[char] + 1
    ENDIF
ENDWHILE

```

```

FOR key, val <- IN ITEMS OF countlist DO
    OUTPUT STRING(Character {key} appears {val} times)
ENDFOR

```

19. Swap 2 variables without using a third variable

[python code]

```

var1 = 3
var2 = "tree"

print(f"variable 1 = {var1}")
print(f"variable 2 = {var2}")

```

```

# If both are numbers → arithmetic swap
if isinstance(var1, (int, float)) and isinstance(var2, (int, float)):
    var1 = var1 + var2
    var2 = var1 - var2
    var1 = var1 - var2

```

```

# If both are strings → concatenation swap
elif isinstance(var1, str) and isinstance(var2, str):

```

```

var1 = var1 + var2
var2 = var1[:len(var1) - len(var2)]
var1 = var1[len(var2):]

# Mixed types → convert var1 to string (your original behaviour)
else:
    var1 = str(var1)
    var1 = var1 + var2
    var2 = var1[:len(var1) - len(var2)]
    var1 = var1[len(var2):]

print(f"variable 1 = {var1}")
print(f"variable 2 = {var2}")

```

[pseudocode]

```

var1 <- 3
var2 <- STRING(tree)

OUTPUT STRING(variable 1 = {var1})
OUTPUT STRING(variable 2 = {var2})

IF TYPE OF var1 == NUMBER AND TYPE OF var2 == NUMBER THEN

    var1 <- var1 + var2
    var2 <- var1 - var2
    var1 <- var1 - var2

ELSE IF TYPE OF var1 == STRING AND TYPE OF var2 == STRING THEN

    var1 <- var1 + var2
    var2 <- SUBSTRING(var1, 0, LENGTH OF var1 - LENGTH OF var2)
    var1 <- SUBSTRING(var1, LENGTH OF var2, LENGTH OF var1)

ELSE

    var1 <- STRING(var1)

    var1 <- var1 + var2
    var2 <- SUBSTRING(var1, 0, LENGTH OF var1 - LENGTH OF var2)
    var1 <- SUBSTRING(var1, LENGTH OF var2, LENGTH OF var1)

ENDIF

OUTPUT STRING(variable 1 = {var1})
OUTPUT STRING(variable 2 = {var2})

```

20. Determine all divisors in a given integer

[python code]

```
numvar = 0
givenNum = 0
numlist = []
loop = True
while loop == True:
    givenNum = input("Enter Number: ")
    try:
        givenNum = int(givenNum)
        loop = False
    except:
        print(f"That's Not a Valid Number, Please Try Again!")
while numvar <= givenNum:
    numvar +=1
    if givenNum % numvar == 0:
        numlist.append(numvar)

while loop == False:
    try:
        x = numlist.pop(0)
        print(f"{x} is a Divisor of {givenNum}")
    except:
        quit()
```

[pseudocode]

```
numvar <- 0
givenNum <- 0
numlist <- EMPTY LIST
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
    givenNum <- INPUT(STRING(Enter Number:))

    TRY
        givenNum <- INTEGER(givenNum)
        loop <- BOOLEAN(False)

    CATCH
        OUTPUT STRING(That's Not a Valid Number, Please Try Again!)
    ENENTRY
ENDWHILE

WHILE numvar <= givenNum DO
    numvar <- numvar + 1
```

```

IF givenNum MOD numvar == 0 THEN
    APPEND numvar TO numlist
ENDIF
ENDWHILE

```

```

WHILE loop == BOOLEAN(False) DO
    TRY
        x <- VALUE(REMOVE FIRST ITEM FROM numlist)
        OUTPUT STRING({x} is a Divisor of {givenNum})

    CATCH
        QUIT PROGRAM
    ENDTRY
ENDWHILE

```

21. Create a process that determines whether a number is even or odd

[*python code*]

```

givenNum = 0
loop = True
while loop == True:
    givenNum = input("Enter Number: ")
    try:
        givenNum = int(givenNum)
        loop = False
    except:
        print(f"That's Not a Valid Integer, Please Try Again!")
if givenNum % 2 == 0:
    print(f"{givenNum} is Even")
else:
    print(f"{givenNum} is Odd")

```

[*pseudocode*]

```

givenNum <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
    givenNum <- INPUT(STRING(Enter Number:))

    TRY
        givenNum <- INTEGER(givenNum)
        loop <- BOOLEAN(False)

    CATCH
        OUTPUT STRING(That's Not a Valid Integer, Please Try Again!)
    ENDTRY
ENDWHILE

```

```

IF givenNum MOD 2 == 0 THEN
    OUTPUT STRING({givenNum} is Even)
ELSE
    OUTPUT STRING({givenNum} is Odd)
ENDIF

```

22. Design a process that computes the factorial of a given integer

[python code]

```

loop = True
while loop == True:
    givenNum = input("Enter Number: ")
    try:
        givenNum = int(givenNum)
        loop = False
    except:
        print(f"That's Not an Integer, Please Try Again!")
total = 1
while givenNum > 0:
    total *= givenNum
    givenNum -= 1
print(f"Factorial = {total}")

```

[pseudocode]

```

givenNum <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
    givenNum <- INPUT(STRING(Enter Number:))

    TRY
        givenNum <- INTEGER(givenNum)
        loop <- BOOLEAN(False)

    CATCH
        OUTPUT STRING(That's Not an Integer, Please Try Again!)
    ENENTRY
ENDWHILE

total <- 1

WHILE givenNum > 0 DO
    total <- total * givenNum
    givenNum <- givenNum - 1

```


ENDWHILE

OUTPUT STRING(Factorial = {total})

23. Write a process that converts numerical scores into grades such as pass merit or distinction

[python code]

```
loop = True
grade = ""
while loop == True:
    score = input("Enter Student Score: ")
    try:
        score = float(score)
        loop = False
    except:
        print(f"That's Not a Number, Please Try Again!")
    try:
        loop = True
        if int(score) < 0 or int(score) > 100:
            raise ValueError()
        else:
            loop = False
    except ValueError:
        print("Score out of range 0 - 100")
if score >= 60 and score < 70:
    print(f"Congratulations, You received a Grade of Pass!")
elif score >= 70 and score < 80:
    print(f"Congratulations, You received a Grade of Merit!")
elif score >= 80 and score < 100:
    print(f"Congratulations, You received a Grade of Distinction!")
elif score == 100:
    print(f"WOW!! you got a Perfect score Excellent!!, Congratulations, You received a Grade of Distinction!")

else:
    print(f"Unfortunately, You did Not pass this time!")
```

[pseudocode]

```
givenNum <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
    givenNum <- INPUT(STRING(Enter Number:))

TRY
    givenNum <- FLOAT(givenNum)
    loop <- BOOLEAN(False)
```

```
CATCH
    OUTPUT STRING(That's Not a Number, Please Try Again!)
ENDTRY
```

```
TRY
    loop <- BOOLEAN(True)

    IF INTEGER(givenNum) < 0 OR INTEGER(givenNum) > 100 THEN
        RAISE ERROR
    ELSE
        loop <- BOOLEAN(False)
    ENDIF
```

```
CATCH
    OUTPUT STRING(SCORE OUT OF RANGE 0 - 100)
ENDTRY
ENDWHILE
```

```
IF givenNum >= 60 AND givenNum < 70 THEN
    OUTPUT STRING(Congratulations, You received a Grade of Pass!)

ELSE IF givenNum >= 70 AND givenNum < 80 THEN
    OUTPUT STRING(Congratulations, You received a Grade of Merit!)

ELSE IF givenNum >= 80 AND givenNum < 100 THEN
    OUTPUT STRING(Congratulations, You received a Grade of Distinction!)

ELSE IF givenNum == 100 THEN
    OUTPUT STRING(WOW!! you got a Perfect score Excellent!!, Congratulations, You received a Grade of Distinction!)

ELSE
    OUTPUT STRING(Unfortunately, You did Not pass this time!)
ENDIF
```

24. Create a process that determines if a password has 8 characters 1 number and 1 uppercase letter
[python code]

```
password = ""
num = False
upper = False
length = False
passlist = []
loop = True
while loop == True:
    password = input("Enter Password, it must be at least 8 Characters and contain at least 1 number and 1 uppercase letter : ")
    try:
        passlist = list(password)
```

```

if len(passlist) < 8:
    raise Exception
else:
    length = True
for x in passlist:
    if x.isdigit():
        num = True
        break
for y in passlist:
    if y.isupper():
        upper = True
        break
if length == True and num == True and upper == True:
    print(f"Password Accepted!!")
else:
    raise Exception
loop = False
except:
    print(f"That's Not a Valid Password, Please Try Again!")
    password = ""
    num = False
    upper = False
    length = False

```

[pseudocode]

```

password <- EMPTY STRING
num <- BOOLEAN(False)
upper <- BOOLEAN(False)
length <- BOOLEAN(False)
passlist <- EMPTY LIST
loop <- BOOLEAN(True)

```

```

WHILE loop == BOOLEAN(True) DO
    password <- INPUT(STRING(Enter Password, it must be at least 8 Characters and contain at least 1 number and 1
uppercase letter :))

```

```

TRY
    passlist <- LIST OF CHARACTERS IN password

```

```

IF LENGTH OF passlist < 8 THEN
    RAISE ERROR
ELSE
    length <- BOOLEAN(True)
ENDIF

```

```

FOR x <- IN RANGE 0 TO LENGTH OF passlist DO
    IF ISDIGIT(passlist[x]) THEN
        num <- BOOLEAN(True)

```

```

        BREAK
    ENDIF
ENDFOR

FOR y <- IN RANGE 0 TO LENGTH OF passlist DO
    IF ISUPPER(passlist[y]) THEN
        upper <- BOOLEAN(True)
        BREAK
    ENDIF
ENDFOR

IF length == BOOLEAN(True) AND num == BOOLEAN(True) AND upper == BOOLEAN(True) THEN
    OUTPUT STRING(Password Accepted!!)
ELSE
    RAISE ERROR
ENDIF

loop <- BOOLEAN(False)

CATCH
    OUTPUT STRING(That's Not a Valid Password, Please Try Again!)
    password <- EMPTY STRING
    num <- BOOLEAN(False)
    upper <- BOOLEAN(False)
    length <- BOOLEAN(False)
ENDTRY
ENDWHILE

```

25. Find the length of the longest substring without repeating characters into a given string

[python code]

```

sentence = "this is a sentence made up of"
startindex = 0
substrlen = 0
offset = 0
sentencelist = list(sentence)
substrlist = []
substring = ""

loop = True
while loop:
    offset += 1
    substring = sentencelist[startindex:offset]
    substrlist.append(substring)

    if offset >= len(sentencelist):
        startindex += 1
        offset = startindex
        if startindex >= len(sentencelist):

```

```
startindex = 0
substrlen = 0
offset = 0
loop = False
```

```
listindex = 0
```

```
loop = True
```

```
while loop:
```

```
    excl = set()
```

```
    duplicate = False
```

```
    for ch in substrlist[listindex]:
```

```
        if ch in excl:
```

```
            duplicate = True
```

```
            break
```

```
        excl.add(ch)
```

```
    if duplicate:
```

```
        substrlist.pop(listindex)
```

```
    else:
```

```
        listindex += 1
```

```
    if listindex >= len(substrlist):
```

```
        loop = False
```

```
loop = True
```

```
index = 0
```

```
maxstrings = []
```

```
maxlength = 0
```

```
while loop:
```

```
    try:
```

```
        current = substrlist[index]
```

```
        currentlen = len(current)
```

```
        if currentlen > maxlength:
```

```
            maxlength = currentlen
```

```
            = [current]
```

```
        elif currentlen == maxlength:
```

```
            maxstrings.append(current)
```

```
        index += 1
```

```
    except:
```

```
        loop = False
```

```
print("The Longest Substrings Without Duplicate Characters Are:")
```

```
print(["".join(s) for s in maxstrings])
```

[pseudocode]

```
sentence <- STRING(this is a sentence made up of)
startindex <- 0
substrlen <- 0
offset <- 0
```

```
sentencelist <- LIST OF CHARACTERS IN sentence
substrlist <- EMPTY LIST
substring <- EMPTY LIST
```

```
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO
  offset <- offset + 1
```

```
  substring <- SUBLIST(sentencelist, startindex, offset)
  APPEND substring TO substrlist
```

```
  IF offset >= LENGTH OF sentencelist THEN
    startindex <- startindex + 1
    offset <- startindex
```

```
  IF startindex >= LENGTH OF sentencelist THEN
    startindex <- 0
    substrlen <- 0
    offset <- 0
    loop <- BOOLEAN(False)
```

```
  ENDIF
```

```
ENDIF
```

```
ENDWHILE
```

```
listindex <- 0
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO
  excl <- EMPTY SET
  duplicate <- BOOLEAN(False)
```

```
  FOR ch <- IN RANGE 0 TO LENGTH OF substrlist[listindex] DO
    currentchar <- substrlist[listindex][ch]
```

```
    IF currentchar IN excl THEN
      duplicate <- BOOLEAN(True)
      BREAK
    ENDIF
```

```
    ADD currentchar TO excl
ENDFOR
```

```
IF duplicate == BOOLEAN(True) THEN
    REMOVE ITEM AT listindex FROM substrlist
ELSE
    listindex <- listindex + 1
ENDIF
```

```
IF listindex >= LENGTH OF substrlist THEN
    loop <- BOOLEAN(False)
ENDIF
ENDWHILE
```

```
loop <- BOOLEAN(True)
index <- 0
maxstrings <- EMPTY LIST
maxlength <- 0
```

```
WHILE loop == BOOLEAN(True) DO
    TRY
        current <- substrlist[index]
        currentlen <- LENGTH OF current

        IF currentlen > maxlength THEN
            maxlength <- currentlen
            maxstrings <- LIST OF [current]

        ELSE IF currentlen == maxlength THEN
            APPEND current TO maxstrings
        ENDIF

        index <- index + 1
    
```

```
CATCH
    loop <- BOOLEAN(False)
ENDTRY
ENDWHILE
```

```
OUTPUT STRING(The Longest Substrings Without Duplicate Characters Are:)
OUTPUT STRING({JOIN(maxstrings)})
```

26. Perform a level order traversal on a binary tree and return node values grouped by depth

[python code]

```
class Node:
    def init(self, value):
        self.value = value
        self.left = None
        self.right = None

def levelOrderTraversal(root):
    if root is None:
        return []

    depth = [root]
    results = []
    loop = True

    while loop:
        level = []
        nextdepth = []
        index = 0

        innerloop = True
        while innerloop:
            try:
                node = depth[index]
                level.append(node.value)

                if node.left is not None:
                    nextdepth.append(node.left)
                if node.right is not None:
                    nextdepth.append(node.right)

                index += 1
            except:
                innerloop = False

        results.append(level)
        depth = nextdepth

        if len(depth) == 0:
            loop = False

    return results
```



```
# #Example Tree
```

```
# # 1
```

```
# # /\
```

```
# # 2 3
```

```
# # /\
```

```
# # 4 5
```

```
root = Node(1)
```

```
root.left = Node(2)
```

```
root.right = Node(3)
```

```
root.left.left = Node(4)
```

```
root.left.right = Node(5)
```

```
print("Node Values Grouped by Depth")
```

```
index = 0
```

```
loop = True
```

```
while loop:
```

```
    try:
```

```
        level = levelOrderTraversal(root)[index]
```

```
        print(f"Level {index}: {level}")
```

```
        index += 1
```

```
    except:
```

```
        loop = False
```

```
[ pseudocode ]
```

```
CLASS Node
```

```
    CONSTRUCTOR ACCEPTS(self, value)
```

```
        PROPERTIES(self.value <- value, self.left <- None, self.right <- None)
```

```
ENDCONSTRUCTOR
```

```
FUNCTION levelOrderTraversal(root)
```

```
    IF root == None THEN
```

```
        RETURN EMPTY LIST
```

```
    ENDIF
```

```
    depth <- LIST OF [root]
```

```
    results <- EMPTY LIST
```

```
    loop <- BOOLEAN(True)
```

```
    WHILE loop == BOOLEAN(True) DO
```

```
        level <- EMPTY LIST
```

```
        nextdepth <- EMPTY LIST
```

```
        index <- 0
```

```
        innerloop <- BOOLEAN(True)
```

```

WHILE innerloop == BOOLEAN(True) DO
  TRY
    node <- depth[index]
    APPEND node.value TO level

    IF node.left != None THEN
      APPEND node.left TO nextdepth
    ENDIF

    IF node.right != None THEN
      APPEND node.right TO nextdepth
    ENDIF

    index <- index + 1

  CATCH
    innerloop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

APPEND level TO results
depth <- nextdepth

IF LENGTH OF depth == 0 THEN
  loop <- BOOLEAN(False)
ENDIF
ENDWHILE

RETURN results
ENDFUNCTION
ENDCLASS

```

```

root <- NEW Node(1)
root.left <- NEW Node(2)
root.right <- NEW Node(3)
root.left.left <- NEW Node(4)
root.left.right <- NEW Node(5)

```

```

OUTPUT STRING(Node Values Grouped by Depth)

```

```

index <- 0
loop <- BOOLEAN(True)

```

```

WHILE loop == BOOLEAN(True) DO
  TRY
    level <- levelOrderTraversal(root)[index]

```

```
OUTPUT STRING(Level {index}: {level})
```

```
index <- index + 1
```

```
CATCH
```

```
loop <- BOOLEAN(False)
```

```
ENDTRY
```

```
ENDWHILE
```

27. Rotate a 2D matrix 90 degrees clockwise in place

[python code]

```
matrix = [
```

```
    [1, 2, 3],
```

```
    [4, 5, 6],
```

```
    [7, 8, 9]
```

```
]
```

```
n = len(matrix)
```

```
print("Matrix Before Rotation:")
```

```
row = 0
```

```
loop = True
```

```
while loop:
```

```
    try:
```

```
        print(matrix[row])
```

```
        row += 1
```

```
    except:
```

```
        loop = False
```

```
row = 0
```

```
loop = True
```

```
while loop:
```

```
    col = row + 1
```

```
    innerloop = True
```

```
    while innerloop:
```

```
        try:
```

```
            temp = matrix[row][col]
```

```
            matrix[row][col] = matrix[col][row]
```

```
            matrix[col][row] = temp
```

```
            col += 1
```

```
        except:
```

```
            innerloop = False
```

```
row += 1
```

```
if row >= n:
```

```
    loop = False
```

```

row = 0
loop = True

while loop:
    left = 0
    right = n - 1
    innerloop = True

    while innerloop:
        if left >= right:
            innerloop = False
        else:
            temp = matrix[row][left]
            matrix[row][left] = matrix[row][right]
            matrix[row][right] = temp
            left += 1
            right -= 1

    row += 1
    if row >= n:
        loop = False

print("\nMatrix After 90° Clockwise Rotation:")
row = 0
loop = True
while loop:
    try:
        print(matrix[row])
        row += 1
    except:
        loop = False

```

[pseudocode]

```

matrix <- LIST OF [
    LIST OF [1, 2, 3],
    LIST OF [4, 5, 6],
    LIST OF [7, 8, 9]
]

n <- LENGTH OF matrix

OUTPUT STRING(Matrix Before Rotation:)
row <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
    TRY
        OUTPUT STRING({matrix[row]})
    CATCH
        loop <- FALSE
    END TRY
END WHILE

```

```
    row <- row + 1
CATCH
    loop <- BOOLEAN(False)
ENDTRY
ENDWHILE
```

```
row <- 0
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO
    col <- row + 1
    innerloop <- BOOLEAN(True)
```

```
    WHILE innerloop == BOOLEAN(True) DO
        TRY
            temp <- matrix[row][col]
            matrix[row][col] <- matrix[col][row]
            matrix[col][row] <- temp
            col <- col + 1
        CATCH
            innerloop <- BOOLEAN(False)
        ENDTRY
    ENDWHILE
```

```
    row <- row + 1
```

```
    IF row >= n THEN
        loop <- BOOLEAN(False)
    ENDIF
ENDWHILE
```

```
row <- 0
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO
    left <- 0
    right <- n - 1
    innerloop <- BOOLEAN(True)
```

```
    WHILE innerloop == BOOLEAN(True) DO
        IF left >= right THEN
            innerloop <- BOOLEAN(False)
        ELSE
            temp <- matrix[row][left]
            matrix[row][left] <- matrix[row][right]
            matrix[row][right] <- temp
```

```

        left <- left + 1
        right <- right - 1
    ENDIF
ENDWHILE

row <- row + 1

IF row >= n THEN
    loop <- BOOLEAN(False)
ENDIF
ENDWHILE

OUTPUT STRING(Matrix After 90° Clockwise Rotation:)
row <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
    TRY
        OUTPUT STRING({matrix[row]})
        row <- row + 1
    CATCH
        loop <- BOOLEAN(False)
    ENDTRY
ENDWHILE

```

28. For a fixed window size sliding across an array return the maximum value within each window position

[python code]

```

arr = [1, 3, -1, -3, 5, 3, 6, 7]
k = 2

```

```

results = []
start = 0
loop = True

```

```

while loop:
    end = start + k

```

```

    if end > len(arr):
        break

```

```

    index = start
    innerloop = True
    maxval = arr[start]

```

```

    while innerloop:
        try:

```

```

        if arr[index] > maxval:
            maxval = arr[index]
        index += 1
    except:
        innerloop = False

    if index >= end:
        innerloop = False

    results.append(maxval)
    start += 1

print(f"Sliding Window Maximums From Window Size {k}:")
print(results)

```

[pseudocode]

```

arr <- LIST OF [1, 3, -1, -3, 5, 3, 6, 7]
k <- 2

```

```

results <- EMPTY LIST
start <- 0
loop <- BOOLEAN(True)

```

```

WHILE loop == BOOLEAN(True) DO
    end <- start + k

```

```

    IF end > LENGTH OF arr THEN
        BREAK
    ENDIF

```

```

    index <- start
    innerloop <- BOOLEAN(True)
    maxval <- arr[start]

```

```

    WHILE innerloop == BOOLEAN(True) DO
        TRY
            IF arr[index] > maxval THEN
                maxval <- arr[index]
            ENDIF

```

```

            index <- index + 1

```

```

        CATCH
            innerloop <- BOOLEAN(False)
        ENDTRY

```

```

    IF index >= end THEN
        innerloop <- BOOLEAN(False)

```

```
ENDIF
ENDWHILE
```

```
APPEND maxval TO results
start <- start + 1
ENDWHILE
```

```
OUTPUT STRING(Sliding Window Maximums From Window Size {k}:)
OUTPUT STRING({results})
```

29. Build a text classification model to categorise text into predefined categories

[python code]

```
trainingdata = {
    "sports": [
        "team wins match",
        "player scores goal",
        "championship game victory"
    ],
    "tech": [
        "new smartphone released",
        "software update improves performance",
        "AI model breakthrough benchmark"
    ],
    "food": [
        "fresh ingredients recipe",
        "chef prepares meal",
        "restaurant serves tasty dishes"
    ]
}
```

```
def tokenise(text):
    return text.lower().split()
```

```
def classify(text):
    words = tokenise(text)
    bestcategory = None
    bestscore = -1
```

```
categories = list(trainingdata.keys())
c_index = 0
loop = True
```

```
while loop:
    try:
        category = categories[c_index]
        score = 0
```



```
s_index = 0
innerloop = True
```

```
while innerloop:
```

```
    try:
        sentence = trainingdata[category][s_index]
        sentence_words = tokenise(sentence)
```

```

        w_index = 0
        wordloop = True
```

```

        while wordloop:
```

```
            try:
                if words[w_index] in sentence_words:
                    score += 1
                    w_index += 1
            except:
                wordloop = False
```

```

        s_index += 1
    except:
        innerloop = False
```

```

# Track best category
if score > bestscore:
    bestscore = score
    bestcategory = category
```

```

    c_index += 1
except:
    loop = False
```

```
return bestcategory
```

```
tests = [
    "the team scored a goal",
    "new processor released",
    "chef cooks tasty meal",
    "fish and chips are tasty"
]
```

```
print("Simple Text Classification:")
index = 0
loop = True
```

```
while loop:
```

```
    try:
        text = tests[index]
        result = classify(text)
```

```
print(f'''{text}' → {result}''')
index += 1
except:
    loop = False
```

[pseudocode]

```
trainingdata <- DICTIONARY OF {
  sports: LIST OF [
    STRING(team wins match),
    STRING(player scores goal),
    STRING(championship game victory)
  ],

  tech: LIST OF [
    STRING(new smartphone released),
    STRING(software update improves performance),
    STRING(AI model breakthrough benchmark)
  ],

  food: LIST OF [
    STRING(fresh ingredients recipe),
    STRING(chef prepares meal),
    STRING(restaurant serves tasty dishes)
  ]
}
```

```
FUNCTION tokenise(text)
  RETURN SPLIT(LOWERCASE(text))
ENDFUNCTION
```

```
FUNCTION classify(text)
  words <- tokenise(text)
  bestcategory <- None
  bestscore <- -1

  categories <- LIST OF KEYS IN trainingdata
  c_index <- 0
  loop <- BOOLEAN(True)

  WHILE loop == BOOLEAN(True) DO
    TRY
      category <- categories[c_index]
      score <- 0

      s_index <- 0
      innerloop <- BOOLEAN(True)
```

```

WHILE innerloop == BOOLEAN(True) DO
  TRY
    sentence <- trainingdata[category][s_index]
    sentence_words <- tokenise(sentence)

    w_index <- 0
    wordloop <- BOOLEAN(True)

    WHILE wordloop == BOOLEAN(True) DO
      TRY
        IF words[w_index] IN sentence_words THEN
          score <- score + 1
        ENDIF

        w_index <- w_index + 1

      CATCH
        wordloop <- BOOLEAN(False)
      ENDTRY
    ENDWHILE

    s_index <- s_index + 1

  CATCH
    innerloop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

IF score > bestscore THEN
  bestscore <- score
  bestcategory <- category
ENDIF

c_index <- c_index + 1

CATCH
  loop <- BOOLEAN(False)
ENDTRY
ENDWHILE

RETURN bestcategory
ENDFUNCTION

```

```

tests <- LIST OF [
  STRING(the team scored a goal),
  STRING(new processor released),

```

```
    STRING(chef cooks tasty meal),  
    STRING(fish and chips are tasty)  
]
```

OUTPUT STRING(Simple Text Classification:)

```
index <- 0  
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO  
    TRY  
        text <- tests[index]  
        result <- classify(text)  
        OUTPUT STRING({text} -> {result})  
        index <- index + 1  
  
    CATCH  
        loop <- BOOLEAN(False)  
    ENDTRY  
ENDWHILE
```

30. Create a topic modelling system to extract underlying topics from a large corpus
[python code]

```
corpus = [  
    "the team won the match and scored many goals",  
    "the player scored a hat trick in the game",  
    "the championship match ended in a victory",  
  
    "new smartphone released with powerful processor and fast performance",  
    "software update improves device performance and battery life",  
    "AI model achieves breakthrough in image recognition",  
  
    "fresh ingredients used in this recipe",  
    "the chef prepares a delicious meal at the restaurant",  
    "the restaurant serves tasty dishes and great food"  
]
```

```
stopwords = ["the", "and", "in", "a", "at", "with", "this", "many"]
```

```
def tokenise(text):  
    words = text.lower().split()  
    clean = []  
    i = 0  
    loop = True  
    while loop:  
        try:  
            if words[i] not in stopwords:  
                clean.append(words[i])
```

```
        i += 1
    except:
        loop = False
    return clean
```

```
word_freq = {}
co_occurrence = {}
doc_tokens = []
```

```
i = 0
loop = True
while loop:
    try:
        text = corpus[i]
        words = tokenise(text)
        doc_tokens.append(words)
```

```
        j = 0
        innerloop = True
        while innerloop:
            try:
                w = words[j]
                if w not in word_freq:
                    word_freq[w] = 0
                word_freq[w] += 1
                j += 1
            except:
                innerloop = False
```

```
        j = 0
        innerloop = True
        while innerloop:
            try:
                w1 = words[j]
                if w1 not in co_occurrence:
                    co_occurrence[w1] = {}
```

```
            k = 0
            inner2 = True
            while inner2:
                try:
                    w2 = words[k]
                    if w1 != w2:
                        if w2 not in co_occurrence[w1]:
                            co_occurrence[w1][w2] = 0
                        co_occurrence[w1][w2] += 1
                    k += 1
                except:
                    inner2 = False
```

```
        j += 1
    except:
        innerloop = False
```

```
    i += 1
except:
    loop = False
```

```
items = list(word_freq.items())
n = len(items)
i = 0
loop = True
while loop:
    if i >= n - 1:
        loop = False
    else:
        j = 0
        innerloop = True
        while innerloop:
            if j >= n - 1:
                innerloop = False
            else:
                if items[j][1] < items[j + 1][1]:
                    temp = items[j]
                    items[j] = items[j + 1]
                    items[j + 1] = temp
                j += 1
        i += 1
```

```
TOP_K = 12
top_words = []
i = 0
loop = True
while loop:
    if i >= TOP_K or i >= len(items):
        loop = False
    else:
        top_words.append(items[i][0])
        i += 1
```

```
topics = []
used = {}
i = 0
loop = True
while loop:
    try:
        w = top_words[i]
```

if w not in used:

topic = [w]

used[w] = True

if w in co_occurrence:

pairs = list(co_occurrence[w].items())

m = len(pairs)

a = 0

sortloop = True

while sortloop:

if a >= m - 1:

sortloop = False

else:

b = 0

innersort = True

while innersort:

if b >= m - 1:

innersort = False

else:

if pairs[b][1] < pairs[b + 1][1]:

temp = pairs[b]

pairs[b] = pairs[b + 1]

pairs[b + 1] = temp

b += 1

a += 1

c = 0

addloop = True

while addloop:

try:

w2 = pairs[c][0]

if w2 not in used:

topic.append(w2)

used[w2] = True

c += 1

if c >= 2:

addloop = False

except:

addloop = False

topics.append(topic)

i += 1

except:

loop = False

label_words = {

"Sports": ["team", "match", "goal", "player", "score", "victory", "game"],

```

    "Tech": ["processor", "software", "device", "update", "model", "performance", "smartphone"],
    "Food": ["chef", "meal", "ingredients", "recipe", "restaurant", "dishes", "food"]
}

def auto_label(topic):
    best_label = None
    best_score = -1

    labels = list(label_words.keys())
    l_index = 0
    loop = True

    while loop:
        try:
            label = labels[l_index]
            score = 0

            t_index = 0
            innerloop = True
            while innerloop:
                try:
                    w = topic[t_index]

                    lw_index = 0
                    lw_loop = True
                    while lw_loop:
                        try:
                            if w == label_words[label][lw_index]:
                                score += 1
                                lw_index += 1
                        except:
                            lw_loop = False

                    t_index += 1
                except:
                    innerloop = False

            if score > best_score:
                best_score = score
                best_label = label

            l_index += 1
        except:
            loop = False

    return best_label, best_score

topic_labels = []

```



```

i = 0
loop = True
while loop:
    try:
        topic = topics[i]
        label, score = auto_label(topic)
        topic_labels.append(label)
        i += 1
    except:
        loop = False

def classify(text):
    words = tokenise(text)
    best_topic = None
    best_label = None
    best_score = -1

    i = 0
    loop = True
    while loop:
        try:
            topic = topics[i]
            label = topic_labels[i]
            score = 0

            j = 0
            innerloop = True
            while innerloop:
                try:
                    if words[j] in topic:
                        score += 1
                    j += 1
                except:
                    innerloop = False

            if score > best_score:
                best_score = score
                best_topic = topic
                best_label = label

            i += 1
        except:
            loop = False

    return best_label, best_topic, best_score

print("Topics -> Category:\n")

i = 0

```

```

loop = True
while loop:
    try:
        print(f"Topic {i}:{topics[i]} -> {topic_labels[i]}")
        i += 1
    except:
        loop = False

```

[pseudocode]

```

corpus <- LIST OF [
    STRING(the team won the match and scored many goals),
    STRING(the player scored a hat trick in the game),
    STRING(the championship match ended in a victory),

    STRING(new smartphone released with powerful processor and fast performance),
    STRING(software update improves device performance and battery life),
    STRING(AI model achieves breakthrough in image recognition),

    STRING(fresh ingredients used in this recipe),
    STRING(the chef prepares a delicious meal at the restaurant),
    STRING(the restaurant serves tasty dishes and great food)
]

```

```

stopwords <- LIST OF [
    STRING(the), STRING(and), STRING(in), STRING(a),
    STRING(at), STRING(with), STRING(this), STRING(many)
]

```

```

FUNCTION tokenise(text)
    words <- SPLIT(LOWERCASE(text))
    clean <- EMPTY LIST
    i <- 0
    loop <- BOOLEAN(True)

    WHILE loop == BOOLEAN(True) DO
        TRY
            IF words[i] NOT IN stopwords THEN
                APPEND words[i] TO clean
            ENDIF
            i <- i + 1
        CATCH
            loop <- BOOLEAN(False)
        ENDTRY
    ENDWHILE

    RETURN clean
ENDFUNCTION

```

```
word_freq <- EMPTY DICTIONARY
co_occurrence <- EMPTY DICTIONARY
doc_tokens <- EMPTY LIST
```

```
i <- 0
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO
  TRY
    text <- corpus[i]
    words <- tokenise(text)
    APPEND words TO doc_tokens
```

```
  j <- 0
  innerloop <- BOOLEAN(True)
```

```
  WHILE innerloop == BOOLEAN(True) DO
    TRY
      w <- words[j]
      IF w NOT IN word_freq THEN
        word_freq[w] <- 0
      ENDIF
      word_freq[w] <- word_freq[w] + 1
      j <- j + 1
    CATCH
      innerloop <- BOOLEAN(False)
    ENDTRY
  ENDWHILE
```

```
  j <- 0
  innerloop <- BOOLEAN(True)
```

```
  WHILE innerloop == BOOLEAN(True) DO
    TRY
      w1 <- words[j]
      IF w1 NOT IN co_occurrence THEN
        co_occurrence[w1] <- EMPTY DICTIONARY
      ENDIF
```

```
      k <- 0
      inner2 <- BOOLEAN(True)
```

```
      WHILE inner2 == BOOLEAN(True) DO
        TRY
          w2 <- words[k]
          IF w1 != w2 THEN
            IF w2 NOT IN co_occurrence[w1] THEN
```

```

        co_occurrence[w1][w2] <- 0
    ENDIF
    co_occurrence[w1][w2] <- co_occurrence[w1][w2] + 1
    ENDIF
    k <- k + 1
    CATCH
        inner2 <- BOOLEAN(False)
    ENDTRY
ENDWHILE

    j <- j + 1
    CATCH
        innerloop <- BOOLEAN(False)
    ENDTRY
ENDWHILE

    i <- i + 1
    CATCH
        loop <- BOOLEAN(False)
    ENDTRY
ENDWHILE

```

```

items <- LIST OF KEY VALUE PAIRS IN word_freq
n <- LENGTH OF items
i <- 0
loop <- BOOLEAN(True)

```

```

WHILE loop == BOOLEAN(True) DO
    IF i >= n - 1 THEN
        loop <- BOOLEAN(False)
    ELSE
        j <- 0
        innerloop <- BOOLEAN(True)

        WHILE innerloop == BOOLEAN(True) DO
            IF j >= n - 1 THEN
                innerloop <- BOOLEAN(False)
            ELSE
                IF items[j][1] < items[j + 1][1] THEN
                    temp <- items[j]
                    items[j] <- items[j + 1]
                    items[j + 1] <- temp
                ENDIF
                j <- j + 1
            ENDIF
        ENDWHILE

        i <- i + 1
    ENDIF
ENDWHILE

```

```
ENDIF  
ENDWHILE
```

```
TOP_K <- 12  
top_words <- EMPTY LIST  
i <- 0  
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO  
  IF i >= TOP_K OR i >= LENGTH OF items THEN  
    loop <- BOOLEAN(False)  
  ELSE  
    APPEND items[i][0] TO top_words  
    i <- i + 1  
  ENDIF  
ENDWHILE
```

```
topics <- EMPTY LIST  
used <- EMPTY DICTIONARY  
i <- 0  
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO  
  TRY  
    w <- top_words[i]  
  
    IF w NOT IN used THEN  
      topic <- LIST OF [w]  
      used[w] <- BOOLEAN(True)  
  
      IF w IN co_occurrence THEN  
        pairs <- LIST OF KEY VALUE PAIRS IN co_occurrence[w]  
  
        m <- LENGTH OF pairs  
        a <- 0  
        sortloop <- BOOLEAN(True)  
  
        WHILE sortloop == BOOLEAN(True) DO  
          IF a >= m - 1 THEN  
            sortloop <- BOOLEAN(False)  
          ELSE  
            b <- 0  
            innersort <- BOOLEAN(True)  
  
            WHILE innersort == BOOLEAN(True) DO  
              IF b >= m - 1 THEN  
                innersort <- BOOLEAN(False)
```

```

ELSE
  IF pairs[b][1] < pairs[b + 1][1] THEN
    temp <- pairs[b]
    pairs[b] <- pairs[b + 1]
    pairs[b + 1] <- temp
  ENDIF
  b <- b + 1
ENDIF
ENDWHILE

```

```

a <- a + 1
ENDIF
ENDWHILE

```

```

c <- 0
addloop <- BOOLEAN(True)

```

```

WHILE addloop == BOOLEAN(True) DO
  TRY
    w2 <- pairs[c][0]
    IF w2 NOT IN used THEN
      APPEND w2 TO topic
      used[w2] <- BOOLEAN(True)
    ENDIF
    c <- c + 1
    IF c >= 2 THEN
      addloop <- BOOLEAN(False)
    ENDIF
  CATCH
    addloop <- BOOLEAN(False)
  ENDTRY
ENDWHILE
ENDIF

```

```

APPEND topic TO topics
ENDIF

```

```

i <- i + 1
CATCH
  loop <- BOOLEAN(False)
ENDTRY
ENDWHILE

```

```

label_words <- DICTIONARY OF {
  Sports: LIST OF [
    STRING(team), STRING(match), STRING(goal),
    STRING(player), STRING(score), STRING(victory),
    STRING(game)
  ]
}

```

```
],
```

```
Tech: LIST OF [
```

```
  STRING(processor), STRING(software), STRING(device),  
  STRING(update), STRING(model), STRING(performance),  
  STRING(smartphone)
```

```
],
```

```
Food: LIST OF [
```

```
  STRING(chef), STRING(meal), STRING(ingredients),  
  STRING(recipe), STRING(restaurant), STRING(dishes),  
  STRING(food)
```

```
]
```

```
}
```

```
FUNCTION auto_label(topic)
```

```
  best_label <- None
```

```
  best_score <- -1
```

```
  labels <- LIST OF KEYS IN label_words
```

```
  l_index <- 0
```

```
  loop <- BOOLEAN(True)
```

```
  WHILE loop == BOOLEAN(True) DO
```

```
    TRY
```

```
      label <- labels[l_index]
```

```
      score <- 0
```

```
      t_index <- 0
```

```
      innerloop <- BOOLEAN(True)
```

```
      WHILE innerloop == BOOLEAN(True) DO
```

```
        TRY
```

```
          w <- topic[t_index]
```

```
          lw_index <- 0
```

```
          lw_loop <- BOOLEAN(True)
```

```
          WHILE lw_loop == BOOLEAN(True) DO
```

```
            TRY
```

```
              IF w == label_words[label][lw_index] THEN
```

```
                score <- score + 1
```

```
              ENDIF
```

```
              lw_index <- lw_index + 1
```

```
            CATCH
```

```
              lw_loop <- BOOLEAN(False)
```

```
            ENDTRY
```

```
          ENDWHILE
```

```

        t_index <- t_index + 1
    CATCH
        innerloop <- BOOLEAN(False)
    ENDTRY
ENDWHILE

IF score > best_score THEN
    best_score <- score
    best_label <- label
ENDIF

l_index <- l_index + 1
CATCH
    loop <- BOOLEAN(False)
ENDTRY
ENDWHILE

RETURN best_label, best_score
ENDFUNCTION

```

```

topic_labels <- EMPTY LIST

```

```

i <- 0
loop <- BOOLEAN(True)

```

```

WHILE loop == BOOLEAN(True) DO
    TRY
        topic <- topics[i]
        label, score <- auto_label(topic)
        APPEND label TO topic_labels
        i <- i + 1
    CATCH
        loop <- BOOLEAN(False)
    ENDTRY
ENDWHILE

```

```

FUNCTION classify(text)
    words <- tokenise(text)
    best_topic <- None
    best_label <- None
    best_score <- -1

    i <- 0
    loop <- BOOLEAN(True)

    WHILE loop == BOOLEAN(True) DO

```



```

TRY
  topic <- topics[i]
  label <- topic_labels[i]
  score <- 0

  j <- 0
  innerloop <- BOOLEAN(True)

  WHILE innerloop == BOOLEAN(True) DO
    TRY
      IF words[j] IN topic THEN
        score <- score + 1
      ENDIF
      j <- j + 1
    CATCH
      innerloop <- BOOLEAN(False)
    ENDTRY
  ENDWHILE

  IF score > best_score THEN
    best_score <- score
    best_topic <- topic
    best_label <- label
  ENDIF

  i <- i + 1
CATCH
  loop <- BOOLEAN(False)
ENDTRY
ENDWHILE

RETURN best_label, best_topic, best_score
ENDFUNCTION

OUTPUT STRING(Topics -> Category:)

i <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
  TRY
    OUTPUT STRING(Topic {i}:{topics[i]} -> {topic_labels[i]})
    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

```

31. Develop a co-reference resolution system to identify relationships between Pronouns and Nouns

[python code]

text = "John gave his dog a bath because they were dirty. Sarah told him to use warm water. Then she asked him to clean his car as he is dirty"

```
# -----
# PRONOUN DEFINITIONS (gender, role)
# -----
pronouns = {
    "he": ("male", "subject"),
    "him": ("male", "object"),
    "his": ("male", "possessive"),
    "she": ("female", "subject"),
    "her": ("female", "object"),
    "they": ("plural", "subject"),
    "them": ("plural", "object"),
    "their": ("plural", "possessive")
}

# -----
# NAME LISTS
# -----
male_names = ["john", "peter", "michael", "david"]
female_names = ["sarah", "mary", "anna", "julia"]

# -----
# COMMON NOUNS
# -----
common_nouns = ["dog", "cat", "car", "bike", "boat", "ship", "motorbike"]

# -----
# ANIMACY CATEGORIES
# -----
animate_nouns = ["dog", "cat", "man", "woman", "boy", "girl"]
anthropomorphic_nouns = ["car", "bike", "boat", "ship", "motorbike"]

# -----
# PLURAL EXCEPTIONS
# -----
plural_exceptions = ["children", "people", "men", "women"]

# -----
# TOKENISE
# -----
def tokenise(text):
    return text.lower().replace(".", "").split()
```

```

words = tokenise(text)

# -----
# IDENTIFY NOUN CANDIDATES
# nouns = (word, index, gender, kind, animacy, number)
# -----
nouns = []

i = 0
loop = True
while loop:
    try:
        w = words[i]
        found = False
        number = "singular"

        # plural exceptions
        e = 0
        e_loop = True
        while e_loop:
            try:
                if w == plural_exceptions[e]:
                    number = "plural"
                    break
                e += 1
            except:
                e_loop = False

        # plural by 's'
        if number == "singular":
            if len(w) > 1 and w.endswith("s"):
                number = "plural"

        # male names
        m = 0
        m_loop = True
        while m_loop:
            try:
                if w == male_names[m]:
                    nouns.append((w, i, "male", "name", "animate", number))
                    found = True
                    m_loop = False
                m += 1
            except:
                m_loop = False

        # female names
        if not found:
            f = 0

```

```

f_loop = True
while f_loop:
    try:
        if w == female_names[f]:
            nouns.append((w, i, "female", "name", "animate", number))
            found = True
            f_loop = False
        f += 1
    except:
        f_loop = False

```

common nouns

if not found:

c = 0

c_loop = True

while c_loop:

try:

if w == common_nouns[c]:

animacy = "inanimate"

animate?

a = 0

a_loop = True

while a_loop:

try:

if w == animate_nouns[a]:

animacy = "animate"

a_loop = False

a += 1

except:

a_loop = False

anthropomorphic?

h = 0

h_loop = True

while h_loop:

try:

if w == anthropomorphic_nouns[h]:

animacy = "anthro"

h_loop = False

h += 1

except:

h_loop = False

nouns.append((w, i, "unknown", "noun", animacy, number))

found = True

c_loop = False

```
        c += 1
    except:
        c_loop = False
```

```
    i += 1
except:
    loop = False
```

```
# -----
# POSSESSIVE PERSONIFICATION RULE
# -----
```

```
i = 0
loop = True
while loop:
    try:
        if words[i] == "his" or words[i] == "her":
            noun_word = words[i + 1]

            n = 0
            n_loop = True
            while n_loop:
                try:
                    w, idx, g, kind, anim, number = nouns[n]
                    if w == noun_word:
                        new_gender = "male"
                        if words[i] == "her":
                            new_gender = "female"
                        nouns[n] = (w, idx, new_gender, kind, "animate", number)
                    n += 1
                except:
                    n_loop = False

            i += 1
    except:
        loop = False
```

```
    i += 1
except:
    loop = False
```

```
# -----
# RESOLVE PRONOUNS → NOUNS
# -----
```

```
corefs = []
```

```
i = 0
loop = True
while loop:
    try:
        w = words[i]

        # check pronoun
        keys = list(pronouns.keys())
```

```
p = 0
p_loop = True
is_pronoun = False
gender = None
role = None
```

```
while p_loop:
    try:
        if w == keys[p]:
            is_pronoun = True
            gender, role = pronouns[w]
            p_loop = False
            p += 1
    except:
        p_loop = False
```

```
if is_pronoun:
    best_noun = None
    best_dist = 9999
```

```
# -----
# PLURAL PRONOUNS
# -----
```

```
if gender == "plural":
    n = 0
    n_loop = True
    while n_loop:
        try:
            noun_word, noun_idx, noun_gender, noun_kind, animacy, number = nouns[n]

            if noun_idx < i and number == "plural":
                dist = i - noun_idx
                if dist < best_dist:
                    best_dist = dist
                    best_noun = noun_word

            n += 1
        except:
            n_loop = False
```

```
# FALLBACK: plural group of singular nouns
if best_noun is None:
    group = []
    n = 0
    n_loop = True
    while n_loop:
        try:
            noun_word, noun_idx, noun_gender, noun_kind, animacy, number = nouns[n]
            if noun_idx < i and number == "singular":
```

```

        group.append((noun_word, noun_idx))
    n += 1
except:
    n_loop = False

if len(group) >= 2:
    nearest = None
    best_dist = 9999

    g = 0
    g_loop = True
    while g_loop:
        try:
            noun_word, noun_idx = group[g]
            dist = i - noun_idx
            if dist < best_dist:
                best_dist = dist
                nearest = noun_word
            g += 1
        except:
            g_loop = False

    best_noun = nearest

# -----
# SINGULAR PRONOUNS
# -----
else:

    # SUBJECT PRONOUNS
    if role == "subject":

        n = 0
        n_loop = True
        while n_loop:
            try:
                noun_word, noun_idx, noun_gender, noun_kind, animacy, number = nouns[n]

                if noun_idx < i and number == "singular":

                    if gender == "male" and animacy == "animate":
                        dist = i - noun_idx
                        if dist < best_dist:
                            best_dist = dist
                            best_noun = noun_word

                    if gender == "female" and (animacy == "animate" or animacy == "anthro"):
                        dist = i - noun_idx
                        if dist < best_dist:

```

```

        best_dist = dist
        best_noun = noun_word

    n += 1
except:
    n_loop = False

# OBJECT / POSSESSIVE
else:
    n = 0
    n_loop = True
    while n_loop:
        try:
            noun_word, noun_idx, noun_gender, noun_kind, animacy, number = nouns[n]

            if noun_idx < i and noun_kind == "name" and number == "singular":
                if noun_gender == gender or noun_gender == "unknown":
                    dist = i - noun_idx
                    if dist < best_dist:
                        best_dist = dist
                        best_noun = noun_word

        n += 1
    except:
        n_loop = False

if best_noun is None:
    best_dist = 9999
    n = 0
    n_loop = True
    while n_loop:
        try:
            noun_word, noun_idx, noun_gender, noun_kind, animacy, number = nouns[n]

            if noun_idx < i and animacy == "animate" and number == "singular":
                dist = i - noun_idx
                if dist < best_dist:
                    best_dist = dist
                    best_noun = noun_word

        n += 1
    except:
        n_loop = False

# record result
corefs.append((w, i, best_noun))

# DYNAMIC GENDER + ANIMACY UPDATE
n = 0

```



```

n_loop = True
while n_loop:
    try:
        noun_word, noun_idx, noun_gender, noun_kind, animacy, number = nouns[n]

        if noun_word == best_noun:

            if noun_gender == "unknown":
                nouns[n] = (noun_word, noun_idx, gender, noun_kind, "animate", number)

            elif animacy == "anthro":
                nouns[n] = (noun_word, noun_idx, gender, noun_kind, "animate", number)

        n += 1
    except:
        n_loop = False

    i += 1
except:
    loop = False

# -----
# PRINT RESULTS
# -----
print("Pronoun, Noun Co-reference Resolution Results:\n")

i = 0
loop = True
while loop:
    try:
        pronoun, pos, noun = corefs[i]
        print(f"Pronoun '{pronoun}' at position {pos} → refers to '{noun}'")
        i += 1
    except:
        loop = False

```

[pseudocode]

text <- STRING(John gave his dog a bath because they were dirty. Sarah told him to use warm water. Then she asked him to clean his car as he is dirty)

pronouns <- DICTIONARY OF {
 he: (male, subject),
 him: (male, object),
 his: (male, possessive),
 she: (female, subject),
 her: (female, object),
 they: (plural, subject),
 them: (plural, object),

```

    their: (plural, possessive)
}

male_names <- LIST OF [
  STRING(john), STRING(peter), STRING(michael), STRING(david)
]

female_names <- LIST OF [
  STRING(sarah), STRING(mary), STRING(anna), STRING(julia)
]

common_nouns <- LIST OF [
  STRING(dog), STRING(cat), STRING(car), STRING(bike),
  STRING(boat), STRING(ship), STRING(motorbike)
]

animate_nouns <- LIST OF [
  STRING(dog), STRING(cat), STRING(man), STRING(woman),
  STRING(boy), STRING(girl)
]

anthropomorphic_nouns <- LIST OF [
  STRING(car), STRING(bike), STRING(boat),
  STRING(ship), STRING(motorbike)
]

plural_exceptions <- LIST OF [
  STRING(children), STRING(people),
  STRING(men), STRING(women)
]

FUNCTION tokenise(text)
  lowered <- LOWERCASE(text)
  cleaned <- REPLACE(lowered, STRING(.), EMPTY STRING)
  RETURN SPLIT(cleaned)
ENDFUNCTION

words <- tokenise(text)

nouns <- EMPTY LIST

i <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
  TRY
    w <- words[i]

```

```
found <- BOOLEAN(False)
number <- STRING(singular)
```

```
e <- 0
e_loop <- BOOLEAN(True)
```

```
WHILE e_loop == BOOLEAN(True) DO
  TRY
    IF w == plural_exceptions[e] THEN
      number <- STRING(plural)
      BREAK
    ENDIF
    e <- e + 1
  CATCH
    e_loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE
```

```
IF number == STRING(singular) THEN
  IF LENGTH OF w > 1 AND w ENDSWITH STRING(s) THEN
    number <- STRING(plural)
  ENDIF
ENDIF
```

```
m <- 0
m_loop <- BOOLEAN(True)
```

```
WHILE m_loop == BOOLEAN(True) DO
  TRY
    IF w == male_names[m] THEN
      APPEND (w, i, STRING(male), STRING(name), STRING(animate), number) TO nouns
      found <- BOOLEAN(True)
      m_loop <- BOOLEAN(False)
    ENDIF
    m <- m + 1
  CATCH
    m_loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE
```

```
IF found == BOOLEAN(False) THEN
  f <- 0
  f_loop <- BOOLEAN(True)
```

```
WHILE f_loop == BOOLEAN(True) DO
  TRY
    IF w == female_names[f] THEN
      APPEND (w, i, STRING(female), STRING(name), STRING(animate), number) TO nouns
      found <- BOOLEAN(True)
```

```

        f_loop <- BOOLEAN(False)
    ENDIF
    f <- f + 1
CATCH
    f_loop <- BOOLEAN(False)
ENDTRY
ENDWHILE
ENDIF

```

```

IF found == BOOLEAN(False) THEN
    c <- 0
    c_loop <- BOOLEAN(True)

```

```

WHILE c_loop == BOOLEAN(True) DO
    TRY
        IF w == common_nouns[c] THEN
            animacy <- STRING(inanimate)

```

```

        a <- 0
        a_loop <- BOOLEAN(True)

```

```

WHILE a_loop == BOOLEAN(True) DO
    TRY
        IF w == animate_nouns[a] THEN
            animacy <- STRING(animate)
            a_loop <- BOOLEAN(False)
        ENDIF
        a <- a + 1
    CATCH
        a_loop <- BOOLEAN(False)
    ENDTRY
ENDWHILE

```

```

h <- 0
h_loop <- BOOLEAN(True)

```

```

WHILE h_loop == BOOLEAN(True) DO
    TRY
        IF w == anthropomorphic_nouns[h] THEN
            animacy <- STRING(anthro)
            h_loop <- BOOLEAN(False)
        ENDIF
        h <- h + 1
    CATCH
        h_loop <- BOOLEAN(False)
    ENDTRY
ENDWHILE

```

```

APPEND (w, i, STRING(unknown), STRING(noun), animacy, number) TO nouns

```

```
        found <- BOOLEAN(True)
        c_loop <- BOOLEAN(False)
    ENDIF
```

```
        c <- c + 1
    CATCH
        c_loop <- BOOLEAN(False)
    ENDTRY
ENDWHILE
ENDIF
```

```
    i <- i + 1
CATCH
    loop <- BOOLEAN(False)
ENDTRY
ENDWHILE
```

```
i <- 0
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO
    TRY
        IF words[i] == STRING(his) OR words[i] == STRING(her) THEN
            noun_word <- words[i + 1]
```

```
        n <- 0
        n_loop <- BOOLEAN(True)
```

```
        WHILE n_loop == BOOLEAN(True) DO
            TRY
                w, idx, g, kind, anim, number <- nouns[n]
```

```
                IF w == noun_word THEN
                    new_gender <- STRING(male)
```

```
                IF words[i] == STRING(her) THEN
                    new_gender <- STRING(female)
                ENDIF
```

```
                nouns[n] <- (w, idx, new_gender, kind, STRING(animate), number)
            ENDIF
```

```
                n <- n + 1
            CATCH
                n_loop <- BOOLEAN(False)
            ENDTRY
        ENDWHILE
    ENDIF
```

```

    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

```

```

corefs <- EMPTY LIST

```

```

i <- 0
loop <- BOOLEAN(True)

```

```

WHILE loop == BOOLEAN(True) DO
  TRY
    w <- words[i]

```

```

    keys <- LIST OF KEYS IN pronouns
    p <- 0
    p_loop <- BOOLEAN(True)
    is_pronoun <- BOOLEAN(False)
    gender <- None
    role <- None

```

```

    WHILE p_loop == BOOLEAN(True) DO
      TRY
        IF w == keys[p] THEN
          is_pronoun <- BOOLEAN(True)
          gender, role <- pronouns[w]
          p_loop <- BOOLEAN(False)
        ENDIF
        p <- p + 1
      CATCH
        p_loop <- BOOLEAN(False)
      ENDTRY
    ENDWHILE

```

```

    IF is_pronoun == BOOLEAN(True) THEN
      best_noun <- None
      best_dist <- 9999

```

```

      IF gender == STRING(plural) THEN
        n <- 0
        n_loop <- BOOLEAN(True)

```

```

        WHILE n_loop == BOOLEAN(True) DO
          TRY
            noun_word, noun_idx, noun_gender, noun_kind, animacy, number <- nouns[n]

```

```

IF noun_idx < i AND number == STRING(plural) THEN
  dist <- i - noun_idx
  IF dist < best_dist THEN
    best_dist <- dist
    best_noun <- noun_word
  ENDIF
ENDIF

n <- n + 1
CATCH
  n_loop <- BOOLEAN(False)
ENDTRY
ENDWHILE

IF best_noun == None THEN
  group <- EMPTY LIST
  n <- 0
  n_loop <- BOOLEAN(True)

  WHILE n_loop == BOOLEAN(True) DO
    TRY
      noun_word, noun_idx, noun_gender, noun_kind, animacy, number <- nouns[n]

      IF noun_idx < i AND number == STRING(singular) THEN
        APPEND (noun_word, noun_idx) TO group
      ENDIF

      n <- n + 1
    CATCH
      n_loop <- BOOLEAN(False)
    ENDTRY
  ENDWHILE

  IF LENGTH OF group >= 2 THEN
    nearest <- None
    best_dist <- 9999

    g <- 0
    g_loop <- BOOLEAN(True)

    WHILE g_loop == BOOLEAN(True) DO
      TRY
        noun_word, noun_idx <- group[g]
        dist <- i - noun_idx

        IF dist < best_dist THEN
          best_dist <- dist
          nearest <- noun_word
        ENDIF
      ENDTRY
    ENDWHILE
  ENDIF

```

```

        g <- g + 1
    CATCH
        g_loop <- BOOLEAN(False)
    ENDTRY
ENDWHILE

```

```

        best_noun <- nearest
    ENDIF
ENDIF

```

```
ELSE
```

```

    IF role == STRING(subject) THEN
        n <- 0
        n_loop <- BOOLEAN(True)

```

```

    WHILE n_loop == BOOLEAN(True) DO
        TRY

```

```

            noun_word, noun_idx, noun_gender, noun_kind, animacy, number <- nouns[n]

```

```

            IF noun_idx < i AND number == STRING(singular) THEN

```

```

                IF gender == STRING(male) AND animacy == STRING(animate) THEN

```

```

                    dist <- i - noun_idx

```

```

                    IF dist < best_dist THEN

```

```

                        best_dist <- dist

```

```

                        best_noun <- noun_word

```

```

                    ENDIF

```

```

                ENDIF

```

```

                IF gender == STRING(female) AND (animacy == STRING(animate) OR animacy == STRING(anthro))

```

```
THEN
```

```

                    dist <- i - noun_idx

```

```

                    IF dist < best_dist THEN

```

```

                        best_dist <- dist

```

```

                        best_noun <- noun_word

```

```

                    ENDIF

```

```

                ENDIF

```

```

            ENDIF

```

```

            n <- n + 1

```

```

        CATCH

```

```

            n_loop <- BOOLEAN(False)

```

```

        ENDTRY

```

```

    ENDWHILE

```

```
ELSE
```

```

    n <- 0

```



```

n_loop <- BOOLEAN(True)

WHILE n_loop == BOOLEAN(True) DO
  TRY
    noun_word, noun_idx, noun_gender, noun_kind, animacy, number <- nouns[n]

    IF noun_idx < i AND noun_kind == STRING(name) AND number == STRING(singular) THEN
      IF noun_gender == gender OR noun_gender == STRING(unknown) THEN
        dist <- i - noun_idx
        IF dist < best_dist THEN
          best_dist <- dist
          best_noun <- noun_word
        ENDIF
      ENDIF
    ENDIF

    n <- n + 1
  CATCH
    n_loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

IF best_noun == None THEN
  best_dist <- 9999
  n <- 0
  n_loop <- BOOLEAN(True)

  WHILE n_loop == BOOLEAN(True) DO
    TRY
      noun_word, noun_idx, noun_gender, noun_kind, animacy, number <- nouns[n]

      IF noun_idx < i AND animacy == STRING(animate) AND number == STRING(singular) THEN
        dist <- i - noun_idx
        IF dist < best_dist THEN
          best_dist <- dist
          best_noun <- noun_word
        ENDIF
      ENDIF

      n <- n + 1
    CATCH
      n_loop <- BOOLEAN(False)
    ENDTRY
  ENDDWHILE
ENDIF
ENDIF
ENDIF

APPEND (w, i, best_noun) TO corefs

```

```

n <- 0
n_loop <- BOOLEAN(True)

WHILE n_loop == BOOLEAN(True) DO
  TRY
    noun_word, noun_idx, noun_gender, noun_kind, animacy, number <- nouns[n]

    IF noun_word == best_noun THEN
      IF noun_gender == STRING(unknown) THEN
        nouns[n] <- (noun_word, noun_idx, gender, noun_kind, STRING(animate), number)
      ENDIF

      IF animacy == STRING(anthro) THEN
        nouns[n] <- (noun_word, noun_idx, gender, noun_kind, STRING(animate), number)
      ENDIF
    ENDIF

    n <- n + 1
  CATCH
    n_loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE
ENDIF

i <- i + 1
CATCH
  loop <- BOOLEAN(False)
ENDTRY
ENDWHILE

```

OUTPUT STRING(Pronoun, Noun Co-reference Resolution Results:)

```

i <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
  TRY
    pronoun, pos, noun <- corefs[i]
    OUTPUT STRING(Pronoun '{pronoun}' at position {pos} -> refers to '{noun}')
    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

```

32. Build a question answering system to answer user queries based on a given text

[python code]

```
def clean_text(t):
    t = t.replace("'", "")
    t = t.replace('"', "")
    t = t.replace("`", "")
    t = t.replace("'''", "")
    t = t.replace("—", "-")
    t = t.replace("-", "-")
    t = t.replace("...", "...")
    return t

# =====
# RAW TEXT (PASTE HERE)
# =====

raw_text = "The Mole had been working very hard all the morning, spring-cleaning his little home. First with brooms, then with dusters; then on ladders and steps and chairs, with a brush and a pail of whitewash; till he had dust in his throat and eyes, and splashes of whitewash all over his black fur, and an overall spring-feeling in his air and heart. Spring was moving in the air above and in the earth below and around him, penetrating even his dark and lowly little house with its spirit of divine discontent and longing. It was small wonder, then, that he suddenly flung his brush on the floor, said 'Bother!' and 'O blow!' and also 'Hang spring-cleaning!' and bolted out of the house. He scraped and scratched and scrabbled and poked, working his little way up through the soft earth, till at last, pop! his nose came out into the sunlight, and he found himself rolling in the warm grass of a great meadow. 'This is fine!' he said to himself. 'This is better than whitewash!'. The sunshine struck hot on his fur, the soft breezes caressed his heated brow, and after the lonely very long time he had spent in his underground home, he felt his heart leap up with joy. He scrambled out of the hole, set off across the meadow, and wandered aimlessly towards the trees and the sparkling water in the distance. He had never seen a river before. It was a sleek, full-bodied animal, chasing and chuckling, gripping things with a gurgling voice, laughing and leaping. It was an otter, a badger, or a water-rat—though he did not know it at the time. He sat on the bank, fascinated by the sight, until he saw a dark hole in the opposite bank right above the water, with a comfortable-looking chair, and fishing poles sticking out. As he sat looking at it, a tiny, bright-eyed face appeared and vanished, then reappeared again. It was the Water Rat. The Rat looked at the Mole, and the Mole looked at the Rat. 'Hullo, Mole!' said the Water Rat. 'Hullo, Rat!' said the Mole. 'Would you like to come over?' enquired the Rat presently. 'Oh, it's all very well to say come over,' replied the Mole, who was a stranger to the river, and rather bewildered. The Rat stepped into a little boat which the Mole had not observed. It was painted blue outside and white within, and looked as neat as a new watch. He sculled across with easy, practiced strokes, and held out a paw as the Mole stepped in. 'It's a wonderful life!' said the Rat. 'There is nothing—absolutely nothing—half so much worth doing as simply messing about in boats.'"

text = clean_text(raw_text)
question = input("Enter your question:\n")

# =====
# BASIC TOKENISATION + SENTENCE SPLITTING
# =====
```

```
def split_sentences(text):
    temp = text.replace("?", ".").replace("!", ".")
    parts = temp.split(".")
    sentences = []
    i = 0
    loop = True
    while loop:
        try:
            s = parts[i].strip()
            if len(s) > 0:
                sentences.append(s)
            i += 1
        except:
            loop = False
    return sentences
```

```
sentences = split_sentences(text)
```

```
def tokenise(text):
    return text.lower().replace(".", "").replace(",", "").replace(";", "").replace(":", "").split()
```

```
q_words = tokenise(question)
```

```
stopwords = ["the", "a", "an", "and", "or", "to", "of", "in", "on", "for", "with", "as", "is", "was", "were",
              "why", "what", "who", "how", "when", "where", "did", "does", "do", "too", "into", "at", "by", "from"]
```

```
# =====
# DEV vs PROD TOGGLE
# =====
```

```
USE_API = False # True = production, False = auto-generate
```

```
# =====
# API STUBS (REPLACE IN PRODUCTION)
# =====
```

```
def api_get_animals(): return ["mole", "rat", "otter", "badger", "water-rat"]
def api_get_places(): return ["meadow", "bank", "river", "house"]
def api_get_verbs(): return ["scrambled", "wandered", "rolled", "bolted"]
def api_get_nouns(): return ["mole", "river", "meadow", "boat", "bank"]
def api_get_whowords(): return ["mole", "rat", "badger", "otter"]
def api_get_whywords(): return ["because", "since", "as", "therefore", "so"]
def api_get_whenwords(): return ["morning", "spring", "day", "evening", "night"]
```

```

# =====
# AUTO-GENERATION HELPERS
# =====

all_words = []
i = 0
loop = True
while loop:
    try:
        all_words.append(tokenise(text)[i])
        i += 1
    except:
        loop = False

def auto_generate_verbs():
    verbs = []
    i = 0
    loop = True
    while loop:
        try:
            w = all_words[i]
            if w.endswith("ing") or w.endswith("ed"):
                verbs.append(w)
            i += 1
        except:
            loop = False
    return verbs

def auto_generate_nouns():
    nouns = []
    i = 0
    loop = True
    while loop:
        try:
            w = all_words[i]
            if not w.endswith("ing") and not w.endswith("ed"):
                nouns.append(w)
            i += 1
        except:
            loop = False
    return nouns

def auto_generate_whowords():
    who = []
    raw_words = raw_text.replace(";", " ").replace(":", " ").split()
    i = 0

```

```

loop = True
while loop:
    try:
        w = raw_words[i]
        if len(w) > 0 and w[0].isupper():
            lw = w.lower().replace(", ", "").replace(".", "").replace("!", "").replace("?", "")
            if lw not in ["i", "he", "she", "they", "it"]:
                who.append(lw)
        i += 1
    except:
        loop = False
return who

```

```

def auto_generate_whywords():
    why = []
    i = 0
    loop = True
    while loop:
        try:
            w = all_words[i]
            if w in ["because", "since", "as", "therefore", "so", "till", "until"]:
                why.append(w)
            i += 1
        except:
            loop = False
    return why

```

```

def auto_generate_whenwords():
    when = []
    i = 0
    loop = True
    while loop:
        try:
            w = all_words[i]
            if w in ["then", "after", "before", "later", "earlier", "day", "night", "time", "morning", "evening", "spring"]:
                when.append(w)
            i += 1
        except:
            loop = False
    return when

```

```

def auto_generate_places():
    endings = []
    i = 0
    loop = True
    while loop:

```

```

try:
    w = all_words[i]
    if w.endswith("valley") or w.endswith("land") or w.endswith("road") or w.endswith("house") or
w.endswith("place") or w.endswith("meadow") or w.endswith("bank"):
        j = 0
        found = False
        while j < len(w) and not found:
            if w[j:] in ["valley", "land", "road", "house", "place", "meadow", "bank"]:
                endings.append(w[j:])
                found = True
                break
            j += 1
        i += 1
except:
    loop = False

```

```

places = []
i = 0
loop = True
while loop:
    try:
        w = all_words[i]
        j = 0
        j_loop = True
        while j_loop:
            try:
                if w.endswith(endings[j]):
                    places.append(w)
                    j += 1
            except:
                j_loop = False
        i += 1
    except:
        loop = False

```

```

return places

```

```

def auto_generate_animals():
    animals = []

    raw_words_clean = raw_text.replace(",", " , ").replace(".", " . ").replace(";", " ; ").replace(":", " : ").replace("!", " ! ").replace("?", " ? ")
    raw_words_list = raw_words_clean.split()

```

```

def looks_like_animal(w):
    w = w.lower()
    if w.endswith("mole"): return True
    if w.endswith("rat"): return True

```

```
if w.endswith("otter"): return True
if w.endswith("badger"): return True
if w == "animal": return True
return False
```

```
i = 0
loop = True
while loop:
    try:
        if raw_words_list[i].lower() == "was" and i > 0 and raw_words_list[i-1].lower() == "it":
            if i+1 < len(raw_words_list):
                nxt = raw_words_list[i+1].lower()
                if looks_like_animal(nxt):
                    animals.append(nxt)
            i += 1
    except:
        loop = False
```

```
raw_words = raw_text.replace(";", " ").replace(":", " ").split()
i = 0
loop = True
while loop:
    try:
        w = raw_words[i]
        if len(w) > 0 and w[0].isupper():
            lw = w.lower().replace(", ", "").replace(".", "").replace("!", "").replace("?", "")
            if looks_like_animal(lw):
                animals.append(lw)
            i += 1
    except:
        loop = False
```

```
i = 0
loop = True
while loop:
    try:
        w = all_words[i]
        if looks_like_animal(w):
            animals.append(w)
        i += 1
    except:
        loop = False
```

```
unique = []
i = 0
loop = True
while loop:
    try:
        if animals[i] not in unique:
```



```

        unique.append(animals[i])
    i += 1
except:
    loop = False

return unique

# =====
# CATEGORY REGISTRY
# =====

CATEGORIES = {
    "animals": {"api": api_get_animals, "auto": auto_generate_animals},
    "places": {"api": api_get_places, "auto": auto_generate_places},
    "verbs": {"api": api_get_verbs, "auto": auto_generate_verbs},
    "nouns": {"api": api_get_nouns, "auto": auto_generate_nouns},
    "who": {"api": api_get_whowords, "auto": auto_generate_whowords},
    "why": {"api": api_get_whywords, "auto": auto_generate_whywords},
    "when": {"api": api_get_whenwords, "auto": auto_generate_whenwords}
}

# =====
# LOAD ALL LISTS (MANUAL LOOPS)
# =====

LISTS = {}

category_names = []
keys = list(CATEGORIES.keys())
i = 0
loop = True
while loop:
    try:
        category_names.append(keys[i])
        i += 1
    except:
        loop = False

i = 0
loop = True
while loop:
    try:
        name = category_names[i]
        if USE_API:
            LISTS[name] = CATEGORIES[name]["api"]()
        else:
            LISTS[name] = CATEGORIES[name]["auto"]()

```

```
    i += 1
except:
    loop = False
```

```
# =====
# SYNONYMS (SUFFIX-BASED)
# =====
```

```
synonyms = []
i = 0
loop = True
while loop:
    try:
        w = all_words[i]
        if w.endswith("s") and len(w) > 1:
            synonyms.append((w, w[:-1]))
        if w.endswith("ed") and len(w) > 2:
            synonyms.append((w, w[:-2]))
        if w.endswith("ing") and len(w) > 3:
            synonyms.append((w, w[:-3]))
        i += 1
    except:
        loop = False
```

```
def words_match(w1, w2):
    if w1 == w2:
        return True
    i = 0
    loop = True
    while loop:
        try:
            a, b = synonyms[i]
            if (w1 == a and w2 == b) or (w1 == b and w2 == a):
                return True
            i += 1
        except:
            loop = False
    return False
```

```
# =====
# QUESTION TYPE + MAIN NOUN + VERBS
# =====
```

```
question_type = "other"
```

```
if len(q_words) > 0:
    if q_words[0] == "where":
```

```

        question_type = "where"
    elif q_words[0] == "who":
        question_type = "who"
    elif q_words[0] == "why":
        question_type = "why"
    elif q_words[0] == "when":
        question_type = "when"
    elif q_words[0] == "what":
        question_type = "what"
    elif q_words[0] == "how":
        question_type = "how"

main_noun = ""
i = 0
loop = True
while loop:
    try:
        w = q_words[i]
        if w not in stopwords:
            main_noun = w
        i += 1
    except:
        loop = False

question_verbs = []
i = 0
loop = True
while loop:
    try:
        w = q_words[i]
        if w.endswith("ing") or w.endswith("ed"):
            question_verbs.append(w)
        i += 1
    except:
        loop = False

# =====
# UNIVERSAL CATEGORY DETECTION
# =====

detected_category = ""

i = 0
loop = True
while loop:
    try:
        w = q_words[i]
        j = 0

```

```

j_loop = True
while j_loop:
    try:
        cname = category_names[j]
        if w == cname or w == cname[:-1]:
            detected_category = cname
            j += 1
    except:
        j_loop = False
i += 1
except:
    loop = False

```

```

# =====
# UNIVERSAL QUERIES
# =====

```

```

if len(q_words) > 1 and q_words[0] == "how" and q_words[1] == "many" and detected_category != "":
    lst = LISTS[detected_category]

```

```

count = 0
i = 0
loop = True
while loop:
    try:
        if lst[i] != "":
            count += 1
            i += 1
    except:
        loop = False

```

```

out = ""
i = 0
loop = True
while loop:
    try:
        out += lst[i] + " "
        i += 1
    except:
        loop = False

```

```

print("\nQuestion:", question)
print("Answer:", str(count) + " " + detected_category + ": " + out.strip())
raise SystemExit

```

```

if detected_category != "":
    if len(q_words) >= 3 and q_words[0] == "are" and q_words[1] == "there":
        lst = LISTS[detected_category]

```

```

if len(lst) > 0:
    out = ""
    i = 0
    loop = True
    while loop:
        try:
            out += lst[i] + " "
            i += 1
        except:
            loop = False

    print("\nQuestion:", question)
    print("Answer: Yes — " + out.strip())
    raise SystemExit
else:
    print("\nQuestion:", question)
    print("Answer: No — there are no " + detected_category)
    raise SystemExit

```

```

if detected_category != "" and "list" in q_words:
    lst = LISTS[detected_category]

```

```

out = ""
i = 0
loop = True
while loop:
    try:
        out += lst[i] + " "
        i += 1
    except:
        loop = False

```

```

print("\nQuestion:", question)
print("Answer:", out.strip())
raise SystemExit

```

```

# =====
# SCORING ENGINE
# =====

```

```

animals = LISTS["animals"]
location_words = LISTS["places"]
who_words = LISTS["who"]
why_words = LISTS["why"]
when_words = LISTS["when"]

```

```

scores = []

```

```

i = 0
loop = True
while loop:
    try:
        sent = sentences[i]
        s_words = tokenise(sent)
        score = 0

        q = 0
        q_loop = True
        while q_loop:
            try:
                qw = q_words[q]
                if qw not in stopwords:
                    s = 0
                    s_loop = True
                    while s_loop:
                        try:
                            if words_match(s_words[s], qw):
                                score += 1
                                s += 1
                        except:
                            s_loop = False
                    q += 1
            except:
                q_loop = False

        v = 0
        v_loop = True
        while v_loop:
            try:
                verb = question_verbs[v]
                s = 0
                s_loop = True
                while s_loop:
                    try:
                        if words_match(s_words[s], verb):
                            score += 4
                            s += 1
                    except:
                        s_loop = False
                v += 1
            except:
                v_loop = False

        s = 0
        s_loop = True
        while s_loop:

```

```
try:
    w = s_words[s]

    if question_type == "where":
        j = 0
        j_loop = True
        while j_loop:
            try:
                if w == location_words[j]:
                    score += 2
                    j += 1
            except:
                j_loop = False
```

```
if question_type == "who":
    j = 0
    j_loop = True
    while j_loop:
        try:
            if w == who_words[j]:
                score += 2
                j += 1
        except:
            j_loop = False
```

```
if question_type == "why":
    j = 0
    j_loop = True
    while j_loop:
        try:
            if w == why_words[j]:
                score += 2
                j += 1
        except:
            j_loop = False
```

```
if question_type == "when":
    j = 0
    j_loop = True
    while j_loop:
        try:
            if w == when_words[j]:
                score += 2
                j += 1
        except:
            j_loop = False
```

```
if question_type == "what":
    if w == main_noun:
```

```

        score += 4
    if w == "for":
        score += 3
    if w == "were" or w == "there":
        score += 2
    if w == "used":
        score += 3

if len(q_words) > 0 and q_words[0] in ["did", "does", "do"]:
    subject = ""
    i2 = 1
    loop2 = True
    while loop2:
        try:
            if q_words[i2] not in stopwords:
                subject = q_words[i2]
                loop2 = False
            i2 += 1
        except:
            loop2 = False

    obj = ""
    i2 = 0
    loop2 = True
    while loop2:
        try:
            if q_words[i2] not in stopwords:
                obj = q_words[i2]
                i2 += 1
        except:
            loop2 = False

    s2 = 0
    loop2 = True
    while loop2:
        try:
            if s_words[s2] == subject:
                score += 3
            if s_words[s2] == obj:
                score += 4
            s2 += 1
        except:
            loop2 = False

    s2 = 0
    loop2 = True
    while loop2:
        try:
            if s_words[s2] in ["was", "were", "had", "approached", "contained", "included"]:

```



```
        score += 2
        s2 += 1
    except:
        loop2 = False
```

```
    s += 1
except:
    s_loop = False
```

```
if i > 0:
    prev_words = tokenise(sentences[i - 1])
    q = 0
    q_loop = True
    while q_loop:
        try:
            qw = q_words[q]
            if qw not in stopwords:
                p = 0
                p_loop = True
                while p_loop:
                    try:
                        if words_match(prev_words[p], qw):
                            score += 1
                            p += 1
                    except:
                        p_loop = False
                q += 1
        except:
            q_loop = False
```

```
if i < len(sentences) - 1:
    next_words = tokenise(sentences[i + 1])
    q = 0
    q_loop = True
    while q_loop:
        try:
            qw = q_words[q]
            if qw not in stopwords:
                n2 = 0
                n2_loop = True
                while n2_loop:
                    try:
                        if words_match(next_words[n2], qw):
                            score += 1
                            n2 += 1
                    except:
                        n2_loop = False
                q += 1
        except:
```

```

        q_loop = False

    scores.append((score, sent))
    i += 1
except:
    loop = False

# =====
# PICK BEST SENTENCE
# =====

best_score = -1
best_sentence = None
i = 0
loop = True
while loop:
    try:
        sc, sent = scores[i]
        if sc > best_score:
            best_score = sc
            best_sentence = sent
        i += 1
    except:
        loop = False

# =====
# ANSWER REWRITING
# =====

def rewrite_answer(sentence, question_type):
    words = tokenise(sentence)

    if len(words) > 0:
        if words[0] == "because":
            words = words[1:]

    if question_type == "where":
        i = 0
        loop = True
        while loop:
            try:
                if words[i] == "at":
                    words[i] = "in"
                i += 1
            except:
                loop = False

```

```
if question_type == "why":
    i = 0
    loop = True
    start = 0
    while loop:
        try:
            if words[i] == "because":
                start = i
                i += 1
        except:
            loop = False
    words = words[start:]
```

```
if question_type == "who":
    if len(words) > 8:
        new_words = []
        j = 0
        loop2 = True
        while loop2:
            try:
                if j < 8:
                    new_words.append(words[j])
                    j += 1
            except:
                loop2 = False
        words = new_words
```

```
if question_type == "what":
    i = 0
    loop = True
    start = 0
    found = False
    while loop:
        try:
            if words[i] == "for":
                start = i
                found = True
                i += 1
        except:
            loop = False
    if found:
        new_words = []
        j = start
        loop2 = True
        while loop2:
            try:
                new_words.append(words[j])
                j += 1
            except:
```

```

        loop2 = False
        words = new_words

rewritten = ""
i = 0
loop = True
while loop:
    try:
        rewritten += words[i] + " "
        i += 1
    except:
        loop = False

return rewritten.strip()

# =====
# FINAL ANSWER
# =====

answer = "I cannot answer that from the given text."
if best_sentence is not None and best_score > 0:
    answer = rewrite_answer(best_sentence, question_type)

print("\nQuestion:", question)
print("Answer:", answer)

```

[pseudocode]

```

FUNCTION clean_text(t)
    t <- REPLACE(t, STRING('), STRING('))
    t <- REPLACE(t, STRING('), STRING('))
    t <- REPLACE(t, STRING("), STRING("))
    t <- REPLACE(t, STRING("), STRING("))
    t <- REPLACE(t, STRING(—), STRING(-))
    t <- REPLACE(t, STRING(-), STRING(-))
    t <- REPLACE(t, STRING(...), STRING(...))
    RETURN t
ENDFUNCTION

raw_text <- STRING(The Mole had been working very hard all the morning...)
text <- clean_text(raw_text)

question <- INPUT(STRING(Enter your question:))

FUNCTION split_sentences(text)
    temp <- REPLACE(text, STRING(?), STRING(.))
    temp <- REPLACE(temp, STRING(!), STRING(.))
    parts <- SPLIT(temp, STRING(.))

```

```

sentences <- EMPTY LIST
i <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
  TRY
    s <- TRIM(parts[i])
    IF LENGTH OF s > 0 THEN
      APPEND s TO sentences
    ENDIF
    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

RETURN sentences
ENDFUNCTION

sentences <- split_sentences(text)

FUNCTION tokenise(text)
  temp <- LOWERCASE(text)
  temp <- REPLACE(temp, STRING(.), EMPTY STRING)
  temp <- REPLACE(temp, STRING(,), EMPTY STRING)
  temp <- REPLACE(temp, STRING(;), EMPTY STRING)
  temp <- REPLACE(temp, STRING(:), EMPTY STRING)
  RETURN SPLIT(temp)
ENDFUNCTION

q_words <- tokenise(question)

stopwords <- LIST OF [
  STRING(the), STRING(a), STRING(an), STRING(and), STRING(or),
  STRING(to), STRING(of), STRING(in), STRING(on), STRING(for),
  STRING(with), STRING(as), STRING(is), STRING(was), STRING(were),
  STRING(why), STRING(what), STRING(who), STRING(how),
  STRING(when), STRING(where), STRING(did), STRING(does),
  STRING(do), STRING(too), STRING(into), STRING(at),
  STRING(by), STRING(from)
]

USE_API <- BOOLEAN(False)

FUNCTION api_get_animals() RETURN LIST OF [STRING(mole), STRING(rat), STRING(otter), STRING(badger),
STRING(water-rat)]
FUNCTION api_get_places() RETURN LIST OF [STRING(meadow), STRING(bank), STRING(river), STRING(house)]

```

```

FUNCTION api_get_verbs() RETURN LIST OF [STRING(scrambled), STRING(wandered), STRING(rolled),
STRING(bolted)]
FUNCTION api_get_nouns() RETURN LIST OF [STRING(mole), STRING(river), STRING(meadow), STRING(boat),
STRING(bank)]
FUNCTION api_get_whowords() RETURN LIST OF [STRING(mole), STRING(rat), STRING(badger), STRING(otter)]
FUNCTION api_get_whywords() RETURN LIST OF [STRING(because), STRING(since), STRING(as), STRING(therefore),
STRING(so)]
FUNCTION api_get_whenwords() RETURN LIST OF [STRING(morning), STRING(spring), STRING(day), STRING(evening),
STRING(night)]

```

```
all_words <- EMPTY LIST
```

```
i <- 0
```

```
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO
```

```
  TRY
```

```
    APPEND tokenise(text)[i] TO all_words
```

```
    i <- i + 1
```

```
  CATCH
```

```
    loop <- BOOLEAN(False)
```

```
  ENDTRY
```

```
ENDWHILE
```

```
FUNCTION auto_generate_verbs()
```

```
  verbs <- EMPTY LIST
```

```
  i <- 0
```

```
  loop <- BOOLEAN(True)
```

```
  WHILE loop == BOOLEAN(True) DO
```

```
    TRY
```

```
      w <- all_words[i]
```

```
      IF w ENDSWITH STRING(ing) OR w ENDSWITH STRING(ed) THEN
```

```
        APPEND w TO verbs
```

```
      ENDIF
```

```
      i <- i + 1
```

```
    CATCH
```

```
      loop <- BOOLEAN(False)
```

```
    ENDTRY
```

```
  ENDWHILE
```

```
  RETURN verbs
```

```
ENDFUNCTION
```

```
FUNCTION auto_generate_nouns()
```

```
  nouns <- EMPTY LIST
```

```
  i <- 0
```

```
  loop <- BOOLEAN(True)
```

```
  WHILE loop == BOOLEAN(True) DO
```

```
    TRY
```

```
      w <- all_words[i]
```

```
      IF NOT w ENDSWITH STRING(ing) AND NOT w ENDSWITH STRING(ed) THEN
```

```

        APPEND w TO nouns
    ENDIF
    i <- i + 1
CATCH
    loop <- BOOLEAN(False)
ENDTRY
ENDWHILE
RETURN nouns
ENDFUNCTION

```

```

FUNCTION auto_generate_whowords()
    who <- EMPTY LIST
    raw_words <- SPLIT(REPLACE(REPLACE(raw_text, STRING(;), STRING( )), STRING(:), STRING( )))
    i <- 0
    loop <- BOOLEAN(True)
    WHILE loop == BOOLEAN(True) DO
        TRY
            w <- raw_words[i]
            IF LENGTH OF w > 0 AND FIRSTCHAR(w) IS UPPERCASE THEN
                lw <- LOWERCASE(w)
                lw <- REPLACE(lw, STRING(,), EMPTY STRING)
                lw <- REPLACE(lw, STRING(.), EMPTY STRING)
                lw <- REPLACE(lw, STRING(!), EMPTY STRING)
                lw <- REPLACE(lw, STRING(?), EMPTY STRING)
                IF lw NOT IN LIST OF [STRING(i), STRING(he), STRING(she), STRING(they), STRING(it)] THEN
                    APPEND lw TO who
                ENDIF
            ENDIF
            i <- i + 1
        CATCH
            loop <- BOOLEAN(False)
        ENDTRY
    ENDWHILE
    RETURN who
ENDFUNCTION

```

```

FUNCTION auto_generate_whywords()
    why <- EMPTY LIST
    i <- 0
    loop <- BOOLEAN(True)
    WHILE loop == BOOLEAN(True) DO
        TRY
            w <- all_words[i]
            IF w IN LIST OF [STRING(because), STRING(since), STRING(as), STRING(therefore), STRING(so), STRING(till),
STRING(until)] THEN
                APPEND w TO why
            ENDIF
            i <- i + 1
        CATCH
            loop <- BOOLEAN(False)
        ENDTRY
    ENDWHILE
    RETURN why
ENDFUNCTION

```

```

CATCH
  loop <- BOOLEAN(False)
ENDTRY
ENDWHILE
RETURN why
ENDFUNCTION

```

```

FUNCTION auto_generate_whenwords()
  when <- EMPTY LIST
  i <- 0
  loop <- BOOLEAN(True)
  WHILE loop == BOOLEAN(True) DO
    TRY
      w <- all_words[i]
      IF w IN LIST OF [
        STRING(then), STRING(after), STRING(before), STRING(later),
        STRING(earlier), STRING(day), STRING(night), STRING(time),
        STRING(morning), STRING(evening), STRING(spring)
      ] THEN
        APPEND w TO when
      ENDIF
      i <- i + 1
    CATCH
      loop <- BOOLEAN(False)
    ENDTRY
  ENDWHILE
  RETURN when
ENDFUNCTION

```

```

FUNCTION auto_generate_places()
  endings <- EMPTY LIST
  i <- 0
  loop <- BOOLEAN(True)
  WHILE loop == BOOLEAN(True) DO
    TRY
      w <- all_words[i]
      IF w ENDSWITH ANY OF [
        STRING(valley), STRING(land), STRING(road),
        STRING(house), STRING(place), STRING(meadow), STRING(bank)
      ] THEN
        j <- 0
        found <- BOOLEAN(False)
        WHILE j < LENGTH OF w AND found == BOOLEAN(False) DO
          TRY
            suffix <- SUBSTRING(w, j, LENGTH OF w)
            IF suffix IN LIST OF [
              STRING(valley), STRING(land), STRING(road),
              STRING(house), STRING(place), STRING(meadow), STRING(bank)
            ] THEN

```



```

        APPEND suffix TO endings
        found <- BOOLEAN(True)
        BREAK
    ENDIF
    j <- j + 1
CATCH
    j <- LENGTH OF w
ENDTRY
ENDWHILE
ENDIF
i <- i + 1
CATCH
    loop <- BOOLEAN(False)
ENDTRY
ENDWHILE

places <- EMPTY LIST
i <- 0
loop <- BOOLEAN(True)
WHILE loop == BOOLEAN(True) DO
    TRY
        w <- all_words[i]
        j <- 0
        j_loop <- BOOLEAN(True)
        WHILE j_loop == BOOLEAN(True) DO
            TRY
                IF w ENDSWITH endings[j] THEN
                    APPEND w TO places
                ENDIF
                j <- j + 1
            CATCH
                j_loop <- BOOLEAN(False)
            ENDTRY
        ENDWHILE
        i <- i + 1
    CATCH
        loop <- BOOLEAN(False)
    ENDTRY
ENDWHILE
RETURN places
ENDFUNCTION

```

```

FUNCTION auto_generate_animals()
    animals <- EMPTY LIST

```

```

    raw_words_clean <- raw_text
    raw_words_clean <- REPLACE(raw_words_clean, STRING(,), STRING( , ))
    raw_words_clean <- REPLACE(raw_words_clean, STRING(.), STRING( . ))
    raw_words_clean <- REPLACE(raw_words_clean, STRING(;), STRING( ; ))

```

```

raw_words_clean <- REPLACE(raw_words_clean, STRING(;), STRING( : ))
raw_words_clean <- REPLACE(raw_words_clean, STRING(!), STRING( ! ))
raw_words_clean <- REPLACE(raw_words_clean, STRING(?), STRING( ? ))

```

```

raw_words_list <- SPLIT(raw_words_clean)

```

```

FUNCTION looks_like_animal(w)
  w <- LOWERCASE(w)
  IF w ENDSWITH STRING(mole) THEN RETURN BOOLEAN(True)
  IF w ENDSWITH STRING(rat) THEN RETURN BOOLEAN(True)
  IF w ENDSWITH STRING(otter) THEN RETURN BOOLEAN(True)
  IF w ENDSWITH STRING(badger) THEN RETURN BOOLEAN(True)
  IF w == STRING(animal) THEN RETURN BOOLEAN(True)
  RETURN BOOLEAN(False)
ENDFUNCTION

```

```

i <- 0
loop <- BOOLEAN(True)
WHILE loop == BOOLEAN(True) DO
  TRY
    IF raw_words_list[i] == STRING(was) AND i > 0 AND raw_words_list[i - 1] == STRING(it) THEN
      nxt <- LOWERCASE(raw_words_list[i + 1])
      IF looks_like_animal(nxt) THEN
        APPEND nxt TO animals
      ENDIF
    ENDIF
    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

```

```

raw_words <- SPLIT(REPLACE(REPLACE(raw_text, STRING(;), STRING( )), STRING(;), STRING( )))

```

```

i <- 0
loop <- BOOLEAN(True)
WHILE loop == BOOLEAN(True) DO
  TRY
    w <- raw_words[i]
    IF FIRSTCHAR(w) IS UPPERCASE THEN
      lw <- LOWERCASE(w)
      lw <- REPLACE(lw, STRING(,), EMPTY STRING)
      lw <- REPLACE(lw, STRING(.), EMPTY STRING)
      lw <- REPLACE(lw, STRING(!), EMPTY STRING)
      lw <- REPLACE(lw, STRING(?), EMPTY STRING)
      IF looks_like_animal(lw) THEN
        APPEND lw TO animals
      ENDIF
    ENDIF
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

```

```

    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDRY
ENDWHILE

```

```

i <- 0
loop <- BOOLEAN(True)
WHILE loop == BOOLEAN(True) DO
  TRY
    w <- all_words[i]
    IF looks_like_animal(w) THEN
      APPEND w TO animals
    ENDIF
    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDRY
ENDWHILE

```

```

unique <- EMPTY LIST
i <- 0
loop <- BOOLEAN(True)
WHILE loop == BOOLEAN(True) DO
  TRY
    IF animals[i] NOT IN unique THEN
      APPEND animals[i] TO unique
    ENDIF
    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDRY
ENDWHILE

```

```

  RETURN unique
ENDFUNCTION

```

```

CATEGORIES <- DICTIONARY OF {
  animals: (api_get_animals, auto_generate_animals),
  places: (api_get_places, auto_generate_places),
  verbs: (api_get_verbs, auto_generate_verbs),
  nouns: (api_get_nouns, auto_generate_nouns),
  who: (api_get_whowords, auto_generate_whowords),
  why: (api_get_whywords, auto_generate_whywords),
  when: (api_get_whenwords, auto_generate_whenwords)
}

```

```

LISTS <- EMPTY DICTIONARY

```

```
category_names <- LIST OF KEYS IN CATEGORIES
```

```
i <- 0
loop <- BOOLEAN(True)
WHILE loop == BOOLEAN(True) DO
  TRY
    name <- category_names[i]
    IF USE_API == BOOLEAN(True) THEN
      LISTS[name] <- CATEGORIES[name].api()
    ELSE
      LISTS[name] <- CATEGORIES[name].auto()
    ENDIF
    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE
```

```
synonyms <- EMPTY LIST
```

```
i <- 0
loop <- BOOLEAN(True)
WHILE loop == BOOLEAN(True) DO
  TRY
    w <- all_words[i]
    IF w ENDSWITH STRING(s) THEN
      APPEND (w, SUBSTRING(w, 0, LENGTH OF w - 1)) TO synonyms
    ENDIF
    IF w ENDSWITH STRING(ed) THEN
      APPEND (w, SUBSTRING(w, 0, LENGTH OF w - 2)) TO synonyms
    ENDIF
    IF w ENDSWITH STRING(ing) THEN
      APPEND (w, SUBSTRING(w, 0, LENGTH OF w - 3)) TO synonyms
    ENDIF
    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE
```

```
FUNCTION words_match(w1, w2)
  IF w1 == w2 THEN RETURN BOOLEAN(True)
```

```
  i <- 0
  loop <- BOOLEAN(True)
```

```
  WHILE loop == BOOLEAN(True) DO
    TRY
```

```

    a, b <- synonyms[i]
    IF (w1 == a AND w2 == b) OR (w1 == b AND w2 == a) THEN
        RETURN BOOLEAN(True)
    ENDIF
    i <- i + 1
CATCH
    loop <- BOOLEAN(False)
ENDTRY
ENDWHILE

RETURN BOOLEAN(False)
ENDFUNCTION

```

```

question_type <- STRING(other)

```

```

IF LENGTH OF q_words > 0 THEN
    IF q_words[0] == STRING(when) THEN question_type <- STRING(when)
    IF q_words[0] == STRING(why) THEN question_type <- STRING(why)
    IF q_words[0] == STRING(what) THEN question_type <- STRING(what)
    IF q_words[0] == STRING(how) THEN question_type <- STRING(how)
ENDIF

```

```

main_noun <- EMPTY STRING

```

```

i <- 0
loop <- BOOLEAN(True)

```

```

WHILE loop == BOOLEAN(True) DO
    TRY
        w <- q_words[i]
        IF w NOT IN stopwords THEN
            main_noun <- w
        ENDIF
        i <- i + 1
    CATCH
        loop <- BOOLEAN(False)
    ENDTRY
ENDWHILE

```

```

question_verbs <- EMPTY LIST

```

```

i <- 0
loop <- BOOLEAN(True)

```

```

WHILE loop == BOOLEAN(True) DO
  TRY
    w <- q_words[i]
    IF w ENDSWITH STRING(ing) OR w ENDSWITH STRING(ed) THEN
      APPEND w TO question_verbs
    ENDIF
    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

```

```

detected_category <- EMPTY STRING

```

```

i <- 0
loop <- BOOLEAN(True)

```

```

WHILE loop == BOOLEAN(True) DO
  TRY
    w <- q_words[i]

    j <- 0
    j_loop <- BOOLEAN(True)

    WHILE j_loop == BOOLEAN(True) DO
      TRY
        cname <- category_names[j]
        IF w == cname OR w == SUBSTRING(cname, 0, LENGTH OF cname - 1) THEN
          detected_category <- cname
        ENDIF
        j <- j + 1
      CATCH
        j_loop <- BOOLEAN(False)
      ENDTRY
    ENDWHILE

    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

```

```

IF LENGTH OF q_words > 1 AND q_words[0] == STRING(how) AND q_words[1] == STRING(many) AND
detected_category != EMPTY STRING THEN
  lst <- LISTS[detected_category]

  count <- 0

```

```

i <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
  TRY
    IF lst[i] != EMPTY STRING THEN
      count <- count + 1
    ENDIF
    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

out <- EMPTY STRING
i <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
  TRY
    out <- out + STRING( ) + lst[i]
    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

OUTPUT STRING(Question: {question})
OUTPUT STRING(Answer: {count} {detected_category} {out})
STOP
ENDIF

IF detected_category != EMPTY STRING AND LENGTH OF q_words >= 3 AND
q_words[0] == STRING(are) AND q_words[1] == STRING(there) THEN

  lst <- LISTS[detected_category]

  IF LENGTH OF lst > 0 THEN
    out <- EMPTY STRING
    i <- 0
    loop <- BOOLEAN(True)

    WHILE loop == BOOLEAN(True) DO
      TRY
        out <- out + STRING( ) + lst[i]
        i <- i + 1
      CATCH
        loop <- BOOLEAN(False)
      ENDTRY
    ENDWHILE
  ENDIF
ENDIF

```

```

    ENDTRY
ENDWHILE

OUTPUT STRING(Question: {question})
OUTPUT STRING(Answer: Yes — {out})
STOP
ELSE
    OUTPUT STRING(Question: {question})
    OUTPUT STRING(Answer: No — there are no {detected_category})
    STOP
ENDIF
ENDIF

IF detected_category != EMPTY STRING AND STRING(list) IN q_words THEN
    lst <- LISTS[detected_category]

    out <- EMPTY STRING
    i <- 0
    loop <- BOOLEAN(True)

    WHILE loop == BOOLEAN(True) DO
        TRY
            out <- out + STRING( ) + lst[i]
            i <- i + 1
        CATCH
            loop <- BOOLEAN(False)
        ENDTRY
    ENDWHILE

    OUTPUT STRING(Question: {question})
    OUTPUT STRING(Answer: {out})
    STOP
ENDIF

animals <- LISTS[STRING(animals)]
location_words <- LISTS[STRING(places)]
who_words <- LISTS[STRING(who)]
why_words <- LISTS[STRING(why)]
when_words <- LISTS[STRING(when)]

scores <- EMPTY LIST

i <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
    TRY
        sent <- sentences[i]

```



```

s_words <- tokenise(sent)
score <- 0

q <- 0
q_loop <- BOOLEAN(True)

WHILE q_loop == BOOLEAN(True) DO
  TRY
    qw <- q_words[q]
    IF qw NOT IN stopwords THEN
      s <- 0
      s_loop <- BOOLEAN(True)

      WHILE s_loop == BOOLEAN(True) DO
        TRY
          IF words_match(s_words[s], qw) THEN
            score <- score + 1
          ENDIF
          s <- s + 1
        CATCH
          s_loop <- BOOLEAN(False)
        ENDTRY
      ENDWHILE
    ENDIF
    q <- q + 1
  CATCH
    q_loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

```

```

v <- 0
v_loop <- BOOLEAN(True)

WHILE v_loop == BOOLEAN(True) DO
  TRY
    verb <- question_verbs[v]
    s <- 0
    s_loop <- BOOLEAN(True)

    WHILE s_loop == BOOLEAN(True) DO
      TRY
        IF words_match(s_words[s], verb) THEN
          score <- score + 4
        ENDIF
        s <- s + 1
      CATCH
        s_loop <- BOOLEAN(False)
      ENDTRY
    ENDWHILE
  ENDTRY
ENDWHILE

```

ENDWHILE

v <- v + 1

CATCH

v_loop <- BOOLEAN(False)

ENDTRY

ENDWHILE

s <- 0

s_loop <- BOOLEAN(True)

WHILE s_loop == BOOLEAN(True) DO

TRY

w <- s_words[s]

IF question_type == STRING(when) THEN

j <- 0

j_loop <- BOOLEAN(True)

WHILE j_loop == BOOLEAN(True) DO

TRY

IF w == location_words[j] THEN

score <- score + 2

ENDIF

j <- j + 1

CATCH

j_loop <- BOOLEAN(False)

ENDTRY

ENDWHILE

ENDIF

IF question_type == STRING(who) THEN

j <- 0

j_loop <- BOOLEAN(True)

WHILE j_loop == BOOLEAN(True) DO

TRY

IF w == who_words[j] THEN

score <- score + 2

ENDIF

j <- j + 1

CATCH

j_loop <- BOOLEAN(False)

ENDTRY

ENDWHILE

ENDIF

IF question_type == STRING(why) THEN

```

j <- 0
j_loop <- BOOLEAN(True)

WHILE j_loop == BOOLEAN(True) DO
  TRY
    IF w == why_words[j] THEN
      score <- score + 2
    ENDIF
    j <- j + 1
  CATCH
    j_loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE
ENDIF

IF question_type == STRING(when) THEN
  j <- 0
  j_loop <- BOOLEAN(True)

  WHILE j_loop == BOOLEAN(True) DO
    TRY
      IF w == when_words[j] THEN
        score <- score + 2
      ENDIF
      j <- j + 1
    CATCH
      j_loop <- BOOLEAN(False)
    ENDTRY
  ENDWHILE
ENDIF

IF question_type == STRING(what) THEN
  IF w == main_noun THEN
    score <- score + 4
  ENDIF
  IF w == STRING(for) THEN
    score <- score + 3
  ENDIF
  IF w == STRING(were) OR w == STRING(there) THEN
    score <- score + 2
  ENDIF
  IF w == STRING(used) THEN
    score <- score + 3
  ENDIF
ENDIF

IF LENGTH OF q_words > 0 AND
  (q_words[0] == STRING(did) OR q_words[0] == STRING(does) OR q_words[0] == STRING(do)) THEN

```

```
subject <- EMPTY STRING
i2 <- 1
loop2 <- BOOLEAN(True)

WHILE loop2 == BOOLEAN(True) DO
  TRY
    IF q_words[i2] NOT IN stopwords THEN
      subject <- q_words[i2]
      loop2 <- BOOLEAN(False)
    ENDIF
    i2 <- i2 + 1
  CATCH
    loop2 <- BOOLEAN(False)
  ENDTRY
ENDWHILE
```

```
obj <- EMPTY STRING
i2 <- 0
loop2 <- BOOLEAN(True)
```

```
WHILE loop2 == BOOLEAN(True) DO
  TRY
    IF q_words[i2] NOT IN stopwords THEN
      obj <- q_words[i2]
    ENDIF
    i2 <- i2 + 1
  CATCH
    loop2 <- BOOLEAN(False)
  ENDTRY
ENDWHILE
```

```
s2 <- 0
loop2 <- BOOLEAN(True)
```

```
WHILE loop2 == BOOLEAN(True) DO
  TRY
    IF s_words[s2] == subject THEN
      score <- score + 3
    ENDIF
    IF s_words[s2] == obj THEN
      score <- score + 4
    ENDIF
    s2 <- s2 + 1
  CATCH
    loop2 <- BOOLEAN(False)
  ENDTRY
ENDWHILE
```

```
s2 <- 0
loop2 <- BOOLEAN(True)
```

```
WHILE loop2 == BOOLEAN(True) DO
  TRY
    IF s_words[s2] IN LIST OF [
      STRING(was), STRING(were), STRING(had),
      STRING(approached), STRING(contained), STRING(included)
    ] THEN
      score <- score + 2
    ENDIF
    s2 <- s2 + 1
  CATCH
    loop2 <- BOOLEAN(False)
  ENDTRY
ENDWHILE
ENDIF
```

```
s <- s + 1
CATCH
  s_loop <- BOOLEAN(False)
ENDTRY
ENDWHILE
```

```
IF i > 0 THEN
  prev_words <- tokenise(sentences[i - 1])
```

```
q <- 0
q_loop <- BOOLEAN(True)
```

```
WHILE q_loop == BOOLEAN(True) DO
  TRY
    qw <- q_words[q]
    IF qw NOT IN stopwords THEN
      p <- 0
      p_loop <- BOOLEAN(True)

      WHILE p_loop == BOOLEAN(True) DO
        TRY
          IF words_match(prev_words[p], qw) THEN
            score <- score + 1
          ENDIF
          p <- p + 1
        CATCH
          p_loop <- BOOLEAN(False)
        ENDTRY
      ENDWHILE
    ENDIF
  ENDTRY
ENDWHILE
ENDIF
```

```
    q <- q + 1
  CATCH
    q_loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE
ENDIF
```

```
IF i < LENGTH OF sentences - 1 THEN
  next_words <- tokenise(sentences[i + 1])
```

```
  q <- 0
  q_loop <- BOOLEAN(True)
```

```
  WHILE q_loop == BOOLEAN(True) DO
    TRY
```

```
      qw <- q_words[q]
      IF qw NOT IN stopwords THEN
        n2 <- 0
        n2_loop <- BOOLEAN(True)
```

```
        WHILE n2_loop == BOOLEAN(True) DO
```

```
          TRY
            IF words_match(next_words[n2], qw) THEN
              score <- score + 1
            ENDIF
            n2 <- n2 + 1
```

```
          CATCH
            n2_loop <- BOOLEAN(False)
```

```
          ENDTRY
```

```
        ENDWHILE
```

```
      ENDIF
```

```
      q <- q + 1
```

```
    CATCH
```

```
      q_loop <- BOOLEAN(False)
```

```
    ENDTRY
```

```
  ENDWHILE
```

```
ENDIF
```

```
APPEND (score, sent) TO scores
```

```
i <- i + 1
```

```
CATCH
```

```
  loop <- BOOLEAN(False)
```

```
ENDTRY
```

```
ENDWHILE
```

```
best_score <- -1
```

```
best_sentence <- None
```

```
i <- 0
```

```
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO
```

```
  TRY
```

```
    sc, sent <- scores[i]
```

```
    IF sc > best_score THEN
```

```
      best_score <- sc
```

```
      best_sentence <- sent
```

```
    ENDIF
```

```
    i <- i + 1
```

```
  CATCH
```

```
    loop <- BOOLEAN(False)
```

```
  ENDTRY
```

```
ENDWHILE
```

```
FUNCTION rewrite_answer(sentence, question_type)
```

```
  words <- tokenise(sentence)
```

```
  IF LENGTH OF words > 0 THEN
```

```
    IF words[0] == STRING(because) THEN
```

```
      words <- SUBLIST(words, 1, LENGTH OF words)
```

```
    ENDIF
```

```
  ENDIF
```

```
  IF question_type == STRING(when) THEN
```

```
    i <- 0
```

```
    loop <- BOOLEAN(True)
```

```
    WHILE loop == BOOLEAN(True) DO
```

```
      TRY
```

```
        IF words[i] == STRING(at) THEN
```

```
          words[i] <- STRING(in)
```

```
        ENDIF
```

```
        i <- i + 1
```

```
      CATCH
```

```
        loop <- BOOLEAN(False)
```

```
      ENDTRY
```

```
    ENDWHILE
```

```
  ENDIF
```

```
  IF question_type == STRING(why) THEN
```

```
    i <- 0
```

```
    loop <- BOOLEAN(True)
```

```
    start <- 0
```

```
    WHILE loop == BOOLEAN(True) DO
```

```
      TRY
```

```

    IF words[i] == STRING(because) THEN
        start <- i
    ENDIF
    i <- i + 1
CATCH
    loop <- BOOLEAN(False)
ENDTRY
ENDWHILE
words <- SUBLIST(words, start, LENGTH OF words)
ENDIF

```

```

IF question_type == STRING(who) THEN
    IF LENGTH OF words > 8 THEN
        new_words <- EMPTY LIST
        j <- 0
        loop2 <- BOOLEAN(True)
        WHILE loop2 == BOOLEAN(True) DO
            TRY
                IF j < 8 THEN
                    APPEND words[j] TO new_words
                ENDIF
                j <- j + 1
            CATCH
                loop2 <- BOOLEAN(False)
            ENDTRY
        ENDWHILE
        words <- new_words
    ENDIF
ENDIF

```

```

IF question_type == STRING(what) THEN
    i <- 0
    loop <- BOOLEAN(True)
    start <- 0
    found <- BOOLEAN(False)
    WHILE loop == BOOLEAN(True) DO
        TRY
            IF words[i] == STRING(for) THEN
                start <- i
                found <- BOOLEAN(True)
            ENDIF
            i <- i + 1
        CATCH
            loop <- BOOLEAN(False)
        ENDTRY
    ENDWHILE
    IF found == BOOLEAN(True) THEN
        new_words <- EMPTY LIST
        j <- start
    
```



```

loop2 <- BOOLEAN(True)
WHILE loop2 == BOOLEAN(True) DO
  TRY
    APPEND words[j] TO new_words
    j <- j + 1
  CATCH
    loop2 <- BOOLEAN(False)
  ENDTRY
ENDWHILE
words <- new_words
ENDIF
ENDIF

rewritten <- EMPTY STRING
i <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
  TRY
    rewritten <- rewritten + words[i] + STRING( )
    i <- i + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

RETURN TRIM(rewritten)
ENDFUNCTION

answer <- STRING(I cannot answer that from the given text.)

IF best_sentence != None AND best_score > 0 THEN
  answer <- rewrite_answer(best_sentence, question_type)
ENDIF

OUTPUT STRING(Question: {question})
OUTPUT STRING(Answer: {answer})

```

33. Implement a method to find the exact square root of a non negative integer using binary search rounded down to the nearest integer without using built in math functions

[python code]

```

def int_sqrt(n):
    if n < 2:
        return n

    left = 1

```

```

right = n // 2
ans = 1

while left <= right:
    mid = (left + right) // 2
    sq = mid * mid

    if sq == n:
        return mid
    elif sq < n:
        ans = mid
        left = mid + 1
    else:
        right = mid - 1

return ans

```

```

integer = 0
loop = True
while loop == True:
    try:
        integer = input("Enter a Positive Integer to find the Square Root: ")
        integer = int(integer)
        if integer <= 0:
            integer = 0
            raise ValueError()
        else:
            loop = False
    except:
        print("That's Not a Valid Number, Please Try Again!")

print(f"The Square Root of {integer} Rounded Down to the Nearest Integer = ", int_sqrt(integer))

```

[pseudocode]

```

FUNCTION int_sqrt(n)
    IF n < 2 THEN
        RETURN n
    ENDIF

    left <- 1
    right <- n INTEGER DIVISION 2
    ans <- 1

    WHILE left <= right DO
        mid <- (left + right) INTEGER DIVISION 2
        sq <- mid * mid

        IF sq == n THEN

```

```

        RETURN mid
    ELIF sq < n THEN
        ans <- mid
        left <- mid + 1
    ELSE
        right <- mid - 1
    ENDIF
ENDWHILE

RETURN ans
ENDFUNCTION

integer <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
    TRY
        integer <- INPUT(STRING(Enter a Positive Integer to find the Square Root: ))
        integer <- INTEGER(integer)

        IF integer <= 0 THEN
            integer <- 0
            RAISE ERROR
        ELSE
            loop <- BOOLEAN(False)
        ENDIF

    CATCH
        OUTPUT STRING(That's Not a Valid Number, Please Try Again!)
    ENDTRY
ENDWHILE

result <- int_sqrt(integer)

OUTPUT STRING(The Square Root of {integer} Rounded Down to the Nearest Integer = {result})

```

34. Find all elements that appear in 2 separate unsorted lists ensuring the result contains no duplicates

[python code]

```

def common(a, b):
    result = []
    i = 0
    # loop through first list
    while i < len(a):
        item = a[i]

```

```

# check if item is in second list
j = 0
found_in_b = False

while j < len(b):
    if b[j] == item:
        found_in_b = True
        break
    j += 1

# if found in b, ensure not already in result
if found_in_b:
    k = 0
    already_in_result = False

    while k < len(result):
        if result[k] == item:
            already_in_result = True
            break
        k += 1

    if not already_in_result:
        result.append(item)

i += 1

return result

```

```

firstlist = [5,8,6,3,6,9,7,0,2]
secondlist =[2,4,3,1,7,9,7,3]

```

```

print(f"{firstlist}\n{secondlist} \nCommon Elements: {common(firstlist, secondlist)}")

```

[pseudocode]

```

FUNCTION common(a, b)
    result <- EMPTY LIST
    i <- 0
    loop <- BOOLEAN(True)

    WHILE loop == BOOLEAN(True) DO
        TRY
            item <- a[i]

            j <- 0
            found_in_b <- BOOLEAN(False)

            WHILE j < LENGTH OF b DO
                IF b[j] == item THEN

```

```

        found_in_b <- BOOLEAN(True)
        BREAK
    ENDIF
    j <- j + 1
ENDWHILE

IF found_in_b == BOOLEAN(True) THEN
    k <- 0
    already_in_result <- BOOLEAN(False)

    WHILE k < LENGTH OF result DO
        IF result[k] == item THEN
            already_in_result <- BOOLEAN(True)
            BREAK
        ENDIF
        k <- k + 1
    ENDWHILE

    IF already_in_result == BOOLEAN(False) THEN
        APPEND item TO result
    ENDIF
ENDIF

i <- i + 1
CATCH
    loop <- BOOLEAN(False)
ENDTRY
ENDWHILE

RETURN result
ENDFUNCTION

firstlist <- LIST OF [5,8,6,3,6,9,7,0,2]
secondlist <- LIST OF [2,4,3,1,7,9,7,3]

OUTPUT STRING({firstlist}\n{secondlist}\nCommon Elements: {common(firstlist, secondlist)})

```

35. Design a memory buffer that holds a fixed number of items. When it runs out of space it must discard the item that hasn't been accessed for the longest time.

[python code]

```

import random

class LRUBuffer:
    def __init__(self, capacity):
        self.capacity = capacity
        self.items = []    # stored values

```

```

self.recent = []    # last-access timestamps
self.access_count = [] # NEW: number of times accessed
self.counter = 0    # global timestamp

# -----
# FIND INDEX OF ITEM
# -----
def find_index(self, value):
    index = 0
    for item in self.items:
        if item == value:
            return index
        index += 1
    return -1

# -----
# FIND LEAST RECENTLY USED INDEX
# -----
def find_lru_index(self):
    lru_index = 0
    lru_value = self.recent[0]

    index = 0
    for r in self.recent:
        if r < lru_value:
            lru_value = r
            lru_index = index
        index += 1

    return lru_index

# -----
# ACCESS ROUTINE (updates recency + count)
# -----
def access(self, value):
    index = self.find_index(value)
    if index != -1:
        self.counter += 1
        self.recent[index] = self.counter

        # increment access count
        self.access_count[index] += 1

        print("Accessed:", value, " - Access Count =", self.access_count[index])
    else:
        print("Access attempted but not found:", value)

# -----
# ADD ROUTINE (inserts or evicts)

```

```

# -----
def add(self, value):
    index = self.find_index(value)

    # If already exists, treat as access
    if index != -1:
        self.counter += 1
        self.recent[index] = self.counter
        self.access_count[index] += 1
        print("Updated existing item:", value, "Access Count =", self.access_count[index])
        return

    # If space available, append
    if len(self.items) < self.capacity:
        self.items.append(value)
        self.counter += 1
        self.recent.append(self.counter)
        self.access_count.append(1)
        print("Added:", value, " - Access Count = 1")
        return

    # Otherwise evict LRU
    lru = self.find_lru_index()
    evicted = self.items[lru]

    self.items[lru] = value
    self.counter += 1
    self.recent[lru] = self.counter
    self.access_count[lru] = 1

    print("Evicted:", evicted, " - Added:", value, "Access Count = 1")

# -----
# VIEW CONTENTS
# -----
def get_items(self):
    return self.items

buf = LRUBuffer(5)

buf.add("A")
buf.add("B")
buf.add("C")
buf.add("D")
buf.add("E")

pool = ["A", "B", "C", "D", "E", "F", "G"]

for _ in range(10):

```

```

index = random.randint(0, len(pool) - 1)
value = pool[index]
buf.access(value)

buf.add("F")
buf.add("G")

print("Final buffer:", buf.get_items())

```

[pseudocode]

CLASS LRUBuffer

CONSTRUCTOR ACCEPTS(self, capacity)

PROPERTIES(

```

    self.capacity <- capacity,
    self.items <- EMPTY LIST,
    self.recent <- EMPTY LIST,
    self.access_count <- EMPTY LIST,
    self.counter <- 0

```

)

ENDCONSTRUCTOR

FUNCTION find_index(value)

```

index <- 0
loop <- BOOLEAN(True)

```

WHILE loop == BOOLEAN(True) DO

TRY

```

    item <- self.items[index]

```

IF item == value THEN

```

    RETURN index

```

ENDIF

```

    index <- index + 1

```

CATCH

```

    loop <- BOOLEAN(False)

```

ENDTRY

ENDWHILE

RETURN -1

ENDFUNCTION

FUNCTION find_lru_index()

```

lru_index <- 0

```

```

lru_value <- self.recent[0]

```



```

index <- 0
loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
  TRY
    r <- self.recent[index]
    IF r < lru_value THEN
      lru_value <- r
      lru_index <- index
    ENDIF
    index <- index + 1
  CATCH
    loop <- BOOLEAN(False)
  ENDTRY
ENDWHILE

RETURN lru_index
ENDFUNCTION

```

```

FUNCTION access(value)
  index <- self.find_index(value)

  IF index != -1 THEN
    self.counter <- self.counter + 1
    self.recent[index] <- self.counter
    self.access_count[index] <- self.access_count[index] + 1

    OUTPUT STRING(Accessed: {value} - Access Count = {self.access_count[index]})
  ELSE
    OUTPUT STRING(Access attempted but not found: {value})
  ENDIF
ENDFUNCTION

```

```

FUNCTION add(value)
  index <- self.find_index(value)

  IF index != -1 THEN
    self.counter <- self.counter + 1
    self.recent[index] <- self.counter
    self.access_count[index] <- self.access_count[index] + 1

    OUTPUT STRING(Updated existing item: {value} Access Count = {self.access_count[index]})
  RETURN
  ENDIF

```

```

IF LENGTH OF self.items < self.capacity THEN
  APPEND value TO self.items

```

```
self.counter <- self.counter + 1
APPEND self.counter TO self.recent
APPEND 1 TO self.access_count
```

```
OUTPUT STRING(Added: {value} - Access Count = 1)
RETURN
```

```
ENDIF
```

```
lru <- self.find_lru_index()
evicted <- self.items[lru]
```

```
self.items[lru] <- value
self.counter <- self.counter + 1
self.recent[lru] <- self.counter
self.access_count[lru] <- 1
```

```
OUTPUT STRING(Evicted: {evicted} - Added: {value} Access Count = 1)
ENDFUNCTION
```

```
FUNCTION get_items()
RETURN self.items
ENDFUNCTION
```

```
ENDCLASS
```

```
buf <- NEW LRUBuffer(5)
```

```
buf.add(STRING(A))
buf.add(STRING(B))
buf.add(STRING(C))
buf.add(STRING(D))
buf.add(STRING(E))
```

```
pool <- LIST OF [STRING(A), STRING(B), STRING(C), STRING(D), STRING(E), STRING(F), STRING(G)]
```

```
i <- 0
WHILE i < 10 DO
index <- RANDOM_INT(0, LENGTH OF pool - 1)
value <- pool[index]
buf.access(value)
i <- i + 1
ENDWHILE
```

```
buf.add(STRING(F))
buf.add(STRING(G))
```

```
OUTPUT STRING(Final buffer: {buf.get_items()})
```

36. An ordered list of numbers was rotated at an unknown pivot point (e.g. [4,5,6,1,2,3]). Find the index of a target value in this list in logarithmic time

[python code]

```
print(f"Binary Search in O(log n) Time")
```

```
def rsearch(nums, target):
```

```
    left = 0
```

```
    right = len(nums) - 1
```

```
    while left <= right:
```

```
        mid = (left + right) // 2
```

```
        mid_val = nums[mid]
```

```
        if mid_val == target:
```

```
            return mid
```

```
    # Left half is sorted
```

```
    if nums[left] <= mid_val:
```

```
        if nums[left] <= target < mid_val:
```

```
            right = mid - 1
```

```
        else:
```

```
            left = mid + 1
```

```
    # Right half is sorted
```

```
    else:
```

```
        if mid_val < target <= nums[right]:
```

```
            left = mid + 1
```

```
        else:
```

```
            right = mid - 1
```

```
    return "Target Value Does Not Exist"
```

```
nums = [4, 5, 6, 1, 2, 3]
```

```
print(nums)
```

```
print(rsearch(nums, 1))
```

```
print(rsearch(nums, 5))
```

```
print(rsearch(nums, 9))
```

[pseudocode]

OUTPUT STRING(Binary Search in O(log n) Time)

FUNCTION rsearch(nums, target)

left <- 0

right <- LENGTH OF nums - 1

WHILE left <= right DO

```

mid <- (left + right) INTEGER DIVISION 2
mid_val <- nums[mid]

IF mid_val == target THEN
    RETURN mid
ENDIF

# Left half sorted
IF nums[left] <= mid_val THEN
    IF nums[left] <= target AND target < mid_val THEN
        right <- mid - 1
    ELSE
        left <- mid + 1
    ENDIF
ELSE
    IF mid_val < target AND target <= nums[right] THEN
        left <- mid + 1
    ELSE
        right <- mid - 1
    ENDIF
ENDIF
ENDWHILE

RETURN STRING(Target Value Does Not Exist)
ENDFUNCTION

```

```

nums <- LIST OF [4, 5, 6, 1, 2, 3]

```

```

OUTPUT STRING({nums})
OUTPUT STRING({rsearch(nums, 1)})
OUTPUT STRING({rsearch(nums, 5)})
OUTPUT STRING({rsearch(nums, 9)})

```

37. Implement a Stack using an Array (push/pop/peek)

[python code]

```

class Stack:
    def init(self):
        self.items = []    # underlying array
        self.top = -1      # index of top element

# -----
# INTERNAL: display stack

```

```

# -----
def show(self):
    print("Stack: ", self.items)

# -----
# PUSH — add item to top (manual append)
# -----
def push(self, value):
    # manually append (no list.append)
    new_length = len(self.items) + 1
    temp = [None] * new_length

    i = 0
    for x in self.items:
        temp[i] = x
        i += 1

    temp[self.top + 1] = value
    self.items = temp
    self.top += 1

    print("Pushed: ", value)
    self.show()

# -----
# POP — remove and return top (manual removal)
# -----
def pop(self):
    if self.is_empty():
        print("Pop attempted on empty stack")
        self.show()
        return None

    value = self.items[self.top]

    # manually shrink array
    new_length = len(self.items) - 1
    temp = [None] * new_length

    i = 0
    while i < new_length:
        temp[i] = self.items[i]
        i += 1

    self.items = temp
    self.top -= 1

    print("Popped: ", value)
    self.show()

```

return value

PEEK — view top without removing

```
def peek(self):
    if self.is_empty():
        print("Peek attempted on empty stack")
        self.show()
        return None
```

```
value = self.items[self.top]
print("Peeked: ", value)
self.show()
return value
```

IS EMPTY

```
def is_empty(self):
    return self.top == -1
```

SIZE

```
def size(self):
    return self.top + 1
```

s = Stack()

s.push(10)

s.push(20)

s.push(30)

s.peek()

s.pop()

s.peek()

s.pop()

s.pop()

s.pop()

s.peek()

[pseudocode]

CLASS Stack

CONSTRUCTOR ACCEPTS(self)

PROPERTIES{

```
        self.items <- EMPTY LIST,  
        self.top <- -1  
    )  
ENDCONSTRUCTOR
```

```
FUNCTION show()  
    OUTPUT STRING(Stack: {self.items})  
ENDFUNCTION
```

```
FUNCTION push(value)  
    new_length <- LENGTH OF self.items + 1  
    temp <- LIST OF SIZE EQUALS new_length FILLED WITH None  
  
    i <- 0  
    loop <- BOOLEAN(True)  
  
    WHILE loop == BOOLEAN(True) DO  
        TRY  
            temp[i] <- self.items[i]  
            i <- i + 1  
        CATCH  
            loop <- BOOLEAN(False)  
        ENDTRY  
    ENDWHILE  
  
    temp[self.top + 1] <- value  
    self.items <- temp  
    self.top <- self.top + 1  
  
    OUTPUT STRING(Pushed: {value})  
    self.show()  
ENDFUNCTION
```

```
FUNCTION pop()  
    IF self.is_empty() == BOOLEAN(True) THEN  
        OUTPUT STRING(Pop attempted on empty stack)  
        self.show()  
        RETURN None  
    ENDIF  
  
    value <- self.items[self.top]  
  
    new_length <- LENGTH OF self.items - 1  
    temp <- LIST OF SIZE EQUALS new_length FILLED WITH None  
  
    i <- 0
```

```

WHILE i < new_length DO
  temp[i] <- self.items[i]
  i <- i + 1
ENDWHILE

self.items <- temp
self.top <- self.top - 1

OUTPUT STRING(Popped: {value})
self.show()
RETURN value
ENDFUNCTION

```

```

FUNCTION peek()
  IF self.is_empty() == BOOLEAN(True) THEN
    OUTPUT STRING(Peek attempted on empty stack)
    self.show()
    RETURN None
  ENDIF

  value <- self.items[self.top]
  OUTPUT STRING(Peeked: {value})
  self.show()
  RETURN value
ENDFUNCTION

```

```

FUNCTION is_empty()
  RETURN self.top == -1
ENDFUNCTION

```

```

FUNCTION size()
  RETURN self.top + 1
ENDFUNCTION

```

```

ENDCLASS

```

```

s <- NEW Stack()

```

```

s.push(10)
s.push(20)
s.push(30)

```

```

s.peek()
s.pop()
s.peek()

```



```
s.pop()
s.pop()
s.pop()
s.peak()
```

[pseudocode]

CLASS Stack

```
CONSTRUCTOR ACCEPTS(self)
  PROPERTIES(
    self.items <- EMPTY LIST,
    self.top <- -1
  )
ENDCONSTRUCTOR
```

```
FUNCTION show()
  OUTPUT STRING(Stack: {self.items})
ENDFUNCTION
```

```
FUNCTION push(value)
  new_length <- LENGTH OF self.items + 1
  temp <- LIST OF SIZE EQUALS new_length FILLED WITH None
```

```
  i <- 0
  loop <- BOOLEAN(True)
```

```
  WHILE loop == BOOLEAN(True) DO
    TRY
      temp[i] <- self.items[i]
      i <- i + 1
    CATCH
      loop <- BOOLEAN(False)
    ENDMETHOD
  ENDWHILE
```

```
  temp[self.top + 1] <- value
  self.items <- temp
  self.top <- self.top + 1
```

```
  OUTPUT STRING(Pushed: {value})
  self.show()
ENDFUNCTION
```

```

FUNCTION pop()
  IF self.is_empty() == BOOLEAN(True) THEN
    OUTPUT STRING(Pop attempted on empty stack)
    self.show()
    RETURN None
  ENDIF

  value <- self.items[self.top]

  new_length <- LENGTH OF self.items - 1
  temp <- LIST OF SIZE EQUALS new_length FILLED WITH None

  i <- 0
  WHILE i < new_length DO
    temp[i] <- self.items[i]
    i <- i + 1
  ENDWHILE

  self.items <- temp
  self.top <- self.top - 1

  OUTPUT STRING(Popped: {value})
  self.show()
  RETURN value
ENDFUNCTION

```

```

FUNCTION peek()
  IF self.is_empty() == BOOLEAN(True) THEN
    OUTPUT STRING(Peek attempted on empty stack)
    self.show()
    RETURN None
  ENDIF

  value <- self.items[self.top]
  OUTPUT STRING(Peeked: {value})
  self.show()
  RETURN value
ENDFUNCTION

```

```

FUNCTION is_empty()
  RETURN self.top == -1
ENDFUNCTION

```

```

FUNCTION size()
  RETURN self.top + 1
ENDFUNCTION

```

ENDCLASS

```
s <- NEW Stack()
```

```
s.push(10)
```

```
s.push(20)
```

```
s.push(30)
```

```
s.peak()
```

```
s.pop()
```

```
s.peak()
```

```
s.pop()
```

```
s.pop()
```

```
s.pop()
```

```
s.peak()
```

38. Find the first non-repeating character in a string

[python code]

```
print(f"First Non Repeating Character In a String")
```

```
def fnon_repeat(s):
```

```
    chars = list(s)
```

```
    counts = [0] * len(chars)
```

```
    # -----
```

```
    # PASS 1 — count occurrences manually
```

```
    # -----
```

```
    i = 0
```

```
    while i < len(chars):
```

```
        j = 0
```

```
        while j < len(chars):
```

```
            if chars[i] == chars[j]:
```

```
                counts[i] += 1
```

```
            j += 1
```

```
        i += 1
```

```
    # -----
```

```
    # PASS 2 — find first character with count = 1
```

```
    # -----
```

```
    i = 0
```

```
    while i < len(chars):
```

```
        if counts[i] == 1:
```

```
            return "String = " + s + " Result: " + chars[i]
```

```
        i += 1
```

```
    return "String = " + s + " Result: " + str(None)
```

```
# -----
# EXAMPLE USAGE
# -----
print(fnon_repeat("swiss"))
print(fnon_repeat("aabbccd"))
print(fnon_repeat("aabbcc"))
```

[pseudocode]

OUTPUT STRING(First Non Repeating Character In a String)

```
FUNCTION fnon_repeat(s)
  chars <- LIST OF CHARACTERS IN s
  counts <- LIST OF SIZE EQUALS LENGTH OF chars FILLED WITH 0

  i <- 0
  WHILE i < LENGTH OF chars DO
    j <- 0
    WHILE j < LENGTH OF chars DO
      IF chars[i] == chars[j] THEN
        counts[i] <- counts[i] + 1
      ENDIF
      j <- j + 1
    ENDWHILE
    i <- i + 1
  ENDWHILE

  i <- 0
  WHILE i < LENGTH OF chars DO
    IF counts[i] == 1 THEN
      RETURN STRING(String = {s} Result: {chars[i]})
    ENDIF
    i <- i + 1
  ENDWHILE

  RETURN STRING(String = {s} Result: {None})
ENDFUNCTION
```

```
OUTPUT STRING({fnon_repeat(STRING(swiss))})
OUTPUT STRING({fnon_repeat(STRING(aabbccd))})
OUTPUT STRING({fnon_repeat(STRING(aabbcc))})
```

39. Detect a cycle in a single linked list (Floyd's Cycle Detection)

[python code]

```
class Node:
```

```
def __init__(self, value):
    self.value = value
    self.next = None
```

```
class LinkedList:
```

```
def __init__(self):
    self.head = None
    self.tail = None
    self.nodes = [] # stores ALL nodes in order
    print("Linked List Created\n")
```

```
# -----
```

```
# APPEND
```

```
# -----
```

```
def append(self, value):
    new_node = Node(value)
    print(f"Appended Value {value}")
    if self.head is None:
        self.head = new_node
        self.tail = new_node
    else:
        self.tail.next = new_node
        self.tail = new_node
```

```
self.nodes.append(new_node)
```

```
# -----
```

```
# PREPEND
```

```
# -----
```

```
def prepend(self, value):
    print(f"Prepended Value {value} To The Front")
    new_node = Node(value)
    if self.head is None:
        self.head = new_node
        self.tail = new_node
    else:
        new_node.next = self.head
        self.head = new_node
```

```
self.nodes.insert(0, new_node)
```

```
# -----
```

```
# INSERT AT INDEX
```

```
# -----
```

```
def insert_at(self, index, value):
    print(f"Inserting Value {value} at Index {index}")
```

```
if index == 0:
```

```

        self.prepend(value)
        return

new_node = Node(value)
temp = self.head
i = 0

while temp is not None and i < index - 1:
    temp = temp.next
    i += 1

if temp is None:
    print("Insert Failed: Index out of range")
    return

new_node.next = temp.next
temp.next = new_node

if new_node.next is None:
    self.tail = new_node

self.nodes.insert(index, new_node)
print(f"Insert Successful: {value} Inserted at Index {index}")

# -----
# CREATE CYCLE USING START + END INDEXES
# -----
def create_cycle(self, start_index, end_index):
    print(f"\nCycle Created: Loop Start index {start_index} → End index {end_index}")
    if start_index < 0 or end_index < 0:
        print("Cycle Creation Failed: Invalid Index")
        return
    if start_index >= len(self.nodes) or end_index >= len(self.nodes):
        print("Cycle Creation Failed: Index Out Of Range")
        return

    start_node = self.nodes[start_index]
    end_node = self.nodes[end_index]

    end_node.next = start_node
    print(f"\nCycle Created: Node {start_index} now points to node {end_index}")

# -----
# REMOVE CYCLE
# -----
def remove_cycle(self):
    print("Remove Cycle Requested")

    slow = self.head

```

```

fast = self.head

# Detect cycle
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
    if slow == fast:
        print("Cycle Detected: Proceeding with Removal")
        break
else:
    print("No cycle found — nothing to remove")
    return

# Find cycle start
slow = self.head
while slow != fast:
    slow = slow.next
    fast = fast.next

cycle_start = slow

# Find node just before cycle start
temp = cycle_start
while temp.next != cycle_start:
    temp = temp.next

# Break the cycle
temp.next = None

# Update tail
self.tail = self.nodes[-1]

print(f"Cycle Removed Successfully: Loop started at Node with Value {cycle_start.value}")

# -----
# LOOP DIAGRAM (perfect alignment)
# -----
def loop_diagram(self):
    print("\nGenerating Loop Diagram...\n")

    nodes = self.nodes # full list guaranteed

    # Detect cycle start + end
    visited = {}
    temp = self.head
    index = 0
    cycle_start = None
    cycle_end = None

```

```

while temp is not None:
    if temp in visited:
        cycle_start = visited[temp]
        cycle_end = index - 1
        break
    visited[temp] = index
    temp = temp.next
    index += 1

# Build chain text
parts = [str(node.value) for node in nodes]
sep = " → "
chain = sep.join(parts)
print(chain)

if cycle_start is None:
    print("No Cycle Detected\n")
    return

# Compute exact character positions
start_char = sum(len(parts[i]) + len(sep) for i in range(cycle_start))
start_center = start_char + len(parts[cycle_start]) // 2

end_char = sum(len(parts[i]) + len(sep) for i in range(cycle_end))
end_center = end_char + len(parts[cycle_end]) // 2

# Bottom connector
print(" " * start_center + "L" + "-" * (end_center - start_center - 1) + "J")

# Cycle message
print(f"Cycle Detected From Node {cycle_start} to Node {cycle_end}\n")

```

```

# -----
# EXAMPLE USAGE
# -----
print("Detect Cycle in Singly Linked List\n")
l = LinkedList()
l.append(10)
l.append(20)
l.append(30)
l.append(40)
l.append(50)
l.append(60)

```

```

l.loop_diagram()

```

```

# Remove cycle

```



```

l.remove_cycle()

l.loop_diagram()

l.append(40)
l.append(50)

# Create cycle: 50 → 20
l.create_cycle(start_index=1, end_index=6)

l.loop_diagram()

# Remove cycle
l.remove_cycle()

print("\nAfter Removing Cycle:")
l.loop_diagram()

```

[pseudocode]

CLASS Node

```

CONSTRUCTOR ACCEPTS(self, value)
  PROPERTIES(
    self.value <- value,
    self.next <- None
  )
ENDCONSTRUCTOR

```

ENDCLASS

CLASS LinkedList

```

CONSTRUCTOR ACCEPTS(self)
  PROPERTIES(
    self.head <- None,
    self.tail <- None,
    self.nodes <- EMPTY LIST
  )
  OUTPUT STRING(Linked List Created\n)
ENDCONSTRUCTOR

```

```

FUNCTION append(value)
  new_node <- NEW Node(value)
  OUTPUT STRING(Appended Value {value})

```

```
IF self.head == None THEN
    self.head <- new_node
    self.tail <- new_node
ELSE
    self.tail.next <- new_node
    self.tail <- new_node
ENDIF

APPEND new_node TO self.nodes
ENDFUNCTION
```

```
FUNCTION prepend(value)
    OUTPUT STRING(Prepended Value {value} To The Front)

    new_node <- NEW Node(value)

    IF self.head == None THEN
        self.head <- new_node
        self.tail <- new_node
    ELSE
        new_node.next <- self.head
        self.head <- new_node
    ENDIF

    INSERT new_node AT INDEX 0 IN self.nodes
ENDFUNCTION
```

```
FUNCTION insert_at(index, value)
    OUTPUT STRING(Inserting Value {value} at Index {index})

    IF index == 0 THEN
        self.prepend(value)
        RETURN
    ENDIF

    new_node <- NEW Node(value)
    temp <- self.head
    i <- 0

    WHILE temp != None AND i < index - 1 DO
        temp <- temp.next
        i <- i + 1
    ENDWHILE

    IF temp == None THEN
        OUTPUT STRING(Insert Failed: Index out of range)
        RETURN
    ENDIF
```

ENDIF

new_node.next <- temp.next

temp.next <- new_node

IF new_node.next == None THEN

self.tail <- new_node

ENDIF

INSERT new_node AT INDEX index IN self.nodes

OUTPUT STRING(Insert Successful: {value} Inserted at Index {index})

ENDFUNCTION

FUNCTION create_cycle(start_index, end_index)

OUTPUT STRING(\nCycle Created: Loop Start index {start_index} -> End index {end_index})

IF start_index < 0 OR end_index < 0 THEN

OUTPUT STRING(Cycle Creation Failed: Invalid Index)

RETURN

ENDIF

IF start_index >= LENGTH OF self.nodes OR end_index >= LENGTH OF self.nodes THEN

OUTPUT STRING(Cycle Creation Failed: Index Out Of Range)

RETURN

ENDIF

start_node <- self.nodes[start_index]

end_node <- self.nodes[end_index]

end_node.next <- start_node

OUTPUT STRING(\nCycle Created: Node {start_index} now points to node {end_index})

ENDFUNCTION

FUNCTION remove_cycle()

OUTPUT STRING(Remove Cycle Requested)

slow <- self.head

fast <- self.head

WHILE fast != None AND fast.next != None DO

slow <- slow.next

fast <- fast.next.next

IF slow == fast THEN

OUTPUT STRING(Cycle Detected: Proceeding with Removal)

BREAK

```

ENDIF
ENDWHILE

IF fast == None OR fast.next == None THEN
    OUTPUT STRING(No cycle found — nothing to remove)
    RETURN
ENDIF

slow <- self.head

WHILE slow != fast DO
    slow <- slow.next
    fast <- fast.next
ENDWHILE

cycle_start <- slow

temp <- cycle_start
WHILE temp.next != cycle_start DO
    temp <- temp.next
ENDWHILE

temp.next <- None

self.tail <- self.nodes[LENGTH OF self.nodes - 1]

OUTPUT STRING(Cycle Removed Successfully: Loop started at Node with Value {cycle_start.value})
ENDFUNCTION

FUNCTION loop_diagram()
    OUTPUT STRING(\nGenerating Loop Diagram...\n)

    nodes <- self.nodes

    visited <- EMPTY DICTIONARY
    temp <- self.head
    index <- 0
    cycle_start <- None
    cycle_end <- None

    WHILE temp != None DO
        IF temp IN visited THEN
            cycle_start <- visited[temp]
            cycle_end <- index - 1
            BREAK
        ENDIF

        visited[temp] <- index
    
```

```
temp <- temp.next
```

```
index <- index + 1
```

```
ENDWHILE
```

```
parts <- EMPTY LIST
```

```
i <- 0
```

```
WHILE i < LENGTH OF nodes DO
```

```
  APPEND STRING(nodes[i].value) TO parts
```

```
  i <- i + 1
```

```
ENDWHILE
```

```
sep <- STRING( -> )
```

```
chain <- JOIN parts WITH sep
```

```
OUTPUT STRING({chain})
```

```
IF cycle_start == None THEN
```

```
  OUTPUT STRING(No Cycle Detected\n)
```

```
  RETURN
```

```
ENDIF
```

```
start_char <- 0
```

```
i <- 0
```

```
WHILE i < cycle_start DO
```

```
  start_char <- start_char + LENGTH OF parts[i] + LENGTH OF sep
```

```
  i <- i + 1
```

```
ENDWHILE
```

```
start_center <- start_char + (LENGTH OF parts[cycle_start] INTEGER DIVISION 2)
```

```
end_char <- 0
```

```
i <- 0
```

```
WHILE i < cycle_end DO
```

```
  end_char <- end_char + LENGTH OF parts[i] + LENGTH OF sep
```

```
  i <- i + 1
```

```
ENDWHILE
```

```
end_center <- end_char + (LENGTH OF parts[cycle_end] INTEGER DIVISION 2)
```

```
OUTPUT STRING(" " * start_center + "L" + "-" * (end_center - start_center - 1) + "J")
```

```
OUTPUT STRING(Cycle Detected From Node {cycle_start} to Node {cycle_end})\n)
```

```
ENDFUNCTION
```

```
ENDCLASS
```

```
OUTPUT STRING(Detect Cycle in Singly Linked List\n)
```

```
l <- NEW LinkedList()
```

```
l.append(10)
```

```
l.append(20)
```

```
l.append(30)
```

```
l.append(40)
```

```
l.append(50)
```

```
l.append(60)
```

```
l.loop_diagram()
```

```
l.remove_cycle()
```

```
l.loop_diagram()
```

```
l.append(40)
```

```
l.append(50)
```

```
l.create_cycle(start_index = 1, end_index = 6)
```

```
l.loop_diagram()
```

```
l.remove_cycle()
```

```
OUTPUT STRING(\nAfter Removing Cycle:)
```

```
l.loop_diagram()
```

40. Implement a queue using 2 stacks

[python code]

```
class Stack:
```

```
    def init(self, name="Stack"):
```

```
        self.items = []
```

```
        self.name = name
```

```
    def push(self, value):
```

```
        self.items.append(value)
```

```
        print(f"{self.name}: pushed {value}")
```

```
    def pop(self):
```

```
        if len(self.items) == 0:
```

```
            print(f"{self.name}: pop attempted on empty stack")
```

```
            return None
```

```
        value = self.items.pop()
```

```
        print(f"{self.name}: popped {value}")
```

```
        return value
```

```
def is_empty(self):
    return len(self.items) == 0

def peek(self):
    if len(self.items) == 0:
        return None
    return self.items[len(self.items) - 1]
```

```
def display(self):
    # Display in your required format
    print(f"{self.name} {self.items}")
```

```
class QueueUsingTwoStacks:
    def __init__(self):
        print("Queue created using two stacks\n")
        self.in_stack = Stack("(Stack1)IN ")
        self.out_stack = Stack("(Stack2)OUT ")

    # -----
    # ENQUEUE
    # -----
    def enqueue(self, value):
        print(f"\nEnqueue: {value}")
        self.in_stack.push(value)
        self.display()

    # -----
    # DEQUEUE
    # -----
    def dequeue(self):
        print("\nDequeue requested")

        if self.out_stack.is_empty():
            print("OUT is empty → transferring from IN")
            while not self.in_stack.is_empty():
                moved = self.in_stack.pop()
                self.out_stack.push(moved)

        if self.out_stack.is_empty():
            print("Queue is empty — nothing to dequeue")
            return None

        value = self.out_stack.pop()
        print(f"Dequeued: {value}")
        self.display()
        return value

    # -----
```

```

# PEEK
# -----
def peek(self):
    print("\nPeek requested")

    if self.out_stack.is_empty():
        while not self.in_stack.is_empty():
            moved = self.in_stack.pop()
            self.out_stack.push(moved)

    value = self.out_stack.peek()
    print(f"Front of queue: {value}")
    self.display()
    return value

# -----
# DISPLAY QUEUE (IN and OUT stacks)
# -----
def display(self):
    print("\nQueue State:")
    self.in_stack.display()
    self.out_stack.display()
    print("")

# -----
# EXAMPLE USAGE
# -----
q = QueueUsingTwoStacks()

q.enqueue(10)
q.enqueue(20)
q.enqueue(30)

q.dequeue()

q.enqueue(40)
q.enqueue(50)

q.peek()

q.dequeue()
q.dequeue()
q.dequeue()

```

[pseudocode]

CLASS Stack


```
CONSTRUCTOR ACCEPTS(self, name)
  PROPERTIES(
    self.items <- EMPTY LIST,
    self.name <- name
  )
ENDCONSTRUCTOR
```

```
FUNCTION push(value)
  APPEND value TO self.items
  OUTPUT STRING({self.name}: pushed {value})
ENDFUNCTION
```

```
FUNCTION pop()
  IF LENGTH OF self.items == 0 THEN
    OUTPUT STRING({self.name}: pop attempted on empty stack)
    RETURN None
  ENDIF

  value <- REMOVE LAST ITEM FROM self.items
  OUTPUT STRING({self.name}: popped {value})
  RETURN value
ENDFUNCTION
```

```
FUNCTION is_empty()
  RETURN LENGTH OF self.items == 0
ENDFUNCTION
```

```
FUNCTION peek()
  IF LENGTH OF self.items == 0 THEN
    RETURN None
  ENDIF

  RETURN self.items[LENGTH OF self.items - 1]
ENDFUNCTION
```

```
FUNCTION display()
  OUTPUT STRING({self.name} {self.items})
ENDFUNCTION
```

```
ENDCLASS
```

CLASS QueueUsingTwoStacks

```
CONSTRUCTOR ACCEPTS(self)
  PROPERTIES(
    self.in_stack <- NEW Stack(STRING((Stack1)IN )),
    self.out_stack <- NEW Stack(STRING((Stack2)OUT ))
  )
  OUTPUT STRING(Queue created using two stacks\n)
ENDCONSTRUCTOR

FUNCTION enqueue(value)
  OUTPUT STRING(\nEnqueue: {value})
  self.in_stack.push(value)
  self.display()
ENDFUNCTION

FUNCTION dequeue()
  OUTPUT STRING(\nDequeue requested)

  IF self.out_stack.is_empty() == BOOLEAN(True) THEN
    OUTPUT STRING(OUT is empty -> transferring from IN)

    WHILE self.in_stack.is_empty() == BOOLEAN(False) DO
      moved <- self.in_stack.pop()
      self.out_stack.push(moved)
    ENDWHILE
  ENDIF

  IF self.out_stack.is_empty() == BOOLEAN(True) THEN
    OUTPUT STRING(Queue is empty -- nothing to dequeue)
    RETURN None
  ENDIF

  value <- self.out_stack.pop()
  OUTPUT STRING(Dequeued: {value})
  self.display()
  RETURN value
ENDFUNCTION

FUNCTION peek()
  OUTPUT STRING(\nPeek requested)

  IF self.out_stack.is_empty() == BOOLEAN(True) THEN
    WHILE self.in_stack.is_empty() == BOOLEAN(False) DO
      moved <- self.in_stack.pop()
      self.out_stack.push(moved)
```

```
ENDWHILE
ENDIF
```

```
value <- self.out_stack.peek()
OUTPUT STRING(Front of queue: {value})
self.display()
RETURN value
ENDFUNCTION
```

```
FUNCTION display()
OUTPUT STRING(\nQueue State:)
self.in_stack.display()
self.out_stack.display()
OUTPUT STRING()
ENDFUNCTION
```

```
ENDCLASS
```

```
q <- NEW QueueUsingTwoStacks()
```

```
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
```

```
q.dequeue()
```

```
q.enqueue(40)
q.enqueue(50)
```

```
q.peek()
```

```
q.dequeue()
q.dequeue()
q.dequeue()
```

41. Solve the N Queens Problem for an arbitrary board size N
[python code]

```
def safe_placement(board, row, col, n):
    r = 0
    while r < row:
        if board[r] == col:
            return False
        r += 1
```

```

r = row - 1
c = col - 1
while r >= 0 and c >= 0:
    if board[r] == c:
        return False
    r -= 1
    c -= 1

```

```

r = row - 1
c = col + 1
while r >= 0 and c < n:
    if board[r] == c:
        return False
    r -= 1
    c += 1

```

```

return True

```

```

def solve_n_queens_util(board, row, n, solutions):
    if row == n:
        solutions.append(board[:])
        return

    col = 0
    while col < n:
        if safe_placement(board, row, col, n):
            board[row] = col
            solve_n_queens_util(board, row + 1, n, solutions)
            board[row] = -1
        col += 1

```

```

def solve_n_queens(n):
    board = [-1] * n
    solutions = []
    solve_n_queens_util(board, 0, n, solutions)
    return solutions

```

```

def print_board(solution):
    n = len(solution)
    r = 0
    while r < n:
        row_str = ""
        c = 0
        while c < n:
            if solution[r] == c:
                row_str += "Q "

```

```

    else:
        row_str += ". "
    c += 1
    print(row_str)
    r += 1
    print("")

```

```

def print_all_solutions(solutions):
    total = len(solutions)
    print(f"\nShowing ALL {total} solutions:\n")
    i = 0
    while i < total:
        print(f"Solution {i + 1}:")
        print_board(solutions[i])
        i += 1

```

```

# =====
# MAIN PROGRAM FLOW
# =====

```

```

# 1. Input N
N = int(input("Enter board size N: "))

```

```

# 2. Solve and store solutions
solutions = solve_n_queens(N)
total = len(solutions)

```

```

print(f"\nSolved {N}-Queens.")
print(f"Total solutions: {total}\n")

```

```

# 3. Display menu loop
index = 0

```

```

while True:
    print("Menu:")
    print(" 1 - Show current solution")
    print(" 2 - Next solution")
    print(" 3 - Previous solution")
    print(" 4 - Go to solution number")
    print(" 5 - Show ALL solutions")
    print(" 6 - Quit")

```

```

choice = input("Enter choice: ").strip()

```

```

# 4. Display results based on choice
if choice == "1":
    if total > 0:

```

```

    print(f"\nShowing solution {index + 1} of {total}:\n")
    print_board(solutions[index])
else:
    print("\nNo solutions exist for this N.\n")

elif choice == "2":
    if total > 0:
        index = (index + 1) % total
        print(f"\nShowing solution {index + 1} of {total}:\n")
        print_board(solutions[index])
    else:
        print("\nNo solutions.\n")

elif choice == "3":
    if total > 0:
        index = (index - 1) % total
        print(f"\nShowing solution {index + 1} of {total}:\n")
        print_board(solutions[index])
    else:
        print("\nNo solutions.\n")

elif choice == "4":
    if total > 0:
        num = input("Enter solution number: ").strip()
        if num.isdigit():
            num = int(num)
            if 1 <= num <= total:
                index = num - 1
                print(f"\nShowing solution {index + 1} of {total}:\n")
                print_board(solutions[index])
            else:
                print("\nInvalid number.\n")
        else:
            print("\nInvalid input.\n")
    else:
        print("\nNo solutions.\n")

elif choice == "5":
    print_all_solutions(solutions)

elif choice == "6":
    print("\nExiting program.\n")
    break

else:
    print("\nInvalid option.\n")

```

[pseudocode]

```
FUNCTION safe_placement(board, row, col, n)
```

```
  r <- 0
```

```
  WHILE r < row DO
```

```
    IF board[r] == col THEN
```

```
      RETURN BOOLEAN(False)
```

```
    ENDIF
```

```
    r <- r + 1
```

```
  ENDWHILE
```

```
  r <- row - 1
```

```
  c <- col - 1
```

```
  WHILE r >= 0 AND c >= 0 DO
```

```
    IF board[r] == c THEN
```

```
      RETURN BOOLEAN(False)
```

```
    ENDIF
```

```
    r <- r - 1
```

```
    c <- c - 1
```

```
  ENDWHILE
```

```
  r <- row - 1
```

```
  c <- col + 1
```

```
  WHILE r >= 0 AND c < n DO
```

```
    IF board[r] == c THEN
```

```
      RETURN BOOLEAN(False)
```

```
    ENDIF
```

```
    r <- r - 1
```

```
    c <- c + 1
```

```
  ENDWHILE
```

```
  RETURN BOOLEAN(True)
```

```
ENDFUNCTION
```

```
FUNCTION solve_n_queens_util(board, row, n, solutions)
```

```
  IF row == n THEN
```

```
    APPEND board[:] TO solutions
```

```
    RETURN
```

```
  ENDIF
```

```
  col <- 0
```

```
  WHILE col < n DO
```

```
    IF safe_placement(board, row, col, n) == BOOLEAN(True) THEN
```

```
      board[row] <- col
```

```
      solve_n_queens_util(board, row + 1, n, solutions)
```

```
      board[row] <- -1
```

```
    ENDIF
```

```
    col <- col + 1
```

```
ENDWHILE  
ENDFUNCTION
```

```
FUNCTION solve_n_queens(n)  
  board <- LIST OF SIZE EQUALS n FILLED WITH -1  
  solutions <- EMPTY LIST  
  solve_n_queens_util(board, 0, n, solutions)  
  RETURN solutions  
ENDFUNCTION
```

```
FUNCTION print_board(solution)  
  n <- LENGTH OF solution  
  r <- 0
```

```
  WHILE r < n DO  
    row_str <- STRING()  
    c <- 0
```

```
    WHILE c < n DO  
      IF solution[r] == c THEN  
        row_str <- row_str + STRING(Q )  
      ELSE  
        row_str <- row_str + STRING(. )  
      ENDIF  
      c <- c + 1  
    ENDWHILE
```

```
    OUTPUT STRING({row_str})  
    r <- r + 1  
  ENDWHILE
```

```
  OUTPUT STRING()  
ENDFUNCTION
```

```
FUNCTION print_all_solutions(solutions)  
  total <- LENGTH OF solutions  
  OUTPUT STRING(\nShowing ALL {total} solutions:\n)  
  
  i <- 0  
  WHILE i < total DO  
    OUTPUT STRING(Solution {i + 1}:)  
    print_board(solutions[i])  
    i <- i + 1
```



```
ENDWHILE
ENDFUNCTION
```

```
# =====
# MAIN PROGRAM FLOW — N QUEENS
# =====
```

```
OUTPUT STRING(Enter board size N:)
N <- INPUT(STRING(Enter board size N))
```

```
solutions <- solve_n_queens(N)
total <- LENGTH OF solutions
```

```
OUTPUT STRING(\nSolved {N}-Queens.)
OUTPUT STRING(Total solutions: {total}\n)
```

```
index <- 0
loop <- BOOLEAN(True)
```

```
WHILE loop == BOOLEAN(True) DO
  OUTPUT STRING(Menu:)
  OUTPUT STRING( 1 - Show current solution)
  OUTPUT STRING( 2 - Next solution)
  OUTPUT STRING( 3 - Previous solution)
  OUTPUT STRING( 4 - Go to solution number)
  OUTPUT STRING( 5 - Show ALL solutions)
  OUTPUT STRING( 6 - Quit)
```

```
choice <- INPUT(STRING(Enter choice: ))
```

```
IF choice == STRING(1) THEN
  IF total > 0 THEN
    OUTPUT STRING(\nShowing solution {index + 1} of {total}:\n)
    print_board(solutions[index])
  ELSE
    OUTPUT STRING(\nNo solutions exist for this N.\n)
  ENDIF
```

```
ELIF choice == STRING(2) THEN
  IF total > 0 THEN
    index <- (index + 1) MOD total
    OUTPUT STRING(\nShowing solution {index + 1} of {total}:\n)
    print_board(solutions[index])
  ELSE
    OUTPUT STRING(\nNo solutions.\n)
  ENDIF
```

```
ELIF choice == STRING(3) THEN
```

```
  IF total > 0 THEN
```

```
    index <- (index - 1) MOD total
```

```
    OUTPUT STRING(\nShowing solution {index + 1} of {total}:\n)
```

```
    print_board(solutions[index])
```

```
  ELSE
```

```
    OUTPUT STRING(\nNo solutions.\n)
```

```
  ENDIF
```

```
ELIF choice == STRING(4) THEN
```

```
  IF total > 0 THEN
```

```
    num <- INPUT(STRING(Enter solution number: ))
```

```
    IF ISDIGIT(num) == BOOLEAN(True) THEN
```

```
      num_int <- INTEGER(num)
```

```
      IF 1 <= num_int AND num_int <= total THEN
```

```
        index <- num_int - 1
```

```
        OUTPUT STRING(\nShowing solution {index + 1} of {total}:\n)
```

```
        print_board(solutions[index])
```

```
      ELSE
```

```
        OUTPUT STRING(\nInvalid number.\n)
```

```
      ENDIF
```

```
    ELSE
```

```
      OUTPUT STRING(\nInvalid input.\n)
```

```
    ENDIF
```

```
  ELSE
```

```
    OUTPUT STRING(\nNo solutions.\n)
```

```
  ENDIF
```

```
ELIF choice == STRING(5) THEN
```

```
  print_all_solutions(solutions)
```

```
ELIF choice == STRING(6) THEN
```

```
  OUTPUT STRING(\nExiting program.\n)
```

```
  loop <- BOOLEAN(False)
```

```
ELSE
```

```
  OUTPUT STRING(\nInvalid option.\n)
```

```
ENDIF
```

```
ENDWHILE
```

42. Find all pairs in an array that match a given number

[python code]

```
import random

def find_pairs(arr, target, decimals=6):
    pairs = []
    i = 0
    n = len(arr)

    while i < n:
        j = i + 1
        while j < n:
            if round(arr[i] + arr[j], decimals) == round(target, decimals):
                pairs.append((arr[i], arr[j]))
            j += 1
        i += 1

    return pairs

# =====
# MAIN PROGRAM FLOW
# =====

# 1. Create and print random array (ints + floats)
size = 20
arr = []
i = 0
while i < size:
    # 50% chance integer, 50% chance float
    if random.random() < 0.5:
        num = random.randint(-20, 20)    # integer
    else:
        num = round(random.uniform(-20, 20), 2) # float
    arr.append(num)
    i += 1

print("Random array:")
print(arr)

# 2. Prompt for target
target = float(input("Enter target number: "))

# 3. Find pairs
pairs = find_pairs(arr, target)
```

```
# 4. Display results
print(f"\nPairs that sum to {target}:\n")
```

```
i = 0
if len(pairs) == 0:
    print("No matching pairs found.")
else:
    while i < len(pairs):
        print(pairs[i])
        i += 1
```

[pseudocode]

```
FUNCTION find_pairs(arr, target, decimals)
    pairs <- EMPTY LIST
    i <- 0
    n <- LENGTH OF arr

    WHILE i < n DO
        j <- i + 1

        WHILE j < n DO
            IF ROUND(arr[i] + arr[j], decimals) == ROUND(target, decimals) THEN
                APPEND TUPLE(arr[i], arr[j]) TO pairs
            ENDIF

            j <- j + 1
        ENDWHILE

        i <- i + 1
    ENDWHILE

    RETURN pairs
ENDFUNCTION
```

```
# =====
# MAIN PROGRAM FLOW — PAIR FINDER
# =====
```

```
size <- 20
arr <- EMPTY LIST
i <- 0

WHILE i < size DO
    rand_val <- RANDOM_FLOAT(0, 1)

    IF rand_val < 0.5 THEN
```

```

    num <- RANDOM_INT(-20, 20)
ELSE
    num <- ROUND(RANDOM_FLOAT(-20, 20), 2)
ENDIF

APPEND num TO arr
i <- i + 1
ENDWHILE

OUTPUT STRING(Random array:)
OUTPUT STRING({arr})

target <- INPUT(STRING(Enter target number: ))

pairs <- find_pairs(arr, target, 6)

OUTPUT STRING(\nPairs that sum to {target}:\n)

i <- 0

IF LENGTH OF pairs == 0 THEN
    OUTPUT STRING(No matching pairs found.)
ELSE
    WHILE i < LENGTH OF pairs DO
        OUTPUT STRING({pairs[i]})
        i <- i + 1
    ENDWHILE
ENDIF

```

43. Solve the Tower of Hanoi problem for N Discs

[python code]

```

print("Towers Of Hanoi")
def display_board(pegs, n):

    level = n - 1
    while level >= 0:
        row = ""

        # Peg A
        if level < len(pegs["A"]):
            row += f" {pegs['A'][level]} "
        else:
            row += " | "

        # Peg B
        if level < len(pegs["B"]):
            row += f" {pegs['B'][level]} "
        else:

```

```

    row += " | "

# Peg C
if level < len(pegs["C"]):
    row += f" {pegs['C'][level]} "
else:
    row += " | "

print(row)
level -= 1

print(" A  B  C\n")

def move_between(pegs, a, b, n):
    # If peg a is empty, move from b → a
    if len(pegs[a]) == 0:
        disc = pegs[b].pop()
        pegs[a].append(disc)
        print(f"Move disc {disc} from {b} to {a}\n")

    # If peg b is empty, move from a → b
    elif len(pegs[b]) == 0:
        disc = pegs[a].pop()
        pegs[b].append(disc)
        print(f"Move disc {disc} from {a} to {b}\n")

    # Otherwise move the smaller disc legally
    elif pegs[a][-1] < pegs[b][-1]:
        disc = pegs[a].pop()
        pegs[b].append(disc)
        print(f"Move disc {disc} from {a} to {b}\n")
    else:
        disc = pegs[b].pop()
        pegs[a].append(disc)
        print(f"Move disc {disc} from {b} to {a}\n")

display_board(pegs, n)

def hanoi(n, pegs):
    A, B, C = "A", "B", "C"

    # Even number of discs → swap target and auxiliary
    if n % 2 == 0:
        B, C = C, B

    total_moves = (2 ** n) - 1

    display_board(pegs, n)

```

```

move = 1
while move <= total_moves:
    if move % 3 == 1:
        move_between(pegs, A, C, n)
    elif move % 3 == 2:
        move_between(pegs, A, B, n)
    else:
        move_between(pegs, B, C, n)

    move += 1
print("Solved!!")

```

```

# =====
# MAIN PROGRAM FLOW
# =====
loop = True
while loop:
    try:
        N = int(input("Enter number of discs: "))
        N = int(N)
        print("Setting up..\n")
        loop = False
    except:
        print(f"Than's Not a Valid Number, Please Try Again!!\n")

```

```

pegs = {
    "A": list(range(N, 0, -1)),
    "B": [],
    "C": []
}

```

```

hanoi(N, pegs)

```

[pseudocode]

OUTPUT STRING(Towers Of Hanoi)

FUNCTION display_board(pegs, n)

 level <- n - 1

 WHILE level >= 0 DO

 row <- STRING()

 IF level < LENGTH OF pegs[A] THEN

 row <- row + STRING({pegs[A][level]})

 ELSE

 row <- row + STRING(|)

```

ENDIF

IF level < LENGTH OF pegs[B] THEN
  row <- row + STRING( {pegs[B][level]} )
ELSE
  row <- row + STRING( | )
ENDIF

IF level < LENGTH OF pegs[C] THEN
  row <- row + STRING( {pegs[C][level]} )
ELSE
  row <- row + STRING( | )
ENDIF

OUTPUT STRING({row})
level <- level - 1
ENDWHILE

OUTPUT STRING( A B C\n)
ENDFUNCTION

FUNCTION move_between(pegs, a, b, n)
  IF LENGTH OF pegs[a] == 0 THEN
    disc <- REMOVE LAST ITEM FROM pegs[b]
    APPEND disc TO pegs[a]
    OUTPUT STRING(Move disc {disc} from {b} to {a}\n)

  ELIF LENGTH OF pegs[b] == 0 THEN
    disc <- REMOVE LAST ITEM FROM pegs[a]
    APPEND disc TO pegs[b]
    OUTPUT STRING(Move disc {disc} from {a} to {b}\n)

  ELIF pegs[a][LENGTH OF pegs[a] - 1] < pegs[b][LENGTH OF pegs[b] - 1] THEN
    disc <- REMOVE LAST ITEM FROM pegs[a]
    APPEND disc TO pegs[b]
    OUTPUT STRING(Move disc {disc} from {a} to {b}\n)

  ELSE
    disc <- REMOVE LAST ITEM FROM pegs[b]
    APPEND disc TO pegs[a]
    OUTPUT STRING(Move disc {disc} from {b} to {a}\n)
  ENDIF

  display_board(pegs, n)
ENDFUNCTION

```



```

FUNCTION hanoi(n, pegs)
  A <- STRING(A)
  B <- STRING(B)
  C <- STRING(C)

  IF n MOD 2 == 0 THEN
    temp <- B
    B <- C
    C <- temp
  ENDIF

  total_moves <- (2 EXP n) - 1

  display_board(pegs, n)

  move <- 1
  WHILE move <= total_moves DO
    IF move MOD 3 == 1 THEN
      move_between(pegs, A, C, n)
    ELIF move MOD 3 == 2 THEN
      move_between(pegs, A, B, n)
    ELSE
      move_between(pegs, B, C, n)
    ENDIF

    move <- move + 1
  ENDWHILE

  OUTPUT STRING(Solved!!)
ENDFUNCTION

```

```

# =====
# MAIN PROGRAM FLOW — TOWERS OF HANOI
# =====

```

```

loop <- BOOLEAN(True)

WHILE loop == BOOLEAN(True) DO
  TRY
    N <- INPUT(STRING(Enter number of discs: ))
    N <- INTEGER(N)
    OUTPUT STRING(Setting up..\n)
    loop <- BOOLEAN(False)
  CATCH
    OUTPUT STRING(That's Not a Valid Number, Please Try Again!!\n)
  ENDTRY

```

ENDWHILE

```
pegs <- DICTIONARY(  
  A = LIST OF SIZE EQUALS N FILLED WITH None,  
  B = EMPTY LIST,  
  C = EMPTY LIST  
)
```

```
i <- 0  
WHILE i < N DO  
  pegs[A][i] <- N - i  
  i <- i + 1  
ENDWHILE
```

hanoi(N, pegs)

44. Find the continuous subarray with the maximum sum (Kadane's algorithm)

[python code]

```
import random
```

```
def kadane(arr):
```

```
    n = len(arr)
```

```
    current_sum = arr[0]
```

```
    best_sum = arr[0]
```

```
    current_start = 0
```

```
    best_ranges = [(0, 0)]
```

```
    i = 1
```

```
    while i < n:
```

```
        # Decide whether to extend or restart
```

```
        if current_sum + arr[i] < arr[i]:
```

```
            current_sum = arr[i]
```

```
            current_start = i
```

```
        else:
```

```
            current_sum = current_sum + arr[i]
```

```
        # New best sum → reset list
```

```
        if current_sum > best_sum:
```

```
            best_sum = current_sum
```

```
            best_ranges = [(current_start, i)]
```

```
        # Equal best sum → add another range
```

```
        elif current_sum == best_sum:
```

```
            best_ranges.append((current_start, i))
```

```

    i += 1

return best_sum, best_ranges

# =====
# MAIN PROGRAM FLOW
# =====

# Generate random array (max 10 elements, mix of ints + floats)
size = random.randint(5, 10)

arr = []
i = 0
while i < size:
    if random.random() < 0.5:
        num = random.randint(-10, 10)    # integer
    else:
        num = round(random.uniform(-10, 10), 2) # float
    arr.append(num)
    i += 1

print("Random array:", arr)

best_sum, ranges = kadane(arr)

print("\nMaximum subarray sum:", best_sum)
print("Number of max subarrays:", len(ranges))

idx = 0
while idx < len(ranges):
    start, end = ranges[idx]
    print(f"\nSubarray {idx+1} Start Index: {start}: ")
    j = start
    while j <= end:
        print(arr[j], end=" ")
        j += 1
    idx += 1

```

[pseudocode]

```

FUNCTION kadane(arr)
    n <- LENGTH OF arr

    current_sum <- arr[0]
    best_sum <- arr[0]

```

```

current_start <- 0
best_ranges <- LIST OF [TUPLE(0, 0)]

i <- 1

WHILE i < n DO

  IF current_sum + arr[i] < arr[i] THEN
    current_sum <- arr[i]
    current_start <- i
  ELSE
    current_sum <- current_sum + arr[i]
  ENDIF

  IF current_sum > best_sum THEN
    best_sum <- current_sum
    best_ranges <- LIST OF [TUPLE(current_start, i)]

  ELIF current_sum == best_sum THEN
    APPEND TUPLE(current_start, i) TO best_ranges
  ENDIF

  i <- i + 1
ENDWHILE

RETURN TUPLE(best_sum, best_ranges)
ENDFUNCTION

```

```

# =====
# MAIN PROGRAM FLOW — KADANE
# =====

```

```

size <- RANDOM_INT(5, 10)

arr <- EMPTY LIST
i <- 0

WHILE i < size DO
  rand_val <- RANDOM_FLOAT(0, 1)

  IF rand_val < 0.5 THEN
    num <- RANDOM_INT(-10, 10)
  ELSE
    num <- ROUND(RANDOM_FLOAT(-10, 10), 2)
  ENDIF

  APPEND num TO arr

```

```

    i <- i + 1
ENDWHILE

OUTPUT STRING(Random array: {arr})

result <- kadane(arr)
best_sum <- result[0]
ranges <- result[1]

OUTPUT STRING(\nMaximum subarray sum: {best_sum})
OUTPUT STRING(Number of max subarrays: {LENGTH OF ranges})

idx <- 0

WHILE idx < LENGTH OF ranges DO
    start <- ranges[idx][0]
    end <- ranges[idx][1]

    OUTPUT STRING(\nSubarray {idx + 1} Start Index: {start}: )

    j <- start
    WHILE j <= end DO
        OUTPUT STRING({arr[j]},)
        j <- j + 1
    ENDWHILE

    idx <- idx + 1
ENDWHILE

```

45. Create instructions for adding 2 matrices of equal size

[python code]

```

import random

print("Steps to Add 2 Equally Sized Matrices.")
# =====
# STEP 1: Choose matrix size
# =====

rows = random.randint(2, 5)
cols = random.randint(2, 5)

print("STEP 1: Matrix size chosen:", rows, "rows x", cols, "columns\n")

# =====
# STEP 2: Generate Matrix A
# =====

```

```
print("STEP 2: Generating Matrix A...\n")
```

```
A = []
r = 0
while r < rows:
    row_list = []
    c = 0
    while c < cols:
        if random.random() < 0.5:
            num = random.randint(-10, 10)
        else:
            num = round(random.uniform(-10, 10), 2)
        row_list.append(num)
        c += 1
    A.append(row_list)
    r += 1
```

```
print("Matrix A:\n")
```

```
r = 0
while r < rows:
    print(A[r])
    r += 1
print("\n")
```

```
# =====
# STEP 3: Generate Matrix B
# =====
```

```
print("STEP 3: Generating Matrix B...\n")
```

```
B = []
r = 0
while r < rows:
    row_list = []
    c = 0
    while c < cols:
        if random.random() < 0.5:
            num = random.randint(-10, 10)
        else:
            num = round(random.uniform(-10, 10), 2)
        row_list.append(num)
        c += 1
    B.append(row_list)
    r += 1
```

```
print("Matrix B:\n")
```

```
r = 0
while r < rows:
```

```

    print(B[r])
    r += 1
print("\n")

# =====
# STEP 4: Add matrices (rounded 2dp)
# =====

print("STEP 4: Adding Matrix A and Matrix B...\n")

Result = []
r = 0
print("Adding..")
while r < rows:
    row_list = []
    c = 0
    while c < cols:
        sum_value = A[r][c] + B[r][c]
        rounded_value = int(sum_value * 100) / 100.0

        row_list.append(rounded_value)

    print(f"A[{r}][{c}] + B[{r}][{c}] → {A[r][c]} + {B[r][c]} = {rounded_value}")

    c += 1
    Result.append(row_list)
    r += 1

# =====
# STEP 5: Print all matrices
# =====

print("\nSTEP 5: Final Matrix\n")

print("\nA + B =")
r = 0
while r < rows:
    print(Result[r])
    r += 1

```

[pseudocode]

OUTPUT STRING(Steps to Add 2 Equally Sized Matrices.)

```

# =====
# STEP 1: Choose matrix size
# =====

```

```
rows <- RANDOM_INT(2, 5)
cols <- RANDOM_INT(2, 5)
```

```
OUTPUT STRING(STEP 1: Matrix size chosen: {rows} rows x {cols} columns\n)
```

```
# =====
# STEP 2: Generate Matrix A
# =====
```

```
OUTPUT STRING(STEP 2: Generating Matrix A...\n)
```

```
A <- EMPTY LIST
r <- 0
```

```
WHILE r < rows DO
  row_list <- EMPTY LIST
  c <- 0
```

```
  WHILE c < cols DO
    rand_val <- RANDOM_FLOAT(0, 1)
```

```
    IF rand_val < 0.5 THEN
      num <- RANDOM_INT(-10, 10)
    ELSE
      num <- ROUND(RANDOM_FLOAT(-10, 10), 2)
    ENDIF
```

```
    APPEND num TO row_list
    c <- c + 1
  ENDWHILE
```

```
  APPEND row_list TO A
  r <- r + 1
ENDWHILE
```

```
OUTPUT STRING(Matrix A:\n)
```

```
r <- 0
WHILE r < rows DO
  OUTPUT STRING({A[r]})
  r <- r + 1
ENDWHILE
```

```
OUTPUT STRING(\n)
```

```
# =====
# STEP 3: Generate Matrix B
```



```
# =====
```

```
OUTPUT STRING(STEP 3: Generating Matrix B...\n)
```

```
B <- EMPTY LIST
```

```
r <- 0
```

```
WHILE r < rows DO
```

```
  row_list <- EMPTY LIST
```

```
  c <- 0
```

```
  WHILE c < cols DO
```

```
    rand_val <- RANDOM_FLOAT(0, 1)
```

```
    IF rand_val < 0.5 THEN
```

```
      num <- RANDOM_INT(-10, 10)
```

```
    ELSE
```

```
      num <- ROUND(RANDOM_FLOAT(-10, 10), 2)
```

```
    ENDIF
```

```
    APPEND num TO row_list
```

```
    c <- c + 1
```

```
  ENDWHILE
```

```
  APPEND row_list TO B
```

```
  r <- r + 1
```

```
ENDWHILE
```

```
OUTPUT STRING(Matrix B:\n)
```

```
r <- 0
```

```
WHILE r < rows DO
```

```
  OUTPUT STRING({B[r]})
```

```
  r <- r + 1
```

```
ENDWHILE
```

```
OUTPUT STRING(\n)
```

```
# =====
```

```
# STEP 4: Add matrices (rounded 2dp)
```

```
# =====
```

```
OUTPUT STRING(STEP 4: Adding Matrix A and Matrix B...\n)
```

```
Result <- EMPTY LIST
```

```
r <- 0
```

```
OUTPUT STRING(Adding..)
```

```

WHILE r < rows DO
    row_list <- EMPTY LIST
    c <- 0

    WHILE c < cols DO
        sum_value <- A[r][c] + B[r][c]

        rounded_value <- (INTEGER(sum_value * 100)) / 100.0

        APPEND rounded_value TO row_list

        OUTPUT STRING(A[{r}][{c}] + B[{r}][{c}] → {A[r][c]} + {B[r][c]} = {rounded_value})

        c <- c + 1
    ENDWHILE

    APPEND row_list TO Result
    r <- r + 1
ENDWHILE

```

```

# =====
# STEP 5: Print all matrices
# =====

```

```

OUTPUT STRING(\nSTEP 5: Final Matrix\n)

```

```

OUTPUT STRING(\nA + B =)

```

```

r <- 0
WHILE r < rows DO
    OUTPUT STRING({Result[r]})
    r <- r + 1
ENDWHILE

```

46. Develop a process for multiplying 2 matrices
[python code]

```

import random

```

```

print("Steps to Multiply 2 Matrices.")

```

```

# =====
# STEP 1: Choose matrix sizes
# =====

```

```

# A is rowsA x colsA

```

B is colsA x colsB (so multiplication is valid)

```
rowsA = random.randint(2, 4)
```

```
colsA = random.randint(2, 4)
```

```
colsB = random.randint(2, 4)
```

```
print("STEP 1: Matrix sizes chosen:")
```

```
print("Matrix A:", rowsA, "x", colsA)
```

```
print("Matrix B:", colsA, "x", colsB, "\n")
```

```
# =====
```

```
# STEP 2: Generate Matrix A
```

```
# =====
```

```
print("STEP 2: Generating Matrix A...\n")
```

```
A = []
```

```
r = 0
```

```
while r < rowsA:
```

```
    row_list = []
```

```
    c = 0
```

```
    while c < colsA:
```

```
        if random.random() < 0.5:
```

```
            num = random.randint(-10, 10)
```

```
        else:
```

```
            num = round(random.uniform(-10, 10), 2)
```

```
        row_list.append(num)
```

```
        c += 1
```

```
    A.append(row_list)
```

```
    r += 1
```

```
print("Matrix A:\n")
```

```
r = 0
```

```
while r < rowsA:
```

```
    print(A[r])
```

```
    r += 1
```

```
print("\n")
```

```
# =====
```

```
# STEP 3: Generate Matrix B
```

```
# =====
```

```
print("STEP 3: Generating Matrix B...\n")
```

```
B = []
```

```
r = 0
```

```
while r < colsA:
```

```
    row_list = []
```

```
    c = 0
```

```

while c < colsB:
    if random.random() < 0.5:
        num = random.randint(-10, 10)
    else:
        num = round(random.uniform(-10, 10), 2)
    row_list.append(num)
    c += 1
B.append(row_list)
r += 1

print("Matrix B:\n")
r = 0
while r < colsA:
    print(B[r])
    r += 1
print("\n")

# =====
# STEP 4: Multiply matrices
# =====

print("STEP 4: Multiplying Matrix A and Matrix B...\n")
print("Multiplying...\n")

# Result will be rowsA x colsB
Result = []

r = 0
while r < rowsA:
    row_list = []
    c = 0
    while c < colsB:

        # Compute dot product of A row r and B column c
        k = 0
        running_sum = 0
        print(f"Element Result[{r}][{c}]:")

        while k < colsA:
            product = A[r][k] * B[k][c]
            rounded_product = int(product * 100) / 100.0
            running_sum += rounded_product

            print(
                f" A[{r}][{k}] * B[{k}][{c}] → "
                f"{A[r][k]} * {B[k][c]} = {rounded_product}"
            )

            k += 1

```

```

# Final rounded value
final_value = int(running_sum * 100) / 100.0
row_list.append(final_value)

print(f" Sum for Result[{r}][{c}] = {final_value}\n")

c += 1

Result.append(row_list)
r += 1

# =====
# STEP 5: Print final matrix
# =====

print("\nSTEP 5: Final Matrix\n")

print("A × B =")
r = 0
while r < rowsA:
    print(Result[r])
    r += 1
[ pseudocode ]

```

OUTPUT STRING(Steps to Multiply 2 Matrices.)

```

# =====
# STEP 1: Choose matrix sizes
# =====

```

```

rowsA <- RANDOM INT(2, 4)
colsA <- RANDOM INT(2, 4)
colsB <- RANDOM INT(2, 4)

```

```

OUTPUT STRING(STEP 1: Matrix sizes chosen:)
OUTPUT STRING(Matrix A: {rowsA} x {colsA})
OUTPUT STRING(Matrix B: {colsA} x {colsB}\n)

```

```

# =====
# STEP 2: Generate Matrix A
# =====

```

OUTPUT STRING(STEP 2: Generating Matrix A...\n)

```

A <- EMPTY LIST
r <- 0

```

```

WHILE r < rowsA DO
  row_list <- EMPTY LIST
  c <- 0

  WHILE c < colsA DO
    rand_val <- RANDOM FLOAT(0, 1)

    IF rand_val < 0.5 THEN
      num <- RANDOM INT(-10, 10)
    ELSE
      num <- ROUND(RANDOM FLOAT(-10, 10), 2)
    ENDIF

    APPEND num TO row_list
    c <- c + 1
  ENDWHILE

  APPEND row_list TO A
  r <- r + 1
ENDWHILE

```

```

OUTPUT STRING(Matrix A:\n)

```

```

r <- 0
WHILE r < rowsA DO
  OUTPUT STRING({A[r]})
  r <- r + 1
ENDWHILE

```

```

OUTPUT STRING(\n)

```

```

# =====
# STEP 3: Generate Matrix B
# =====

```

```

OUTPUT STRING(STEP 3: Generating Matrix B...\n)

```

```

B <- EMPTY LIST
r <- 0

```

```

WHILE r < colsA DO
  row_list <- EMPTY LIST
  c <- 0

  WHILE c < colsB DO
    rand_val <- RANDOM FLOAT(0, 1)

    IF rand_val < 0.5 THEN

```

```

        num <- RANDOM INT(-10, 10)
    ELSE
        num <- ROUND(RANDOM FLOAT(-10, 10), 2)
    ENDIF

    APPEND num TO row_list
    c <- c + 1
ENDWHILE

APPEND row_list TO B
r <- r + 1
ENDWHILE

OUTPUT STRING(Matrix B:\n)

r <- 0
WHILE r < colsA DO
    OUTPUT STRING({B[r]})
    r <- r + 1
ENDWHILE

OUTPUT STRING(\n)

# =====
# STEP 4: Multiply matrices
# =====

OUTPUT STRING(STEP 4: Multiplying Matrix A and Matrix B...\n)
OUTPUT STRING(Multiplying...\n)

Result <- EMPTY LIST
r <- 0

WHILE r < rowsA DO
    row_list <- EMPTY LIST
    c <- 0

    WHILE c < colsB DO
        k <- 0
        running_sum <- 0

        OUTPUT STRING(Element Result[{r}][{c}]:)

        WHILE k < colsA DO
            product <- A[r][k] * B[k][c]
            rounded_product <- (INTEGER(product * 100)) / 100.0
            running_sum <- running_sum + rounded_product

```

```

    OUTPUT STRING( A[{r}][{k}] * B[{k}][{c}] → {A[r][k]} * {B[k][c]} = {rounded_product})

    k <- k + 1
ENDWHILE

final_value <- (INTEGER(running_sum * 100)) / 100.0
APPEND final_value TO row_list

OUTPUT STRING( Sum for Result[{r}][{c}] = {final_value}\n)

    c <- c + 1
ENDWHILE

APPEND row_list TO Result
    r <- r + 1
ENDWHILE

# =====
# STEP 5: Print final matrix
# =====

OUTPUT STRING(\nSTEP 5: Final Matrix\n)
OUTPUT STRING(A × B =)

r <- 0
WHILE r < rowsA DO
    OUTPUT STRING({Result[r]})
    r <- r + 1
ENDWHILE

```

47. Create an algorithm that updates inventory quantities after purchases and deliveries
[python code]

```

import random

print("Inventory System\n")

# =====
# INVENTORY CLASS
# =====

class Inventory:

    def __init__(self, items, qty):
        self.items = items
        self.qty = qty

# -----

```



```

# Display Method
# -----
def display(self):
    print("Current Inventory:")
    i = 0
    while i < len(self.items):
        print(self.items[i], "→", self.qty[i])
        i += 1
    print("\n")

# -----
# Getter Method
# -----
def get_quantity(self, item_name):
    i = 0
    while i < len(self.items):
        if self.items[i] == item_name:
            return self.qty[i]
        i += 1
    return None

# -----
# Setter Method
# -----
def set_quantity(self, item_name, new_qty):
    i = 0
    while i < len(self.items):
        if self.items[i] == item_name:
            self.qty[i] = new_qty
        i += 1

# -----
# Remove Items With Zero Stock
# -----
def remove_zero_stock(self):

    i = 0
    while i < len(self.items):
        if self.qty[i] == 0:
            print(f"REMOVED: {self.items[i]} (stock reached zero)")

            # Remove item and quantity
            j = i
            while j < len(self.items) - 1:
                self.items[j] = self.items[j + 1]
                self.qty[j] = self.qty[j + 1]
                j += 1

            # Remove last element

```

```
self.items.pop()
self.qty.pop()
```

```
# Do NOT increment i — list has shifted
else:
    i += 1
```

```
# -----
```

```
# Update Purchases
```

```
# -----
```

```
def update_purchases(self, purchase_items, purchase_qty):
    print("\nUpdating Inventory After Purchases...\nCustomers")
```

```
i = 0
```

```
while i < len(purchase_items):
```

```
    item = purchase_items[i]
```

```
    amount = purchase_qty[i]
```

```
j = 0
```

```
while j < len(self.items):
```

```
    if self.items[j] == item:
```

```
        new_qty = self.qty[j] - amount
```

```
        if new_qty < 0:
```

```
            new_qty = 0
```

```
        self.qty[j] = new_qty
```

```
        print(f"Purchased: {amount} {item} → New Qty = {new_qty}")
```

```
    j += 1
```

```
i += 1
```

```
print("\n")
```

```
self.display()
```

```
self.remove_zero_stock()
```

```
# -----
```

```
# Update Deliveries
```

```
# -----
```

```
def update_deliveries(self, delivery_items, delivery_qty):
    print("\nUpdating Inventory After Deliveries...\nCompanies")
```

```
i = 0
```

```
while i < len(delivery_items):
```

```
    item = delivery_items[i]
```

```
    amount = delivery_qty[i]
```

```
j = 0
```

```
while j < len(self.items):
```

```
    if self.items[j] == item:
```

```
        new_qty = self.qty[j] + amount
```

```

        self.qty[j] = new_qty
        print(f"Deliverered: {amount} {item} → New Qty = {new_qty}")
        j += 1
        i += 1
    print("\n")
    self.display()
    self.remove_zero_stock()

```

```

# =====
# STEP 1: Generate Random Inventory
# =====

```

```

item_names = ["Apples", "Bananas", "Oranges", "Pears", "Grapes"]
item_qty = []

```

```

i = 0
while i < len(item_names):
    qty = random.randint(10, 100)
    item_qty.append(qty)
    i += 1

```

```

inventory = Inventory(item_names, item_qty)

```

```

print("Random Inventory Generated\n")

```

```

# =====
# STEP 2: Display Current Inventory
# =====
inventory.display()

```

```

# =====
# STEP 3: Generate Random Purchases
# =====

```

```

purchase_items = ["Apples", "Grapes"]
purchase_qty = [random.randint(1, 10), random.randint(1, 10)]

```

```

print("Random Purchases Generated:")
i = 0
while i < len(purchase_items):
    print(purchase_items[i], "→", purchase_qty[i])
    i += 1

```

```

inventory.update_purchases(purchase_items, purchase_qty)

```

```

# =====
# STEP 4: Generate Random Deliveries
# =====

```

```

delivery_items = ["Bananas", "Oranges"]
delivery_qty = [random.randint(5, 20), random.randint(5, 20)]

print("Random Deliveries Generated:")
i = 0
while i < len(delivery_items):
    print(delivery_items[i], "→", delivery_qty[i])
    i += 1

inventory.update_deliveries(delivery_items, delivery_qty)

```

[pseudocode]

```

OUTPUT STRING(Inventory System\n)

```

```

# =====
# INVENTORY CLASS
# =====

```

```

CLASS Inventory

```

```

    CONSTRUCTOR ACCEPTS(self, items, qty)
        PROPERTIES(
            self.items <- items,
            self.qty <- qty
        )
    ENDCONSTRUCTOR

```

```

    FUNCTION display()
        OUTPUT STRING(Current Inventory:)
        i <- 0

        WHILE i < LENGTH OF self.items DO
            OUTPUT STRING({self.items[i]} → {self.qty[i]})
            i <- i + 1
        ENDWHILE

```

```

        OUTPUT STRING(\n)
    ENDFUNCTION

```

```

    FUNCTION get_quantity(item_name)
        i <- 0

        WHILE i < LENGTH OF self.items DO
            IF self.items[i] == item_name THEN

```

```

        RETURN self.qty[i]
    ENDIF
    i <- i + 1
ENDWHILE

RETURN None
ENDFUNCTION

```

```

FUNCTION set_quantity(item_name, new_qty)
    i <- 0

    WHILE i < LENGTH OF self.items DO
        IF self.items[i] == item_name THEN
            self.qty[i] <- new_qty
        ENDIF
        i <- i + 1
    ENDWHILE
ENDFUNCTION

```

```

FUNCTION remove_zero_stock()
    i <- 0

    WHILE i < LENGTH OF self.items DO
        IF self.qty[i] == 0 THEN
            OUTPUT STRING(REMOVED: {self.items[i]} (stock reached zero))

            j <- i
            WHILE j < LENGTH OF self.items - 1 DO
                self.items[j] <- self.items[j + 1]
                self.qty[j] <- self.qty[j + 1]
                j <- j + 1
            ENDWHILE

            REMOVE LAST ITEM FROM self.items
            REMOVE LAST ITEM FROM self.qty

        ELSE
            i <- i + 1
        ENDIF
    ENDWHILE
ENDFUNCTION

```

```

FUNCTION update_purchases(purchase_items, purchase_qty)
    OUTPUT STRING(\nUpdating Inventory After Purchases...\nCustomers)

    i <- 0

```

```

WHILE i < LENGTH OF purchase_items DO
  item <- purchase_items[i]
  amount <- purchase_qty[i]

  j <- 0
  WHILE j < LENGTH OF self.items DO
    IF self.items[j] == item THEN
      new_qty <- self.qty[j] - amount

      IF new_qty < 0 THEN
        new_qty <- 0
      ENDIF

      self.qty[j] <- new_qty
      OUTPUT STRING(Purchased: {amount} {item} → New Qty = {new_qty})
    ENDIF

    j <- j + 1
  ENDWHILE

  i <- i + 1
ENDWHILE

OUTPUT STRING(\n)
self.display()
self.remove_zero_stock()
ENDFUNCTION

```

```

FUNCTION update_deliveries(delivery_items, delivery_qty)
  OUTPUT STRING(\nUpdating Inventory After Deliveries...\nCompanies)

  i <- 0
  WHILE i < LENGTH OF delivery_items DO
    item <- delivery_items[i]
    amount <- delivery_qty[i]

    j <- 0
    WHILE j < LENGTH OF self.items DO
      IF self.items[j] == item THEN
        new_qty <- self.qty[j] + amount
        self.qty[j] <- new_qty
        OUTPUT STRING(Delivered: {amount} {item} → New Qty = {new_qty})
      ENDIF

      j <- j + 1
    ENDWHILE

    i <- i + 1
  ENDWHILE

```

```

ENDWHILE

OUTPUT STRING(\n)
self.display()
self.remove_zero_stock()
ENDFUNCTION

ENDCLASS


# =====
# STEP 1: Generate Random Inventory
# =====

item_names <- LIST OF ["Apples", "Bananas", "Oranges", "Pears", "Grapes"]
item_qty <- EMPTY LIST

i <- 0
WHILE i < LENGTH OF item_names DO
  qty <- RANDOM INT(10, 100)
  APPEND qty TO item_qty
  i <- i + 1
ENDWHILE

inventory <- NEW Inventory(item_names, item_qty)

OUTPUT STRING(Random Inventory Generated\n)


# =====
# STEP 2: Display Current Inventory
# =====

inventory.display()


# =====
# STEP 3: Generate Random Purchases
# =====

purchase_items <- LIST OF ["Apples", "Grapes"]
purchase_qty <- LIST OF [
  RANDOM INT(1, 10),
  RANDOM INT(1, 10)
]

OUTPUT STRING(Random Purchases Generated:)

```

```

i <- 0
WHILE i < LENGTH OF purchase_items DO
  OUTPUT STRING({purchase_items[i]} → {purchase_qty[i]})
  i <- i + 1
ENDWHILE

```

```

inventory.update_purchases(purchase_items, purchase_qty)

```

```

# =====
# STEP 4: Generate Random Deliveries
# =====

```

```

delivery_items <- LIST OF ["Bananas", "Oranges"]
delivery_qty <- LIST OF [
  RANDOM INT(5, 20),
  RANDOM INT(5, 20)
]

```

```

OUTPUT STRING(Random Deliveries Generated:)

```

```

i <- 0
WHILE i < LENGTH OF delivery_items DO
  OUTPUT STRING({delivery_items[i]} → {delivery_qty[i]})
  i <- i + 1
ENDWHILE

```

```

inventory.update_deliveries(delivery_items, delivery_qty)

```

48. Develop a recursive process that searches through nested folders to find a specific file type
[python code]

```

import os

print("Recursive File Search System\n")

# =====
# FUNCTION: Recursive Search
# =====

def search_files_recursive(folder_path, file_extension, results):
  print("Searching:", folder_path)

  try:
    items = os.listdir(folder_path)
  except:
    print("Cannot access:", folder_path)
  return

```



```

i = 0
while i < len(items):
    item = items[i]
    full_path = os.path.join(folder_path, item)

    # If it's a file
    if os.path.isfile(full_path):
        if item.endswith(file_extension):
            print("FOUND:", full_path)
            results.append(full_path)

    # If it's a folder → recurse
    elif os.path.isdir(full_path):
        search_files_recursive(full_path, file_extension, results)

    i += 1

# =====
# MAIN PROCESS
# =====

start_folder = "C:/example/start/path" # Change to your folder
target_extension = ".txt"             # Change to any file type

found_files = []

print("Starting recursive search...\n")
search_files_recursive(start_folder, target_extension, found_files)

print("\nSearch complete.\nFiles found:")
i = 0
while i < len(found_files):
    print(found_files[i])
    i += 1

[ pseudocode ]

OUTPUT STRING(Recursive File Search System\n)

# =====
# FUNCTION: Recursive Search
# =====

FUNCTION search_files_recursive(folder_path, file_extension, results)

    OUTPUT STRING(Searching: {folder_path})

```

```

TRY
    items <- LIST DIRECTORY CONTENTS(folder_path)
CATCH
    OUTPUT STRING(Cannot access: {folder_path})
    RETURN
ENDTRY

i <- 0

WHILE i < LENGTH OF items DO
    item <- items[i]
    full_path <- JOIN PATH(folder_path, item)

    IF ISFILE(full_path) == BOOLEAN(True) THEN
        IF ENDSWITH(item, file_extension) == BOOLEAN(True) THEN
            OUTPUT STRING(FOUND: {full_path})
            APPEND full_path TO results
        ENDIF

        ELIF ISDIRECTORY(full_path) == BOOLEAN(True) THEN
            search_files_recursive(full_path, file_extension, results)
        ENDIF

        i <- i + 1
    ENDWHILE

ENDFUNCTION

```

```

# =====
# MAIN PROCESS
# =====

```

```

start_folder <- STRING(C:/example/start/path)
target_extension <- STRING(.txt)

found_files <- EMPTY LIST

OUTPUT STRING(Starting recursive search...\n)

search_files_recursive(start_folder, target_extension, found_files)

OUTPUT STRING(\nSearch complete.\nFiles found:)

i <- 0
WHILE i < LENGTH OF found_files DO
    OUTPUT STRING({found_files[i]})
    i <- i + 1

```

ENDWHILE

49. Create a pathfinding solution that guides a user through a maze using graph traversal.

[python code]

```
import random

print("Auto Maze Generator and BFS Solver\n")

# =====
# SHUFFLE DIRECTIONS
# =====

def shuffle_dirs(dirs):
    for i in range(len(dirs)):
        j = random.randint(0, len(dirs)-1)
        dirs[i], dirs[j] = dirs[j], dirs[i]

# =====
# DFS MAZE CARVING (STEP SIZE = 2)
# =====

def carve_maze(maze, rows, cols):
    stack = []

    r, c = 1, 1
    maze[r][c] = " "

    stack.append((r, c))

    while stack:
        r, c = stack[-1]

        dirs = [(0,2), (0,-2), (2,0), (-2,0)]
        shuffle_dirs(dirs)

        carved = False

        for dr, dc in dirs:
            nr = r + dr
            nc = c + dc

            if 1 <= nr < rows-1 and 1 <= nc < cols-1:
                if maze[nr][nc] == "█":
                    maze[r + dr//2][c + dc//2] = " "
                    maze[nr][nc] = " "
                    stack.append((nr, nc))
                    carved = True
```

```
break
```

```
if not carved:  
    stack.pop()
```

```
# =====  
# BFS SOLVER  
# =====
```

```
def solve_maze_bfs(maze, rows, cols):
```

```
    start = end = None
```

```
    for r in range(rows):  
        for c in range(cols):  
            if maze[r][c] == "S":  
                start = (r, c)  
            elif maze[r][c] == "E":  
                end = (r, c)
```

```
    queue = [start]  
    visited = {start: None}
```

```
    i = 0  
    while i < len(queue):  
        r, c = queue[i]
```

```
        if (r, c) == end:  
            break
```

```
        for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:  
            nr = r + dr  
            nc = c + dc
```

```
        if 0 <= nr < rows and 0 <= nc < cols:  
            if maze[nr][nc] != "X" and (nr, nc) not in visited:  
                visited[(nr, nc)] = (r, c)  
                queue.append((nr, nc))
```

```
        i += 1
```

```
# Reconstruct path
```

```
path = []  
node = end  
while node is not None:  
    path.insert(0, node)  
    node = visited.get(node)
```

```
return path
```

```
# =====  
# MARK FINAL PATH  
# =====
```

```
def mark_path(maze, path):  
    for r, c in path:  
        if maze[r][c] not in ("S", "E"):  
            maze[r][c] = "•"
```

```
# =====  
# DISPLAY MAZE  
# =====
```

```
def display_maze(maze):  
    for row in maze:  
        print("".join(row))
```

```
# =====  
# PUBLIC FUNCTION: create_grid(rows, cols)  
# =====
```

```
def create_grid(rows, cols):
```

```
    # Carving grid must be odd  
    carve_rows = rows if rows % 2 == 1 else rows - 1  
    carve_cols = cols if cols % 2 == 1 else cols - 1
```

```
    # Build full maze  
    maze = [["█" for _ in range(cols)] for _ in range(rows)]
```

```
    # Build carving grid  
    carve = [["█" for _ in range(carve_cols)] for _ in range(carve_rows)]
```

```
    # Carve perfect maze  
    carve_maze(carve, carve_rows, carve_cols)
```

```
    # Copy carved region into full maze  
    for r in range(carve_rows):  
        for c in range(carve_cols):  
            maze[r][c] = carve[r][c]
```

```
    # Ensure S and E are placed on carved cells  
    maze[1][1] = "S"  
    maze[carve_rows-2][carve_cols-2] = "E"
```

```
# Solve with BFS
path = solve_maze_bfs(maze, rows, cols)

# Mark final path
mark_path(maze, path)

# Display
display_maze(maze)

return maze
```

```
create_grid(11, 20)
```

[pseudocode]

```
OUTPUT STRING(Auto Maze Generator and BFS Solver\n)
```

```
# =====
```

```
# SHUFFLE DIRECTIONS
```

```
# =====
```

```
FUNCTION shuffle_dirs(dirs)
```

```
  i <- 0
```

```
  WHILE i < LENGTH OF dirs DO
```

```
    j <- RANDOM INT(0, LENGTH OF dirs - 1)
```

```
    temp <- dirs[i]
```

```
    dirs[i] <- dirs[j]
```

```
    dirs[j] <- temp
```

```
  i <- i + 1
```

```
  ENDWHILE
```

```
ENDFUNCTION
```

```
# =====
```

```
# DFS MAZE CARVING (STEP SIZE = 2)
```

```
# =====
```

```
FUNCTION carve_maze(maze, rows, cols)
```

```
    stack <- EMPTY LIST
```

```
    r <- 1
```

```
    c <- 1
```

```
    maze[r][c] <- STRING( )
```

```
    APPEND TUPLE(r, c) TO stack
```

```
    WHILE LENGTH OF stack > 0 DO
```

```
        r <- stack[LENGTH OF stack - 1][0]
```

```
        c <- stack[LENGTH OF stack - 1][1]
```

```
        dirs <- LIST OF [
```

```
            TUPLE(0, 2),
```

```
            TUPLE(0, -2),
```

```
            TUPLE(2, 0),
```

```
            TUPLE(-2, 0)
```

```
        ]
```

```
        shuffle_dirs(dirs)
```

```
        carved <- BOOLEAN(False)
```

```
i <- 0
```

```
WHILE i < LENGTH OF dirs DO
```

```
  dr <- dirs[i][0]
```

```
  dc <- dirs[i][1]
```

```
  nr <- r + dr
```

```
  nc <- c + dc
```

```
  IF 1 <= nr AND nr < rows - 1 AND 1 <= nc AND nc < cols - 1 THEN
```

```
    IF maze[nr][nc] == STRING() THEN
```

```
      maze[r + (dr INTEGER DIVISION 2)][c + (dc INTEGER DIVISION 2)] <- STRING( )
```

```
      maze[nr][nc] <- STRING( )
```

```
      APPEND TUPLE(nr, nc) TO stack
```

```
      carved <- BOOLEAN(True)
```

```
      BREAK
```

```
    ENDIF
```

```
  ENDIF
```

```
  i <- i + 1
```

```
ENDWHILE
```

```
IF carved == BOOLEAN(False) THEN
```

```
  REMOVE LAST ITEM FROM stack
```

```
ENDIF
```

```
ENDWHILE
```

```
ENDFUNCTION
```



```
# =====
```

```
# BFS SOLVER
```

```
# =====
```

```
FUNCTION solve_maze_bfs(maze, rows, cols)
```

```
    start <- None
```

```
    end <- None
```

```
    r <- 0
```

```
    WHILE r < rows DO
```

```
        c <- 0
```

```
        WHILE c < cols DO
```

```
            IF maze[r][c] == STRING(S) THEN
```

```
                start <- TUPLE(r, c)
```

```
            ELIF maze[r][c] == STRING(E) THEN
```

```
                end <- TUPLE(r, c)
```

```
            ENDIF
```

```
            c <- c + 1
```

```
        ENDWHILE
```

```
        r <- r + 1
```

```
    ENDWHILE
```

```
    queue <- LIST OF [start]
```

```
    visited <- EMPTY DICTIONARY
```

```
    visited([start]) <- None
```

```
    i <- 0
```

```
    WHILE i < LENGTH OF queue DO
```

```
        r <- queue[i][0]
```

```
        c <- queue[i][1]
```

```
IF TUPLE(r, c) == end THEN
```

```
    BREAK
```

```
ENDIF
```

```
dirs <- LIST OF [
```

```
    TUPLE(1, 0),
```

```
    TUPLE(-1, 0),
```

```
    TUPLE(0, 1),
```

```
    TUPLE(0, -1)
```

```
]
```

```
j <- 0
```

```
WHILE j < LENGTH OF dirs DO
```

```
    dr <- dirs[j][0]
```

```
    dc <- dirs[j][1]
```

```
    nr <- r + dr
```

```
    nc <- c + dc
```

```
IF 0 <= nr AND nr < rows AND 0 <= nc AND nc < cols THEN
```

```
    IF maze[nr][nc] != STRING() THEN
```

```
        IF visited~GET([TUPLE(nr, nc)]) == None THEN
```

```
            visited([TUPLE(nr, nc)]) <- TUPLE(r, c)
```

```
            APPEND TUPLE(nr, nc) TO queue
```

```
        ENDIF
```

```
    ENDIF
```

```
ENDIF
```

```
j <- j + 1
```

ENDWHILE

i <- i + 1

ENDWHILE

path <- EMPTY LIST

node <- end

WHILE node != None DO

INSERT node AT FRONT OF path

node <- visited~GET([node])

ENDWHILE

RETURN path

ENDFUNCTION

=====

MARK FINAL PATH

=====

FUNCTION mark_path(maze, path)

i <- 0

WHILE i < LENGTH OF path DO

r <- path[i][0]

c <- path[i][1]

IF maze[r][c] != STRING(S) AND maze[r][c] != STRING(E) THEN

maze[r][c] <- STRING(•)

ENDIF

i <- i + 1

ENDWHILE

ENDFUNCTION

=====

DISPLAY MAZE

=====

FUNCTION display_maze(maze)

r <- 0

WHILE r < LENGTH OF maze DO

row_str <- STRING()

c <- 0

WHILE c < LENGTH OF maze[r] DO

row_str <- row_str + maze[r][c]

c <- c + 1

ENDWHILE

OUTPUT STRING({row_str})

r <- r + 1

ENDWHILE

ENDFUNCTION

```
# =====
```

```
# PUBLIC FUNCTION: create_grid(rows, cols)
```

```
# =====
```

```
FUNCTION create_grid(rows, cols)
```

```
    carve_rows <- rows
```

```
    IF rows MOD 2 == 0 THEN
```

```
        carve_rows <- rows - 1
```

```
    ENDIF
```

```
    carve_cols <- cols
```

```
    IF cols MOD 2 == 0 THEN
```

```
        carve_cols <- cols - 1
```

```
    ENDIF
```

```
    maze <- EMPTY LIST
```

```
    r <- 0
```

```
    WHILE r < rows DO
```

```
        row <- EMPTY LIST
```

```
        c <- 0
```

```
        WHILE c < cols DO
```

```
            APPEND STRING() TO row
```

```
            c <- c + 1
```

```
        ENDWHILE
```

```
        APPEND row TO maze
```

```
        r <- r + 1
```

```
    ENDWHILE
```

```
    carve <- EMPTY LIST
```

```
r <- 0
```

```
WHILE r < carve_rows DO
```

```
  row <- EMPTY LIST
```

```
  c <- 0
```

```
  WHILE c < carve_cols DO
```

```
    APPEND STRING() TO row
```

```
    c <- c + 1
```

```
  ENDWHILE
```

```
  APPEND row TO carve
```

```
  r <- r + 1
```

```
ENDWHILE
```

```
carve_maze(carve, carve_rows, carve_cols)
```

```
r <- 0
```

```
WHILE r < carve_rows DO
```

```
  c <- 0
```

```
  WHILE c < carve_cols DO
```

```
    maze[r][c] <- carve[r][c]
```

```
    c <- c + 1
```

```
  ENDWHILE
```

```
  r <- r + 1
```

```
ENDWHILE
```

```
maze[1][1] <- STRING(S)
```

```
maze[carve_rows - 2][carve_cols - 2] <- STRING(E)
```

```
path <- solve_maze_bfs(maze, rows, cols)
```

```
mark_path(maze, path)
```

```
display_maze(maze)
```

```
RETURN maze
```

```
ENDFUNCTION
```

```
create_grid(11, 20)
```

50. Develop an algorithm enabling autonomous drones to coordinate movement while avoiding collisions.

[python code]

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
```

```
# =====
# CONFIG CLASS
# =====

class Config:
    SPACE_SIZE = 400
    NUM_DRONES = 40
    NUM_OBSTACLES = 15

    MAX_SPEED = 2
    NEIGHBOUR_RADIUS = 20
    SEPARATION_RADIUS = 14

    COHESION_WEIGHT = 0.02
    ALIGNMENT_WEIGHT = 0.05
    SEPARATION_WEIGHT = 0.45
    GOAL_WEIGHT = 0.1

    OBSTACLE_RADIUS = 8
    OBSTACLE_REPULSION = 0.25
    OBSTACLE_EDGE_MARGIN = 15

    BOUNDARY_FORCE = 0.15
    DT = 1.0

    TARGET_RADIUS = 10.0
```

```

# =====
# ENVIRONMENT CLASS
# =====
class Environment:
    def __init__(self, config: Config):
        self.cfg = config

    def random_point(self):
        return np.array([
            np.random.uniform(0, self.cfg.SPACE_SIZE),
            np.random.uniform(0, self.cfg.SPACE_SIZE)
        ], dtype=float)

    def generate_safe_point(self, avoid_point, obstacles):
        while True:
            p = self.random_point()

            if p[0] < 20 or p[0] > self.cfg.SPACE_SIZE - 20:
                continue
            if p[1] < 20 or p[1] > self.cfg.SPACE_SIZE - 20:
                continue

            too_close = False
            for obs in obstacles:
                if np.linalg.norm(p - obs) < self.cfg.OBSTACLE_RADIUS * 4:
                    too_close = True
                    break
            if too_close:
                continue

            if np.linalg.norm(p - avoid_point) < 30:
                continue

            return p

    def generate_obstacles(self, point_a, min_dist=20):
        obstacles = []

        while len(obstacles) < self.cfg.NUM_OBSTACLES:
            obs = self.random_point()

            if not (self.cfg.OBSTACLE_EDGE_MARGIN < obs[0] < self.cfg.SPACE_SIZE - self.cfg.OBSTACLE_EDGE_MARGIN):
                continue
            if not (self.cfg.OBSTACLE_EDGE_MARGIN < obs[1] < self.cfg.SPACE_SIZE - self.cfg.OBSTACLE_EDGE_MARGIN):
                continue

            if np.linalg.norm(obs - point_a) < min_dist:
                continue

```



```

too_close = False
for o in obstacles:
    if np.linalg.norm(obs - o) < self.cfg.OBSTACLE_RADIUS * 3:
        too_close = True
        break
if too_close:
    continue

```

```

obstacles.append(obs)

```

```

return obstacles

```

```

# =====

```

```

# TARGET POINT CLASS

```

```

# =====

```

```

class TargetPoint:

```

```

    def __init__(self, position, radius):
        self.position = position.astype(float)
        self.radius = radius

```

```

    def reached(self, drone_positions):
        distances = np.linalg.norm(drone_positions - self.position, axis=1)
        return distances.min() < self.radius

```

```

# =====

```

```

# DRONE CLASS

```

```

# =====

```

```

class Drone:

```

```

    def __init__(self, position, velocity, cfg: Config):
        self.position = position.astype(float)
        self.velocity = velocity.astype(float)
        self.cfg = cfg

```

```

    def update(self, acceleration):
        self.velocity += acceleration * self.cfg.DT

```

```

        speed = np.linalg.norm(self.velocity)
        if speed > self.cfg.MAX_SPEED:
            self.velocity = (self.velocity / speed) * self.cfg.MAX_SPEED

```

```

        self.position += self.velocity * self.cfg.DT

```

```

        for i in range(2):
            if self.position[i] < 0:
                self.position[i] = 0
                self.velocity[i] += self.cfg.BOUNDARY_FORCE
            elif self.position[i] > self.cfg.SPACE_SIZE:

```

```
self.position[i] = self.cfg.SPACE_SIZE
self.velocity[i] -= self.cfg.BOUNDARY_FORCE
```

```
# =====
```

```
# SWARM CLASS
```

```
# =====
```

```
class Swarm:
```

```
def __init__(self, num_drones, start_pos, cfg: Config):
    self.cfg = cfg
    self.drones = []
    for _ in range(num_drones):
        vel = (np.random.rand(2) - 0.5) * 2
        self.drones.append(Drone(start_pos.copy(), vel, cfg))
```

```
def step(self, target: TargetPoint, obstacles):
    positions = np.array([d.position for d in self.drones])
    velocities = np.array([d.velocity for d in self.drones])
```

```
    for i, drone in enumerate(self.drones):
```

```
        if np.linalg.norm(drone.position - target.position) < target.radius:
            acceleration = (target.position - drone.position) * self.cfg.GOAL_WEIGHT
            drone.update(acceleration)
            continue
```

```
        diff = positions - drone.position
        dist = np.linalg.norm(diff, axis=1)
```

```
        neighbours_mask = (dist > 0) & (dist < self.cfg.NEIGHBOUR_RADIUS)
        neighbours_idx = np.where(neighbours_mask)[0]
```

```
        acceleration = np.zeros(2)
```

```
        if len(neighbours_idx) > 0:
            neighbour_positions = positions[neighbours_idx]
            neighbour_velocities = velocities[neighbours_idx]
```

```
            cohesion = (neighbour_positions.mean(axis=0) - drone.position) * self.cfg.COHESION_WEIGHT
            alignment = (neighbour_velocities.mean(axis=0) - drone.velocity) * self.cfg.ALIGNMENT_WEIGHT
```

```
            close_idx = np.where(dist < self.cfg.SEPARATION_RADIUS)[0]
            separation = np.zeros(2)
            if len(close_idx) > 0:
```

```
                separation = -(positions[close_idx] - drone.position).sum(axis=0) * self.cfg.SEPARATION_WEIGHT
```

```
            acceleration += cohesion + alignment + separation
```

```
        acceleration += (target.position - drone.position) * self.cfg.GOAL_WEIGHT
```

```

for obstacle in obstacles:
    to_obs = drone.position - obstacle
    dist_obs = np.linalg.norm(to_obs)

    radial = to_obs / (dist_obs + 1e-6)
    tangent = np.array([-radial[1], radial[0]])

    dir_to_goal = target.position - drone.position
    if np.dot(tangent, dir_to_goal) < 0:
        tangent = -tangent

    HARD_LOCK_RADIUS = self.cfg.OBSTACLE_RADIUS * 3.0
    SOFT_LOCK_RADIUS = self.cfg.OBSTACLE_RADIUS * 5.0
    MIN_SAFE = self.cfg.OBSTACLE_RADIUS * 1.8

    if dist_obs < MIN_SAFE:
        acceleration = radial * (self.cfg.OBSTACLE_REPULSION * 4.0)
        drone.velocity = -drone.velocity * 0.5
        continue

    if dist_obs < HARD_LOCK_RADIUS:
        acceleration = tangent * (self.cfg.OBSTACLE_REPULSION * 2.0)
        continue

    if dist_obs < SOFT_LOCK_RADIUS:
        avoid = (
            radial * (self.cfg.OBSTACLE_REPULSION * 0.4) +
            tangent * (self.cfg.OBSTACLE_REPULSION * 1.2)
        )
        acceleration += avoid

drone.update(acceleration)

```

```

# =====
# MISSION CONTROLLER
# =====

```

```

class MissionController:
    def __init__(self, env: Environment, cfg: Config):
        self.env = env
        self.cfg = cfg

        self.point_a = TargetPoint(env.random_point(), cfg.TARGET_RADIUS)
        self.obstacles = env.generate_obstacles(self.point_a.position)
        self.point_b = TargetPoint(env.generate_safe_point(self.point_a.position, self.obstacles), cfg.TARGET_RADIUS)

    def update_targets(self, swarm_positions):
        if self.point_b.reached(swarm_positions):

```

```

self.point_a = self.point_b
new_pos = self.env.generate_safe_point(self.point_a.position, self.obstacles)
self.point_b = TargetPoint(new_pos, self.cfg.TARGET_RADIUS)

```

```

# =====
# VISUALISATION CLASS
# =====

```

```

class Visualizer:

```

```

    def __init__(self, mission: MissionController, swarm: Swarm, cfg: Config):

```

```

        self.mission = mission

```

```

        self.swarm = swarm

```

```

        self.cfg = cfg

```

```

        self.fig, self.ax = plt.subplots(figsize=(6, 6))

```

```

        self.ax.set_xlim(0, cfg.SPACE_SIZE)

```

```

        self.ax.set_ylim(0, cfg.SPACE_SIZE)

```

```

        self.scat = self.ax.scatter([], [], s=20, c='blue')

```

```

        self.point_a_marker = self.ax.scatter(mission.point_a.position[0], mission.point_a.position[1], c='green', s=80)

```

```

        self.point_b_marker = self.ax.scatter(mission.point_b.position[0], mission.point_b.position[1], c='purple', s=80)

```

```

        for obs in mission.obstacles:

```

```

            circle = plt.Circle(obs, cfg.OBSTACLE_RADIUS, color='red')

```

```

            self.ax.add_patch(circle)

```

```

    def init(self):

```

```

        self.scat.set_offsets(np.empty((0, 2)))

```

```

        return self.scat, self.point_a_marker, self.point_b_marker

```

```

    def update(self, frame):

```

```

        self.swarm.step(self.mission.point_b, self.mission.obstacles)

```

```

        positions = np.array([d.position for d in self.swarm.drones])

```

```

        self.scat.set_offsets(positions)

```

```

        self.mission.update_targets(positions)

```

```

        self.point_a_marker.set_offsets(self.mission.point_a.position)

```

```

        self.point_b_marker.set_offsets(self.mission.point_b.position)

```

```

        return self.scat, self.point_a_marker, self.point_b_marker

```

```

    def animate(self):

```

```

        self.anim = FuncAnimation(

```

```

            self.fig,

```

```

            self.update,

```

```

            init_func=self.init,

```

```

            frames=2000,

```

```

            interval=40,

```

```
        blit=False
    )
    plt.show()
```

```
# =====
# SIMULATION CLASS
# =====
class Simulation:
    def __init__(self):
        self.cfg = Config()
        self.env = Environment(self.cfg)
        self.mission = MissionController(self.env, self.cfg)
        self.swarm = Swarm(self.cfg.NUM_DRONES, self.mission.point_a.position, self.cfg)
        self.visualizer = Visualizer(self.mission, self.swarm, self.cfg)

    def run(self):
        self.visualizer.animate()
```

```
# =====
# RUN SIMULATION
# =====
Simulation().run()
```

[pseudocode]

OUTPUT STRING(Auto Drone Swarm Simulation\n)

```
# =====
# CLASS: Config
# =====
```

CLASS Config

```
SPACE_SIZE <- 400
NUM_DRONES <- 40
NUM_OBSTACLES <- 15
```

```
MAX_SPEED <- 2
NEIGHBOUR_RADIUS <- 20
SEPARATION_RADIUS <- 14
```

```
COHESION_WEIGHT <- 0.02
ALIGNMENT_WEIGHT <- 0.05
SEPARATION_WEIGHT <- 0.45
GOAL_WEIGHT <- 0.1
```

```
OBSTACLE_RADIUS <- 8
OBSTACLE_REPULSION <- 0.25
OBSTACLE_EDGE_MARGIN <- 15
```

```
BOUNDARY_FORCE <- 0.15
DT <- 1.0
```

```
TARGET_RADIUS <- 10.0
```

```
ENDCLASS
```

```
# =====
# CLASS: Environment
# =====
```

```
CLASS Environment
```

```
CONSTRUCTOR ACCEPTS(self, config)
  PROPERTIES(self.cfg <- config)
ENDCONSTRUCTOR
```

```
FUNCTION random_point()
  x <- RANDOM FLOAT(0, self.cfg.SPACE_SIZE)
  y <- RANDOM FLOAT(0, self.cfg.SPACE_SIZE)
  RETURN LIST OF [x, y]
ENDFUNCTION
```

```
FUNCTION generate_safe_point(avoid_point, obstacles)
  LOOP FOREVER
    p <- self.random_point()

    IF p[0] < 20 OR p[0] > self.cfg.SPACE_SIZE - 20 THEN
      CONTINUE
    ENDIF

    IF p[1] < 20 OR p[1] > self.cfg.SPACE_SIZE - 20 THEN
      CONTINUE
    ENDIF

    too_close <- BOOLEAN(False)

    i <- 0
    WHILE i < LENGTH OF obstacles DO
      IF NORM(p - obstacles[i]) < self.cfg.OBSTACLE_RADIUS * 4 THEN
```

```

        too_close <- BOOLEAN(True)
        BREAK
    ENDIF
    i <- i + 1
ENDWHILE

IF too_close == BOOLEAN(True) THEN
    CONTINUE
ENDIF

IF NORM(p - avoid_point) < 30 THEN
    CONTINUE
ENDIF

RETURN p
ENDLOOP
ENDFUNCTION

FUNCTION generate_obstacles(point_a, min_dist)
    obstacles <- EMPTY LIST

    WHILE LENGTH OF obstacles < self.cfg.NUM_OBSTACLES DO
        obs <- self.random_point()

        IF obs[0] <= self.cfg.OBSTACLE_EDGE_MARGIN OR
            obs[0] >= self.cfg.SPACE_SIZE - self.cfg.OBSTACLE_EDGE_MARGIN THEN
            CONTINUE
        ENDIF

        IF obs[1] <= self.cfg.OBSTACLE_EDGE_MARGIN OR
            obs[1] >= self.cfg.SPACE_SIZE - self.cfg.OBSTACLE_EDGE_MARGIN THEN
            CONTINUE
        ENDIF

        IF NORM(obs - point_a) < min_dist THEN
            CONTINUE
        ENDIF

        too_close <- BOOLEAN(False)

        i <- 0
        WHILE i < LENGTH OF obstacles DO
            IF NORM(obs - obstacles[i]) < self.cfg.OBSTACLE_RADIUS * 3 THEN
                too_close <- BOOLEAN(True)
                BREAK
            ENDIF
            i <- i + 1
        ENDWHILE
    
```

```

        IF too_close == BOOLEAN(True) THEN
            CONTINUE
        ENDIF

        APPEND obs TO obstacles
    ENDWHILE

    RETURN obstacles
ENDFUNCTION

ENDCLASS

# =====
# CLASS: TargetPoint
# =====

CLASS TargetPoint

    CONSTRUCTOR ACCEPTS(self, position, radius)
    PROPERTIES(
        self.position <- position,
        self.radius <- radius
    )
ENDCONSTRUCTOR

    FUNCTION reached(drone_positions)
        distances <- NORM(drone_positions - self.position, AXIS=1)
        RETURN distances MINIMUM < self.radius
    ENDFUNCTION

ENDCLASS

# =====
# CLASS: Drone
# =====

CLASS Drone

    CONSTRUCTOR ACCEPTS(self, position, velocity, cfg)
    PROPERTIES(
        self.position <- position,
        self.velocity <- velocity,
        self.cfg <- cfg
    )

```



```
)  
ENDCONSTRUCTOR
```

```
FUNCTION update(acceleration)  
    self.velocity <- self.velocity + acceleration * self.cfg.DT  
  
    speed <- NORM(self.velocity)  
    IF speed > self.cfg.MAX_SPEED THEN  
        self.velocity <- (self.velocity / speed) * self.cfg.MAX_SPEED  
    ENDIF  
  
    self.position <- self.position + self.velocity * self.cfg.DT  
  
    i <- 0  
    WHILE i < 2 DO  
        IF self.position[i] < 0 THEN  
            self.position[i] <- 0  
            self.velocity[i] <- self.velocity[i] + self.cfg.BOUNDARY_FORCE  
  
            ELIF self.position[i] > self.cfg.SPACE_SIZE THEN  
                self.position[i] <- self.cfg.SPACE_SIZE  
                self.velocity[i] <- self.velocity[i] - self.cfg.BOUNDARY_FORCE  
            ENDIF  
  
            i <- i + 1  
        ENDWHILE  
    ENDFUNCTION
```

```
ENDCLASS
```

```
# =====  
# CLASS: Swarm  
# =====
```

```
CLASS Swarm
```

```
CONSTRUCTOR ACCEPTS(self, num_drones, start_pos, cfg)  
    PROPERTIES(self.cfg <- cfg)  
    self.drones <- EMPTY LIST  
  
    i <- 0  
    WHILE i < num_drones DO  
        vel <- LIST OF [  
            RANDOM FLOAT(-1, 1),  
            RANDOM FLOAT(-1, 1)  
        ]
```

```

    APPEND NEW Drone(start_pos, vel, cfg) TO self.drones
    i <- i + 1
ENDWHILE
ENDCONSTRUCTOR

```

```

FUNCTION step(target, obstacles)

```

```

    positions <- LIST OF SIZE EQUALS LENGTH OF self.drones FILLED WITH EMPTY LIST
    velocities <- LIST OF SIZE EQUALS LENGTH OF self.drones FILLED WITH EMPTY LIST

```

```

    i <- 0

```

```

    WHILE i < LENGTH OF self.drones DO
        positions[i] <- self.drones[i].position
        velocities[i] <- self.drones[i].velocity
        i <- i + 1
    ENDWHILE

```

```

    i <- 0

```

```

    WHILE i < LENGTH OF self.drones DO
        drone <- self.drones[i]

```

```

        IF NORM(drone.position - target.position) < target.radius THEN
            acceleration <- (target.position - drone.position) * self.cfg.GOAL_WEIGHT
            drone.update(acceleration)
            i <- i + 1
            CONTINUE
        ENDIF

```

```

        diff <- positions - drone.position
        dist <- NORM(diff, AXIS=1)

```

```

        neighbours_mask <- (dist > 0) AND (dist < self.cfg.NEIGHBOUR_RADIUS)
        neighbours_idx <- WHERE(neighbours_mask)

```

```

        acceleration <- LIST OF [0, 0]

```

```

        IF LENGTH OF neighbours_idx > 0 THEN
            neighbour_positions <- positions[neighbours_idx]
            neighbour_velocities <- velocities[neighbours_idx]

```

```

            cohesion <- (MEAN(neighbour_positions) - drone.position) * self.cfg.COHESSION_WEIGHT
            alignment <- (MEAN(neighbour_velocities) - drone.velocity) * self.cfg.ALIGNMENT_WEIGHT

```

```

            close_idx <- WHERE(dist < self.cfg.SEPARATION_RADIUS)
            separation <- LIST OF [0, 0]

```

```

            IF LENGTH OF close_idx > 0 THEN
                separation <- -(SUM(positions[close_idx] - drone.position)) * self.cfg.SEPARATION_WEIGHT
            ENDIF

```

```

    acceleration <- acceleration + cohesion + alignment + separation
ENDIF

acceleration <- acceleration + (target.position - drone.position) * self.cfg.GOAL_WEIGHT

j <- 0
WHILE j < LENGTH OF obstacles DO
    obstacle <- obstacles[j]

    to_obs <- drone.position - obstacle
    dist_obs <- NORM(to_obs)

    radial <- to_obs / (dist_obs + 0.000001)
    tangent <- LIST OF [-radial[1], radial[0]]

    dir_to_goal <- target.position - drone.position

    IF DOT(tangent, dir_to_goal) < 0 THEN
        tangent <- -tangent
    ENDIF

    HARD_LOCK <- self.cfg.OBSTACLE_RADIUS * 3
    SOFT_LOCK <- self.cfg.OBSTACLE_RADIUS * 5
    MIN_SAFE <- self.cfg.OBSTACLE_RADIUS * 1.8

    IF dist_obs < MIN_SAFE THEN
        acceleration <- radial * (self.cfg.OBSTACLE_REPULSION * 4)
        drone.velocity <- -drone.velocity * 0.5
        j <- j + 1
        CONTINUE
    ENDIF

    IF dist_obs < HARD_LOCK THEN
        acceleration <- tangent * (self.cfg.OBSTACLE_REPULSION * 2)
        j <- j + 1
        CONTINUE
    ENDIF

    IF dist_obs < SOFT_LOCK THEN
        avoid <- radial * (self.cfg.OBSTACLE_REPULSION * 0.4) +
            tangent * (self.cfg.OBSTACLE_REPULSION * 1.2)
        acceleration <- acceleration + avoid
    ENDIF

    j <- j + 1
ENDWHILE

drone.update(acceleration)

```

```

        i <- i + 1
    ENDWHILE
ENDFUNCTION

```

```

ENDCLASS

```

```

# =====
# CLASS: MissionController
# =====

```

```

CLASS MissionController

```

```

    CONSTRUCTOR ACCEPTS(self, env, cfg)
    PROPERTIES(self.env <- env, self.cfg <- cfg)

```

```

        self.point_a <- NEW TargetPoint(env.random_point(), cfg.TARGET_RADIUS)
        self.obstacles <- env.generate_obstacles(self.point_a.position)
        self.point_b <- NEW TargetPoint(env.generate_safe_point(self.point_a.position, self.obstacles),
cfg.TARGET_RADIUS)
    ENDCONSTRUCTOR

```

```

    FUNCTION update_targets(swarm_positions)
    IF self.point_b.reached(swarm_positions) THEN
        self.point_a <- self.point_b
        new_pos <- self.env.generate_safe_point(self.point_a.position, self.obstacles)
        self.point_b <- NEW TargetPoint(new_pos, self.cfg.TARGET_RADIUS)
    ENDIF
    ENDFUNCTION

```

```

ENDCLASS

```

```

# =====
# CLASS: Visualizer
# =====

```

```

CLASS Visualizer

```

```

    CONSTRUCTOR ACCEPTS(self, mission, swarm, cfg)
    PROPERTIES(
        self.mission <- mission,
        self.swarm <- swarm,
        self.cfg <- cfg
    )
    ENDCONSTRUCTOR

```

```
FUNCTION init()
  RETURN None
ENDFUNCTION
```

```
FUNCTION update(frame)
  self.swarm.step(self.mission.point_b, self.mission.obstacles)
  positions <- LIST OF [drone.position FOR EACH drone IN self.swarm.drones]

  self.mission.update_targets(positions)

  RETURN None
ENDFUNCTION
```

```
FUNCTION animate()
  OUTPUT STRING(Animation running...)
ENDFUNCTION
```

```
ENDCLASS
```

```
# =====
# CLASS: Simulation
# =====
```

```
CLASS Simulation
```

```
CONSTRUCTOR ACCEPTS(self)
  self.cfg <- NEW Config()
  self.env <- NEW Environment(self.cfg)
  self.mission <- NEW MissionController(self.env, self.cfg)
  self.swarm <- NEW Swarm(self.cfg.NUM_DRONES, self.mission.point_a.position, self.cfg)
  self.visualizer <- NEW Visualizer(self.mission, self.swarm, self.cfg)
ENDCONSTRUCTOR
```

```
FUNCTION run()
  self.visualizer.animate()
ENDFUNCTION
```

```
ENDCLASS
```

```
# =====
```

RUN SIMULATION

=====

NEW Simulation() RUN()

Custom Rules Used to generate pseudocode

All other rules align with cambridge teaching guide (Cambridge Assessment International Education, 2024)

1. Assignment & Operators

- Assignment: variable <- value
- Comparison: ==, !=, <, >, <=, >=
- Exponent: x EXP y
- Integer division: x INTEGER DIVISION y
- Break statement: BREAK

2. Data Types & Literals

- Typed lists: LIST OF [5, 2.3, STRING(hello), CHAR(a), BOOLEAN(True)]
- Empty values: EMPTY LIST, EMPTY DICTIONARY, EMPTY STRING, EMPTY SET
- String literal: STRING(text)
- Character literal: CHAR(x)
- Tuple literal: TUPLE(a, b)

3. Input & Output

- Output formatting: OUTPUT STRING(The value is {value})
- Input rule: var <- INPUT(STRING(prompt))

4. List & Collection Rules

- List operations:
 - APPEND x TO list
 - REMOVE LAST ITEM FROM list
 - REMOVE FIRST ITEM FROM list
 - list[index]
 - LENGTH OF list
- Join rule: JOIN(list, EMPTY STRING)
- 2D list creation (nested WHILE rule):

Code

```
grid <- EMPTY LIST
r <- 0
WHILE r < rows DO
  row <- EMPTY LIST
  c <- 0
  WHILE c < cols DO
    APPEND fill_value TO row
    c <- c + 1
  ENDWHILE
  APPEND row TO grid
  r <- r + 1
ENDWHILE
```

5. Class & Constructor Rules

- Class definition:

Code

```
CLASS Name
```

```
...
```

```
ENDCLASS
```

- Constructor rule:

Code

```
CONSTRUCTOR ACCEPTS(self, args)
```

```
  PROPERTIES(self.x <- value, ...)
```

```
ENDCONSTRUCTOR
```

6. Looping Rules

- IN RANGE loop:

Code

```
FOR i <- IN RANGE 0 TO n DO
```

```
...
```

```
ENDFOR
```

- Strict WHILE loop:

Code

```
WHILE condition DO
```

```
...
```

```
ENDWHILE
```

7. Random Rules

- Random integer: RANDOM INT(x, y)
- Random float: RANDOM FLOAT(x, y)
- Random boolean: RANDOM BOOLEAN()

8. Dictionary Rules

Dictionary SET

Code

```
visited([TUPLE(nr, nc)]) <- TUPLE(r, c)
```

Dictionary GET (final form)

Code

```
node <- visited~GET([node])
```

9. Error Handling

Code

```
TRY
```

```
...
```

```
CATCH
```

```
...
```

```
ENDTRY
```

10. Guided Links

All key concepts must support Guided Link formatting.

Examples:

- typed lists
- dictionary rules
- constructor rule

11. Structural Rules

- One continuous pseudocode block
- No Pythonisms
- Strict formatting consistency
- All operators and constructs must follow the dialect exactly

References

Cambridge Assessment International Education (2024) *2026 pseudocode guide for teachers* [PDF]. Available at: <https://www.cambridgeinternational.org/Images/697401-2026-pseudocode-guide-for-teachers.pdf> (Accessed: July 2026).